



AOS-CX 10.18.xxxx REST Guide

AOS-CX 10.18.xxxx REST Guide

Published: May 2026

© Copyright 2026 – Hewlett Packard Enterprise Development LP

Notices

The information provided here is subject to change without notice. Hewlett Packard Enterprise's products and services are covered only by the express warranty statements that come with them. This document does not constitute an additional warranty. Hewlett Packard Enterprise is not responsible for any technical or editorial errors or omissions in this document.

Confidential computer software. You must have a valid license from Hewlett Packard Enterprise to possess, use, or copy the software. In accordance with FAR 12.211 and 12.212, Commercial Computer Software, Computer Software Documentation, and Technical Data for Commercial Items are licensed to the U.S. Government under the vendor's standard commercial license.

Links to third-party websites will take you outside of the Hewlett Packard Enterprise website. Hewlett Packard Enterprise has no control over and is not responsible for the information outside the Hewlett Packard Enterprise website.

Open source code

This product includes code licensed under certain open source licenses which require source compliance. The corresponding source for these components is available upon request. This offer is valid to anyone in receipt of this information and shall expire three years following the date of the final distribution of this product version by Hewlett Packard Enterprise Company. To obtain such source code, please check if the code is available in the HPE Software Center at <https://myenterpriselicense.hpe.com/cwp-ui/software> but, if not, send a written request for specific software version and product for which you want the open source code. Along with the request, please send a check or money order in the amount of US \$10.00 to:

Hewlett Packard Enterprise Company

Attn: General Counsel

WW Corporate Headquarters

1701 E Mossy Oaks Rd, Spring, TX 77389

United States of America.



Legal disclaimer

The resource assets in this document may include abbreviated and/or legacy terminology for products. See [**www.arubanetworks.com**](http://www.arubanetworks.com) for current and complete product lines and names.

Table of contents

About this document.....	12
Applicable products.....	12
Latest version available online.....	13
Command syntax notation conventions.....	13
About the examples.....	14
Identifying switch ports and interfaces.....	15
Identifying modular switch components.....	17
Introduction to the AOS-CX REST API.....	18
REST API versions.....	18
Switches upgrading to AOS-CX 10.15.....	19
Compatibility.....	19
REST API access modes.....	22
Read-write access mode.....	22
Read-only access mode.....	23
REST API URI.....	23
Parts of a URI.....	23
URI path, including path parameters.....	23
URI prefix.....	24
Server URL.....	24
Path parameters.....	25
Query component.....	25
"Dot" notation for Network Analytics Engine URIs only.....	26
Resources.....	26
Resource collections and singletons.....	26
Collections.....	27
Subcollections.....	27
Singletons.....	28
Categories of resource attributes.....	28
Configuration attributes.....	28
Writable attributes.....	29
Status attributes.....	29
Statistics attributes.....	29
Attribute categories might vary.....	29
Enabling Access to the REST API.....	30

Setting the admin password.....	31
Showing the REST API access configuration.....	32
Disabling access to the REST API.....	32
HTTPS server commands.....	33
https-server authentication certificate	33
https-server authentication password	35
https-server max-user-sessions	36
https-server rest access-mode	37
https-server rest firmware-site-distribution	38
https-server session close all	39
https-server session-timeout	40
https-server vrf	41
show https-server	43
show https-server authentication	44
show https-server 1rest images	45
Accessing the AOS-CX REST API.....	46
Authenticating REST API sessions.....	47
User groups and access authorization.....	49
AOS-CX REST API Reference (UI).....	49
Accessing the REST API using the AOS-CX REST API Reference.....	50
Logging in and logging out using the AOS-CX REST API Reference.....	51
AOS-CX REST API Reference basics.....	51
AOS-CX REST API Reference home page.....	52
Write methods (POST, PUT, PATCH, and DELETE).....	59
Considerations when making configuration changes.....	59
Considerations for ports and interfaces.....	60
Hardware (system) interfaces.....	60
LAG interfaces.....	60
VLAN interfaces.....	61
Write methods (POST, PUT) supported in read-only mode.....	61
GET method usage and considerations.....	62
GET method parameters.....	62
Wildcard character support.....	63
Attributes parameter	64
Count parameter	65
Depth parameter	66
Filter parameter	70
Selector parameter	74

POST method usage and considerations.....	77
PUT method usage and considerations.....	78
PUT request body requirements.....	78
PUT behavior.....	79
Exceptions to the PUT strict replace behavior.....	79
Best practice for building the PUT request body.....	79
PATCH method usage and considerations.....	80
DELETE method usage and considerations.....	80
REST requests and accounting logs.....	81
AOS-CX REST API reference summary.....	81
Switch REST API access default.....	82
Switch REST API default access mode.....	82
Enabling access to the Web UI and REST API.....	82
Setting the REST API access mode to read-write.....	82
Showing the REST API access configuration.....	82
AOS-CX REST API Reference URL:.....	83
REST API versions and switch software versions.....	83
Getting REST API version information from a switch.....	83
Protocol.....	84
Port.....	84
Request and response body format.....	84
Session idle timeout.....	84
Session hard timeout.....	84
Authentication.....	84
HTTPS client sessions.....	85
VSX peer switch access.....	85
Polling with Event Log Subscriptions.....	85
Elements of a URI Request Filter.....	85
Event ID.....	86
Priority.....	86
Interval.....	87
Limit.....	87
Combining Filters	87
Event Log Subscription Response messages.....	89
Firmware Upgrade.....	91
Introduction.....	91
Limitations.....	91
Check Firmware Status.....	91

Upload firmware image.....	92
Remote Location.....	92
Parameters.....	93
Local file.....	93
Parameters.....	93
Verify upgrade process.....	94
Boot System Image.....	94
Parameters.....	95
Rest Use Case Examples.....	95
Use Case 1.....	98
Use Case 2.....	100
Use Case 3.....	101
Use Case 4.....	102
Use Case 5.....	103
Firmware Site Distribution.....	104
Introduction.....	104
Requirements.....	104
Limitations.....	105
Rest Management Interface.....	105
Seed Initialization.....	105
Start Upgrade Process.....	106
CLI Management Interface.....	107
Seed Initialization.....	107
Step One.....	108
Step Two.....	108
Step Three.....	108
Step Four.....	108
Start Upgrade Process.....	109
Verify Upgrade Process.....	109
Interaction between REST management interface and CLI.....	110
Using Curl Commands.....	110
About the curl command examples.....	110
Getting the REST API versions on the switch.....	111
Accessing the REST API using curl.....	112
Logging in using curl.....	113
Passing the cookie back to the switch.....	115
Logging Out Using Curl.....	116
Examples.....	117

Example: GET method.....	118
Example: Getting and deleting certificates using REST APIs.....	119
Getting a list of all certificates.....	119
Getting a certificate.....	119
Deleting a certificate.....	120
Example: Generating a self-signed certificate using REST APIs.....	120
Example: Getting and installing a signed leaf certificate using REST APIs.....	121
Example: Associating a leaf certificate with a switch feature using REST APIs.....	125
Example: Configuration management using REST APIs.....	126
Downloading a configuration.....	127
Downloading the startup configuration:.....	127
Uploading a configuration.....	127
Copying a configuration.....	128
Example: Log operations using REST APIs.....	130
Event logs.....	130
Accounting (audit) logs.....	131
Example: Ping operations using REST APIs.....	132
Example: Traceroute operations using REST APIs.....	132
Example: User management using REST APIs.....	133
Creating a user.....	133
Changing a password.....	134
Deleting a user.....	135
Example: Creating an ACL with an interface using REST APIs.....	135
Example: Creating a VLAN and a VLAN interface using REST APIs.....	137
Example: Enabling routing on an interface.....	137
Example: PATCH Method.....	139
Enabling a VLAN.....	139
Enabling Central.....	139
Changing the Source IP of a VRF.....	140
Using GET and PATCH to Update the admin state of a VLAN.....	140
Using PATCH to Update a Non-configurable attribute.....	142
AnyCLI.....	143
Commands available per platform.....	144
CLI operations.....	144
CLI commands operations.....	145
Swagger.....	145
Full URI.....	145
CURL example.....	145

Error codes.....	145
Allowed commands.....	146
Full example.....	151
Secure Mode.....	152
Commands available per platform.....	157
VSX peer switches and REST API access.....	158
Examples of curl commands.....	159
Example: Interacting with a VSX peer switch.....	160
AOS-CX real-time notifications subsystem.....	160
Secure WebSocket Protocol connections for notifications.....	162
Notification topics as switch resource URIs.....	162
Rules for topic URIs.....	162
Notification security features.....	164
AOS-CX real-time notifications subsystem reference summary.....	164
Connection protocol.....	164
Port.....	164
Message format.....	165
Message types.....	165
Authorization.....	165
Notification resource URI.....	165
Session idle timeout.....	165
Session hard timeout.....	166
Subscription persistence.....	166
Configuration maximums.....	166
Enabling the notifications subsystem on a switch.....	166
Establishing a secure WebSocket connection through a web browser.....	166
Establishing a secure WebSocket connection using a script.....	167
Subscribing to topics.....	167
Unsubscribing from topics.....	169
Subscription throttling.....	170
Parts of a subscribe message.....	172
Subscribe message example.....	172
Components of a subscribe message.....	173
Parts of a subscription success message.....	173
Example success message.....	173
Components of subscription success message.....	174
Components of a topic.....	175
Parts of a notification message.....	175

Notification message examples.....	176
Components of a notification message.....	178
Components of a topic.....	178
Disconnection scenarios.....	179
Example: Browser-based WebSocket connection.....	180
About the example.....	180
Example screen.....	180
Example HTML source.....	181
Example: Getting information about current subscribers.....	184
Live Switch Events	185
Notification Messages.....	187
Unsubscription.....	189
Limitations.....	190
Firewall Mode Strict.....	190
Overview.....	190
Configuring System Firewall Mode.....	191
Configure VLAN Firewall Mode Strict Bypass.....	192
Show Firewall Mode Status.....	193
Troubleshooting.....	193
General troubleshooting tips.....	194
Connectivity.....	194
Resources, attributes, and behaviors.....	194
GET, PUT, PATCH, POST, and DELETE methods.....	195
Hardware and other features.....	197
REST API response codes.....	198
Error "admin' password is not set"	199
Error "certificate verify failed" returned from curl command.....	200
HTTP 400 error "Invalid Operation".....	200
HTTP 400 error "Value is not configurable" or "Bad Request".....	201
HTTP 401 error "Authorization Required"	201
Solution 1.....	202
Solution 2.....	202
HTTP 401 error "Login failed: session limit reached".....	202
HTTP 403 error "Forbidden" on a write request.....	203
HTTP 403 error "Forbidden" on a GET request.....	203
HTTP 404 error "Page not found" when accessing the switch URL.....	204
HTTP 404 error "Object not found" on object with "ports/" or "interfaces/" in URI Path.....	204

HTTP 404 error "Object not found" returned from a switch that supports multiple REST API versions (10.04 and later).....	205
HTTP 404 error "Object not found" when using a write method.....	205
HTTP 404 error "Page not found" when using a write method.....	205
Logout Fails.....	206
Support and Other Resources.....	206
Accessing HPE Aruba Networking Support.....	207
Accessing Updates.....	208
Warranty Information.....	208
Regulatory Information.....	209
Documentation Feedback.....	209

About this document

This document describes features of the AOS-CX network operating system. It is intended for administrators responsible for installing, configuring, and managing HPE Aruba Networking switches on a network.

Subtopics

Applicable products

Latest version available online

Command syntax notation conventions

About the examples

Identifying switch ports and interfaces

Identifying modular switch components

Applicable products

This document applies to the following products:

- HPE Aruba Networking 4100i Switch Series (JL817A, JL818A)
- HPE Aruba Networking 5420 Switch Series (S0U67A, S0U55A, S0U63A, S0U64A, S0U65A, S0U75A, S0U72A, S0U78A, S0U58A, S0U73A, S0U74A, S0U71A, S0U76A, S0U70A, S0U77A, S0U60A, S0U61A, S0U62A, S0U66A, S0U68A)
- HPE Aruba Networking 6000 Switch Series (R8N85A, R8N86A, R8N87A, R8N88A, R8N89A, R9Y03A, S4R20A, S4R21A, S4R22A, S4R23A, S4R24A, S4R25A, S4R26A, S4R27A, S4R28, S4R29A)
- HPE Aruba Networking 6100 Switch Series (JL675A, JL676A, JL677A, JL678A, JL679A)
- HPE Aruba Networking 6200 Switch Series (JL724A, JL725A, JL726A, JL727A, JL728A, R8Q67A, R8Q68A, R8Q69A, R8Q70A, R8Q71A, R8V08A, R8V09A, R8V10A, R8V11A, R8V12A, R8Q72A, JL724B, JL725B, JL726B, JL727B, JL728B, S0M81A, S0M82A, S0M83A, S0M84A, S0M85A, S0M86A, S0M87A, S0M88A, S0M89A, S0M90A, S0G13A, S0G14A, S0G15A, S0G16A, S0G17A)
- HPE Aruba Networking 6300 Switch Series (JL658A, JL659A, JL660A, JL661A, JL662A, JL663A, JL664A, JL665A, JL666A, JL667A, JL668A, JL762A, R8S89A, R8S90A, R8S91A, R8S92A, S0E91A, S0X44A, S3L75A, S3L76A, S3L77A, S4P41A, S4P42A, S4P43A, S4P44A, S4P45A, S4P46A, S4P47A, S4P48A)
- HPE Aruba Networking 6400 Switch Series (R0X31A, R0X38B, R0X38C, R0X39B, R0X39C, R0X40B, R0X40C, R0X41A, R0X41C, R0X42A, R0X42C, R0X43A, R0X43C, R0X44A, R0X44C, R0X45A, R0X45C, R0X26A, R0X27A, JL741A, S0E48A, S0E48A #0D1, S1T83A, S1T83A #0D1)

- HPE Aruba Networking 8100 Switch Series (R9W94A, R9W95A, R9W96A, R9W97A)
- HPE Aruba Networking 8320 Switch Series (JL479A, JL579A, JL581A)
- HPE Aruba Networking 8325 Switch Series (JL624A, JL625A, JL626A, JL627A)
- HPE Aruba Networking 8325H Switch Series (S4B20A, S4B21A, S4B22A, S4B23A, S2T42A, S2T46A, S2T47A, S2T48A)
- HPE Aruba Networking 8325P Switch Series (S0G12A, S4A48A)
- HPE Aruba Networking 8360 Switch Series (JL700A, JL701A, JL702A, JL703A, JL706A, JL707A, JL708A, JL709A, JL710A, JL711A, JL700C, JL701C, JL702C, JL703C, JL706C, JL707C, JL708C, JL709C, JL710C, JL711C, JL704C, JL705C, JL719C, JL718C, JL717C, JL720C, JL722C, JL721C)
- HPE Aruba Networking 8400 Switch Series (JL366A, JL363A, JL687A)
- HPE Aruba Networking 9300 Switch Series (R9A29A, R9A30A, R8Z96A, S0F81A, S0F82A, S0F83A)
- HPE Aruba Networking 9300S Switch Series (S0F81A, S0F82A, S0F83A, S0F84A, S0F85A, S0F86A, S0F88A, S0F95A, S0F96A)
- HPE Aruba Networking 10000 Switch Series (R8P13A, R8P14A)
- HPE Aruba Networking 10040 Switch Series (S4R58A, S4R59A)

Latest version available online

Updates to this document can occur after initial publication. For the latest versions of product documentation, see the links provided in [Support and Other Resources](#).

Command syntax notation conventions

Convention	Usage
example - text	Identifies commands and their options and operands , code examples, filenames, pathnames, and output displayed in a command window. Items that appear like the example text in the previous column are to be entered exactly as shown and are required unless enclosed in brackets ([]).

Convention	Usage
example - text	In code and screen examples, indicates text entered by a user.
Any of the following: • <code><example - text></code> • <code><example - text></code> • <code>example - text</code> • <code>example - text</code>	Identifies a placeholder—such as a parameter or a variable—that you must substitute with an actual value in a command or in code: • For output formats where italic text cannot be displayed, variables are enclosed in angle brackets (<code>< ></code>). Substitute the text—including the enclosing angle brackets—with an actual value. • For output formats where italic text can be displayed, variables might or might not be enclosed in angle brackets. Substitute the text including the enclosing angle brackets, if any, with an actual value.
	Vertical bar. A logical OR that separates multiple items from which you can choose only one. Any spaces that are on either side of the vertical bar are included for readability and are not a required part of the command syntax.
{ }	Braces. Indicates that at least one of the enclosed items is required.
[]	Brackets. Indicates that the enclosed item or items are optional.
... or ...	Ellipsis: • In code and screen examples, a vertical or horizontal ellipsis indicates an omission of information. • In syntax using brackets and braces, an ellipsis indicates items that can be repeated. When an item followed by ellipses is enclosed in brackets, zero or more items can be specified.

About the examples

Examples in this document are representative and might not match your particular switch or environment.

The slot and port numbers in this document are for illustration only and might be unavailable on your switch.

Understanding the CLI prompts

When illustrating the prompts in the command line interface (CLI), this document uses the generic term **switch**, instead of the host name of the switch. For example:

```
switch>
```

The CLI prompt indicates the current command context. For example:

```
switch>
```

Indicates the operator command context.

```
switch#
```

Indicates the manager command context.

```
switch(CONTEXT-NAME) #
```

Indicates the configuration context for a feature. For example:

```
switch(config-if) #
```

Identifies the **interface** context.

Variable information in CLI prompts

In certain configuration contexts, the prompt may include variable information. For example, when in the VLAN configuration context, a VLAN number appears in the prompt:

```
switch(config-vlan-100) #
```

When referring to this context, this document uses the syntax:

```
switch(config-vlan-<VLAN-ID>) #
```

Where <VLAN-ID> is a variable representing the VLAN number.

Identifying switch ports and interfaces

Physical ports on the switch and their corresponding logical software interfaces are identified using the format: **member/slot/port**.

On the HPE Aruba Networking 4100i Switch Series

- member: Always 1. VSF is not supported on this switch.
- slot: Always 1. This is not a modular switch, so there are no slots.
- port: Physical number of a port on the switch.

For example, the logical interface **1/1/4** in software is associated with physical port 4 on the switch.

On the HPE Aruba Networking 6000 and 6100 Switch Series

- member: Always 1. VSF is not supported on this switch.
- slot: Always 1. This is not a modular switch, so there are no slots.
- port: Physical number of a port on the switch.

For example, the logical interface **1/1/4** in software is associated with physical port 4 on the switch.

On the HPE Aruba Networking 6200 Switch Series

- member: Member number of the switch in a Virtual Switching Framework (VSF) stack. Range: 1 to 8. The primary switch is always member 1. If the switch is not a member of a VSF stack, then member is 1.
- slot: Always 1. This is not a modular switch, so there are no slots.
- port: Physical number of a port on the switch.

For example, the logical interface **1/1/4** in software is associated with physical port 4 in slot 1 on member 1.

On the HPE Aruba Networking 6300 Switch Series

- member: Member number of the switch in a Virtual Switching Framework (VSF) stack. Range: 1 to 10. The primary switch is always member 1. If the switch is not a member of a VSF stack, then member is 1.
- slot: Always 1. This is not a modular switch, so there are no slots.
- port: Physical number of a port on the switch.

For example, the logical interface **1/1/4** in software is associated with physical port 4 on member 1.

On the HPE Aruba Networking 6400 and 5420 Switch Series

- member: Always 1. VSF is not supported on this switch.
- slot: Specifies physical location of a module in the switch chassis.
 - Management modules are on the front of the switch in slots 1/1 and 1/2.
 - Line modules are on the front of the switch starting in slot 1/3.
- port: Physical number of a port on a line module.

For example, the logical interface **1/3/4** in software is associated with physical port 4 in slot 3 on member 1.

On the HPE Aruba Networking 8xxx, 93xx, and 10xxx Switch Series

- member: Always 1. VSF is not supported on this switch.
- slot: Always 1. This is not a modular switch, so there are no slots.
- port: Physical number of a port on the switch.

For example, the logical interface **1/1/4** in software is associated with physical port 4 on the switch.



NOTE

If using breakout cables, the port designation changes to x:y, where x is the physical port and y is the lane when split. For example, the logical interface 1/1/4:2 in software is associated with lane 2 on physical port 4 in slot 1 on member 1.

On the HPE Aruba Networking 8400 Switch Series

- member: Always 1. VSF is not supported on this switch.
- slot: Specifies physical location of a module in the switch chassis.
 - Management modules are on the front of the switch in slots 1/5 and 1/6.
 - Line modules are on the front of the switch in slots 1/1 through 1/4, and 1/7 through 1/10.
- port: Physical number of a port on a line module

For example, the logical interface **1/1/4** in software is associated with physical port 4 in slot 1 on member 1.

Identifying modular switch components

- Power supplies are on the front of the switch behind the bezel above the management modules. Power supplies are labeled in software in the format: member/power supply:
 - member: 1.
 - power supply: 1 to 4.
- Fans are on the rear of the switch and are labeled in software as: member/tray/fan:
 - member: 1.
 - tray: 1 to 4.
 - fan: 1 to 4.
- Fabric modules are not labeled on the switch but are labeled in software in the format: member/module:
 - member: 1.
 - member: 1 or 2.
- The display module on the rear of the switch is not labeled with a member or slot number.

Introduction to the AOS-CX REST API



NOTE

The HPE Aruba Networking 6000 Switch Series and 6100 Switch Series only support the default VRF and has no management port. Therefore, references in this guide to other VRFs or the management port do not apply to the 6000 switches and 6100 switches. Configuration for these switches should be done over an SVI having a physical port with access to the SVI, since the physical ports in the 6000 and 6100 are not routed.

Switches running the AOS-CX software are fully programmable with a REST (REpresentational State Transfer) API, allowing easy integration with other devices both on premises and in the cloud. This programmability—combined with the ArHPE Aruba Networking uba Network Analytics Engine—accelerates network administrator understanding of and response to network issues.

The AOS-CX REST API is a web service that performs operations on switch resources using HTTPS **POST**, **GET**, **PUT**, **PATCH**, and **DELETE** methods.

The AOS-CX REST API enables programmatic access to the AOS-CX configuration and state database at the heart of the switch. By using a structured model, changes to the content and formatting of the CLI output do not affect the programs you write. The configuration is stored in a structured database, instead of a text file, making it easier to roll back changes, and dramatically reducing the risk of downtime and performance issues.



NOTE

Additional API documentation is available on the HPE Aruba Networking Developer Hub, at <https://developer.arubanetworks.com/hpe-aruba-networking-aoscx/docs/about>

Subtopics

[REST API versions](#)

[REST API access modes](#)

[REST API URI](#)

[Resources](#)

REST API versions

AOS-CX switches support access through multiple versions of the REST API. The REST API versions supported on the AOS-CX switches are v10.04, and versions v10.08-v10.18.xxxx.



NOTE

REST v1 is deactivated and no longer supported with AOS-CX 10.12

The version declared in the REST request must match one of the versions of the REST API supported on the switch. The REST API version is included in the Uniform Resource Identifier (URI) used in REST requests.

In the following example, the REST API version is `v10.15`:

```
https://192.0.2.5/rest/v10.15/latest/system
```

In the following example, the REST API version is `v10.12`:

```
https://192.0.2.5/rest/v10.12/system
```

Subtopics

Switches upgrading to AOS-CX 10.15

Compatibility

Switches upgrading to AOS-CX 10.15

The REST API now supports a new URI, `<ip-addr>/api/v10.15/#/hpe_anw_central`. Starting with AOS-CX 10.15, new APIs that support HPE Aruba Networking Central will appear at this URI. The legacy URI `<ip-addr>/api/v10.15/#/aruba_central` will continue to contain all APIs introduced in AOS-CX 10.04 through AOS-CX 10.14.



NOTE

Switches introduced in 10.15 or later, such as the 5420, will only support the 10.15 API schema with the **hpe-anw-central** URI.

Compatibility

The following table shows the compatibility of AOS-CX switches with older REST API versions. To maintain compatibility with older versions on new AOS-CX switches, old versions continue to be published and use the same schema as the newest REST API version.

Switch	REST v10.04 API	REST v10.08 API	REST v10.09 API	REST v10.10 API	REST v10.11- v 10.16 API	REST v10.17 API
6100	YES	YES	YES	YES	YES	YES
5420	YES (with 10.17 schema)	YES (with 10.17 schema)	YES (with 10.17 schema)	YES (with 10.17 schema)	YES (with 10.17 schema)	YES
6200	YES	YES	YES	YES	YES	YES
6300	YES	YES	YES	YES	YES	YES
6400	YES	YES	YES	YES	YES	YES
8320	YES	YES	YES	YES	YES	YES
8325	YES	YES	YES	YES	YES	YES
8360 Series J L7xxA	YES	YES	YES	YES	YES	YES
8400	YES	YES	YES	YES	YES	YES
4100i	YES (with 10.17 schema)	YES	YES	YES	YES	YES
6000	YES (with 10.17 schema)	YES	YES	YES	YES	YES
8360 Series J L7xxC	YES (with 10.17 schema)	YES (with 10.17 schema)	YES	YES	YES	YES
10000	YES (with 10.17 schema)	YES (with 10.17 schema)	YES	YES	YES	YES
9300	YES (with 10.17 schema)	YES (with 10.17 schema)	YES (with 10.17 schema)	YES	YES	YES
6000 Series R xxxxx	YES (with 10.17 schema)	YES (with 10.17 schema)	YES (with 10.17 schema)	YES (with 10.17 schema)	YES	YES
6100 Series R xxxxx	YES (with 10.17 schema)	YES (with 10.17 schema)	YES (with 10.17 schema)	YES (with 10.17 schema)	YES	YES
6200 Series J LxxxxA	YES (with 10.17 schema)	YES (with 10.17 schema)	YES (with 10.17 schema)	YES (with 10.17 schema)	YES	YES

The following table shows the state of the Swagger UI page for each AOS-CX switch.

Switch	Swagger v10.04	Swagger v10.08	Swagger v10.09	Swagger v10.10	Swagger v10.11-10.16	Swagger v10.17
6100	YES	YES	YES	YES	YES	YES
5420	YES (with 10.17 schema)	YES (with 10.17 schema)	YES (with 10.17 schema)	YES (with 10.17 schema)	YES (with 10.17 schema)	YES
6200	YES	YES	YES	YES	YES	YES
6300	YES	YES	YES	YES	YES	YES
6400	YES	YES	YES	YES	YES	YES
6400	YES	YES	YES	YES	YES	YES
6400	YES	YES	YES	YES	YES	YES
8320	YES	YES	YES	YES	YES	YES
8325	YES	YES	YES	YES	YES	YES
8360 JL7xxA	YES	YES	YES	YES	YES	YES
8400	YES	YES	YES	YES	YES	YES
4100i	YES, (with 10.17 schema)	YES	YES	YES	YES	YES
6000	YES, (with 10.17 schema)	YES	YES	YES	YES	YES
8360 JL7xxC	YES, (with 10.17 schema)	YES, (with 10.17 schema)	YES	YES	YES	YES
10000	YES, (with 10.17 schema)	YES, (with 10.17 schema)	YES	YES	YES	YES
9300	YES, (with 10.17 schema)	YES, (with 10.17 schema)	YES, (with 10.17 schema)	YES	YES	YES
6000 Rxxxxx	YES, (with 10.17 schema)	YES, (with 10.17 schema)	YES, (with 10.17 schema)	YES, (with 10.17 schema)	YES	YES
6100 Rxxxxx	YES, (with 10.17 schema)	YES, (with 10.17 schema)	YES, (with 10.17 schema)	YES, (with 10.17 schema)	YES	YES

Switch	Swagger v10.04	Swagger v10.08	Swagger v10.09	Swagger v10.10	Swagger v10.11-10.16	Swagger v10.17
6200 JLxxxA	YES, (with 10.17 schema)	YES, (with 10.17 schema)	YES, (with 10.17 schema)	YES, (with 10.17 schema)	YES	YES

REST API access modes

The REST API supports two access modes:

- read-write (default)
- read-only

The default **read-write** access mode is not displayed in the **show running-configuration** command. You can change the access mode to read-only using the **https-server rest access-mode read-only** CLI command from the global configuration (**config**) context. You can validate the mode set using the **show https-server** command.

Subtopics

Read-write access mode

Read-only access mode

Read-write access mode

In the read-write access mode:

- The AOS-CX REST API Reference shows most of the supported read and write methods for all switch resources.
- The REST API can access and change every configurable aspect of the switch as modeled in the configuration and state database.



CAUTION

The REST API is powerful, but must be used with extreme caution: For most values, no semantic validation is performed on the data that you write to the database, and configuration errors can destabilize the switch.

Read-only access mode

In the read-only access mode:

- Most switch resources support only GET methods, but some resources allow PUT or POST methods. For example, you can use POST to log into the switch, use PUT to upload a new running configuration, or use POST to upload a new firmware version.
- Read-only mode applies to all clients, including HPE Aruba Networking Central, HPE Aruba Networking Fabric Composer and NetEdit.
- For most switch resources, the AOS-CX REST API Reference does not show any write methods (POST, PUT, PATCH, and DELETE) the resource might support. To show those write methods, read-write mode must be enabled.
- A request to a read-only resource returns the code **405 Method Not Allowed**.
- Once read-only mode is enabled, it can only be disabled through the switch command-line interface.

REST API URI

A switch resource is indicated by its Uniform Resource Identifier (URI). A URI is the location of a specific web resource. A URI can be made up of several components, including the host name or IP address, port number, the path, and an optional query string.

Subtopics

Parts of a URI

Parts of a URI

The two main parts of a URI are the path and the (optional) query component.

Subtopics

URI path, including path parameters

Query component

URI path, including path parameters

The path is the part of the URI starting with the server URL and ending with the resource ID. In URIs that have a query component, the path is everything before the question mark (?). The path has a hierarchy. In a path, the forward slash (/) indicates the hierarchical relationship between resources.

Because the forward slash has a special meaning, the forward slash characters that are part of the URI path must be percent-encoded with the code %2F, which represents the forward slash. For example, the following URI represents the resource utilization for the management module in slot 1/5:

```
https://192.0.2.5/rest/v10.xx/system/subsystems/management_module,1%2F5?
attributes=resource_utilization
```

Subtopics

URI prefix

Server URL

Path parameters

URI prefix

The URI prefix is the system URL and REST API version information. This information is specific to a particular switch and REST API version, and is the same for every REST API request to that switch.

Script writers often create a variable for the URI prefix. Using a variable enables the writer to update a script or use the same script logic for a different switch by updating the value of the URI prefix variable.

The URI prefix contains the following:

Server URL

The web server address of the switch.

Examples:

- **https://192.0.2.5**
- **https://10.17.0.1**
- **https://myswitch.mycompany.com**

If Virtual Switching Extension (VSX) is enabled, you can access most resources of the peer switch from this switch by adding `/vsx-peer` in the URI path between the server URL and `/rest`. For more information about VSX, see [VSX Peer Switches and REST API Access](#).

For example:

```
GET https://192.0.2.5/vsx-peer/rest/v10.xx/system/vsx?attributes=oper_status
```

REST API and version identifier

For example: `/rest/v10.xx`

Path parameters

A path parameter is a part of the URI path that can vary. Typically path parameters indicate a specific instance of a resource in a collection, such as a specific VLAN in the `vlan`s collection. The path can contain several path parameters. Path parameters are indicated by braces `{ }`.

For example, the AOS-CX REST API Reference displays the resource for specific VLAN as the following:

```
/system/vlans/{id}
```

When you send a request for VLAN 10, the URI you provide must substitute the VLAN ID, `10`, for the `{id}` query parameter. For example:

```
/system/vlans/10
```

In the AOS-CX REST API Reference, you enter the value of the path parameter in the **Value** field of the **id** parameter.

Query component

In many cases, the unique identification of a resource requires a URI that contains both a path and a query component. The query component is sometimes called the query string.

For example, CPU utilization is a resource represented by the following URI:

```
https://192.0.2.5/rest/v10.xx/system/subsystems/management_module,1%2F5?attributes=resource_utilization
```

In a URI, the question mark (`?`) indicates the beginning of the query component. The query component contains nonhierarchical data, and the format of the query string depends on the implementation of the REST API.

The query component often contains " `<key>=<value>` " pairs separated by the ampersand (`&`) character. Multiple attribute values are supported and are separated by commas. For example:

```
https://192.0.2.5/rest/v10.xx/system/vlans?depth=2&attributes=id,name,type
```

Subtopics

"Dot" notation for Network Analytics Engine URIs only

"Dot" notation for Network Analytics Engine URIs only

When a URI defines a monitor in an HPE Aruba Networking Network Analytics Engine (NAE) script, attribute values in the query string support an additional dot notation that the Network Analytics Engine uses to access additional information. For example:

```
https://192.0.2.5/rest/v10.09/system/subsystems/management_module,1%2F5?
attributes=resource_utilization.cpu
```

The dot notation is supported for certain URIs that define monitors only in NAE scripts.

Resources

In a REST API, the primary representation of data is called a **resource**. A resource is a representation of an entity in the system as a URI. The entities can include hardware objects, statistical information, configuration information, and status information. The URI might or might not include a query component. Resources are nouns—anything that can be named can be a resource.

Examples of resources:

- The resource utilization information:

```
https://192.0.0.5/rest/v10.xx/system/subsystems?
attributes=resource_utilization
```

- The list of configured VLANs:

```
https://192.0.2.5/rest/v10.xx/system/vlans
```

- The list of all users:

```
https://192.0.2.5/rest/v10.xx/system/users
```

- The user with the ID: `myadmin`:

```
https://192.0.2.5/rest/v10.xx/system/users/myadmin
```

- The secondary firmware image:

```
https://192.0.2.5/rest/v10.xx/firmware?image=secondary
```

Subtopics

Resource collections and singletons

Categories of resource attributes

Resource collections and singletons

Subtopics

Collections

A collection is a directory of resources managed by the server. Typically, a resource collection contains multiple resource instances and the collection name is in the plural form.

For example:

- `/system/vlans`
- `/system/users`
- `/fullconfigs`

A GET request to a collection returns the set of JSON objects representing the members of the collection. The following curl example shows the GET request and response returned for the `vlans` collection:

```
$ curl -k GET -b /tmp/auth_cookie "https://192.0.2.5/rest/v10.04/system/vlans"
{
  "1": "/rest/v10.04/system/vlans/1",
  "10": "/rest/v10.04/system/vlans/10",
  "20": "/rest/v10.04/system/vlans/20"
}
```

Each URI in the list represents a configured VLAN.

To get the JSON data for VLAN 10, you must either send the GET request to the URI representing VLAN 10 (`/rest/v10.xx/system/vlans/10`), or you must use the depth parameter to expand the list of URIs in the `vlans` collection to get the JSON data for all the VLANs in the collection.

Subcollections

A single resource instance can also contain subcollections of resources.

- In the following example, `vlans` is a subcollection of the `system` resource:

```
/system/vlans
```

- In the following example, `routes` is a subcollection of the `default` VRF resource instance:

```
/system/vrfs/default/routes
```

Singletons

There are some resources that can only have one instance. These resources are called singletons and the resource collection name is in the singular form.

For example:

- `/system`
- `/system/vsx`
- `/firmware`

Because there is only one resource in a singleton collection, GET requests return the JSON representation of the resource instead of a URI list of one item. In addition, you do not need to supply a resource ID in the URL of a GET request. For example, the following GET request to the firmware URI returns the JSON data that represents the firmware resource:

```
$ curl -k GET -b /tmp/auth_cookie "https://192.0.2.5/rest/v10.xx/firmware"
{
  "current_version": "TL.10.00.0006E-686-g4a43ab9",
  "primary_version": "TL.10.00.0006E-686-g4a43ab9",
  "secondary_version": "",
  "default_image": "primary",
  "booted_image": "primary"
}
```

Categories of resource attributes

Resources can contain many attributes, and they are organized into the following categories to enable more efficient management:

Subtopics

Configuration attributes

Writable attributes

Status attributes

Statistics attributes

Attribute categories might vary

Configuration attributes

Configuration attributes represent user-owned data. Although an attribute must be in the configuration category to be modified by a user, not all attributes in the configuration category can be modified after the

resource instance is created. Configuration attributes that cannot be changed after the resource is created are called **immutable** attributes. This distinction matters when using a PUT request, because immutable attributes cannot be included in the request body.

For example, a VLAN ID is an immutable attribute. You cannot change the ID of the VLAN after the VLAN is created. The VLAN name, in contrast, is a **mutable (writable)** attribute. You can change the VLAN name after the VLAN is created.

Writable attributes

Writable attributes are the subset of configuration attributes that are **mutable**. Writable attributes can be modified by a user after the resource is created. When using the PUT method to modify a resource, only writable attributes can be included in the request body.

In REST v10.04 and later versions, the GET method `selector` parameter includes a value of `writable`, which enables you to get only the mutable configuration attributes of a resource.

Status attributes

Status attributes contain system-owned data such as the admin account and various status fields. You cannot create or modify instances of attributes in this category.

Statistics attributes

Statistics attributes contain system-owned data such as counters. You cannot create or modify instances of attributes in this category.

Attribute categories might vary

A given attribute need not necessarily be in the same category from resource to resource, or even resource instance to resource instance. If the system owns an instance of a resource, attributes of that resource (which might be configuration attributes if a user owns the resource instance) become status attributes, which cannot be modified by users.

For example, a user can create VLANs. However, the system can also create VLANs. System-owned VLANs have many attributes that are considered to be in the status category and not the configuration category. The status category is used when the data is owned by the system and cannot be overwritten by a user.

Often a resource has a single attribute that indicates whether the resource is owned by the system or by a user. For example, for a VLAN, the `type` attribute indicates whether the VLAN was created by a user.

When this indicator attribute indicates that the resource is owned by the system, the other attributes that might have been in the configuration category are categorized as status attributes. Likewise, when the indicator attribute indicates that the resource is owned by a user, the other configuration attributes remain available for modification by users. In other words, the categories for other attributes on the resource follow the indicator attribute.

Enabling Access to the REST API

The AOS-CX Web UI and AOS-CX real-time notifications subsystem rely on the REST API, therefore, all three are enabled or disabled together.

To access the REST API, Web UI, or notifications subsystem, the HTTPS server must be enabled on the specified VRF. The VRF you specify determines from which network the features can be accessed. You can enable access on multiple VRFs, including user-defined VRFs, by entering the **`https-server vrf`** command for each VRF on which you want to enable access.

Prerequisites

- You must be in the global configuration context: **`switch(config)#`**.
- For the password-based authentication, the password for the `admin` user must be configured on the switch.
- For the certificate-based authentication method, the trust anchor (TA) profile is needed to validate the client certificate. Also, a RADIUS server must be configured on the switch. For more information on configuring certificates and managing certificates, see the AOS-CX Security Guide.

Procedure

Enable HTTPS server access for the specified VRF.

For example:

- To enable access on all data ports on the switch, specify the default VRF:

```
switch(config) # https-server vrf default
```



NOTE

The HPE Aruba Networking 6000 Switch Series and 6100 Switch Series only supports the **default** VRF.

- To enable access on the OOBM port (management interface IP address), specify the management VRF (not applicable to the 6000 and 6100):

```
switch(config) # https-server vrf mgmt
```

- To enable access on ports that are members of the VRF named **vrfprogs**, specify **vrfprogs**:

```
switch(config) # https-server vrf vrfprogs
```

In the case of password authentication, if the switch responds with the following error, the `admin` user must have a valid password:

```
Failed to enable https-server on VRF mgmt. 'admin' password is not set
```

The switch is shipped from the factory with a default user named **admin** without a password. The **admin** user must set a valid password before HTTPS servers can be enabled.

Subtopics

Setting the admin password

Showing the REST API access configuration

Disabling access to the REST API

Setting the admin password

Use the following API to login as the admin.

```
POST /rest/v10.xx/login?username=admin
```

A new session is started and a response code `268` is returned along with the message: "Session is restricted. Administrator password must be set before continuing."



NOTE

This session is valid only to change the admin password and logout from the REST API UI. Any other request will return a Forbidden code (403).

Use the following API to change the admin password. Ellipses (...) represent data not included in the example.

```
PUT /rest/v10.xx/system/users/admin
{
...
"password": "<enter the password>"
...
}
```

After the password is changed successfully, the session restriction is removed.

Showing the REST API access configuration

To show the REST API access configuration, in the manager context (`#`) of the CLI, enter the **show https-server** command.

For example:

```
switch# show https-server
HTTPS Server Configuration
-----
VRF                : mgmt, default
REST Access Mode   : read-write
```



NOTE

The HPE Aruba Networking 6000 Switch Series and 6100 Switch Series only supports the **default** VRF.

The command output lists the VRFs on which access to REST API is enabled and shows the current REST API access mode.

If access is not enabled on any VRF, the VRF configuration is displayed as `<none>`.

For example:

```
switch# show https-server
HTTPS Server Configuration
-----
VRF                : <none>
REST Access Mode   : read-write
```

Disabling access to the REST API



NOTE

The AOS-CX Web UI and AOS-CX real-time notifications subsystem rely on the REST API, therefore, all three are enabled or disabled together.

Prerequisites

You must be in the global configuration context: `switch(config)#`.

Procedure

Disable HTTPS server access for the specified VRF by using the `no` form of the **https-server vrf** command.

For example, the following command disables REST API access on the switch data ports in the default VRF:

```
switch(config)# no https-server vrf default
```

You can use the `show https-server` command to show the current configuration:

```
switch# show https-server
HTTPS Server Configuration
-----
VRF                : mgmt
REST Access Mode   : read-write
```

HTTPS server commands

Subtopics

- [https-server authentication certificate](#)
- [https-server authentication password](#)
- [https-server max-user-sessions](#)
- [https-server rest access-mode](#)
- [https-server rest firmware-site-distribution](#)
- [https-server session close all](#)
- [https-server session-timeout](#)
- [https-server vrf](#)
- [show https-server](#)
- [show https-server authentication](#)
- [show https-server 1rest images](#)

https-server authentication certificate

Syntax

```
https-server authentication certificate [authorization radius] [username  
{<CERT-FIELD>}]
```

Description

Enables authentication using an x509 certificate for authentication. When this option is configured, the https-server uses the user specified certificate for authentication, and the specified authorization mechanism is used to obtain the corresponding user role. The username embedded in the certificate is used for authorization with a remote user database.

Enabling password authentication is the only way of disabling certificate authentication.



NOTE

Only one authentication method can be enabled at a time. If you want to disable certificate-based authentication, then the password-based authentication must be enabled.

Parameter	Description
<code><AUTHORIZATION-RADIUS></code>	Specifies that after certificate authentication succeeds, instead of prompting for a password, the HTTPS server checks the RADIUS server only for authorization. When this parameter is omitted, <code>authorization radius</code> is still the assumed active setting.
<code><CERT-FIELD></code>	Selects which certificate username field is to be used for authorization. - Specify <code>user_principal_name</code> to use the certificate UserPrincipalName (UPN) field. This is the default. - Specify <code>common_name</code> to use the certificate Common Name (CN) field. When this parameter is omitted, <code>user_principal_name</code> is assumed.

Example

Enabling authentication using the certificate:

```
switch(config)# https-server authentication certificate authorization
radius username common_name
```

Command History

Release	Modification
10.11	Command introduced.

Command Information

Platforms	Command context	Authority
All platforms	config	Administrators or local user group members with execution rights for this command.

https-server authentication password

Syntax

```
https-server authentication password
```

Description

Enables authentication using username and password, which corresponds to the default authentication mechanism. Enabling the password authentication mode disables the certificate authentication mode.



NOTE

Only one authentication method can be enabled at a time.

Example

Enabling authentication using the password:

```
switch(config) # https-server authentication password
```

Command History

Release	Modification
10.11	Command introduced.

Command Information

Platforms	Command context	Authority
All platforms	config	Administrators or local user group members with execution rights for this command.

https-server max-user-sessions

Syntax

```
https-server max-user-sessions <SESSION-AMT>
```

Description

Sets the maximum amount of concurrent open sessions for any given user through the HTTPS server. The amount of concurrent open sessions may have an impact on system performance, so it is recommended to set this value to the minimum necessary.

Parameter	Description
<code><SESSION-AMT></code>	Specifies the maximum number of user sessions allowed. Default: 6. Maximum value: 8.

Example

Set the maximum number of concurrent user sessions to the maximum of 8:

```
switch(config) # https-server max-user-sessions 8
```

Command History

Release	Modification
10.08	Command introduced

Command Information

Platforms	Command context	Authority
All platforms	config	Administrators or local user group members with execution rights for this command.

https-server rest access-mode

Syntax

```
https-server rest access-mode {read-only | read-write}
```

Description

Changes the REST API access mode. The default mode is read-write. This command does not affect Central connections, which have permission to alter configurations regardless of the access mode set on the switch.

Parameter	Description
read-write	Selects the read/write mode. Allows POST, PUT, PATCH, and DELETE methods to be called on all configurable elements in the switch database.
read-only	Selects the read - only mode. Write access to most switch resources through the REST API is disabled.

Usage

Setting the mode to `read-write` on the REST API allows POST, PUT, PATCH, and DELETE methods to be called on all configurable elements in the switch database.

By default, REST APIs in the device are in the read-write mode. Some switch resources allow POST, PUT, PATCH, and DELETE regardless of REST API mode. REST APIs that are required to support the Web UI or the Network Analytics Engine expose POST, PUT, PATCH, or DELETE operations, even if the REST API access mode is set to read-only.

The REST API in read/write mode is intended for use by advanced programmers who have a good understanding of the system schema and data relationships in the switch database.



CAUTION

Because the REST API in read/write mode can access every configurable element in the database, it is powerful but must be used with extreme caution: No semantic validation is performed on the data you write to the database, and configuration errors can destabilize the switch.

On 6300 switches or 6400 switches, by default, the HTTPS server is enabled in `read-write` mode on the `mgmt` VRF. If you enable the HTTPS server on a different VRF, the HTTPS server is enabled in `read-only` mode.

Example

```
switch(config)# https-server rest access-mode read-only
```

Command History

Release	Modification
10.07 or earlier	- -

Command Information

Platforms	Command context	Authority
All platforms	config	Administrators or local user group members with execution rights for this command.

https-server rest firmware-site-distribution

Syntax

```
https-server rest firmware-site-distribution
no https-server rest firmware-site-distribution
```

Description

This command enables or disables the Firmware Site Distribution server listening on port 9443.

The firmware site distribution allows you to use a switch to distribute a firmware image file to other switches in the same network. This prevents the switches from connecting to the cloud or an external network to download a firmware image file.

On enabling the firmware site distribution, it exposes a REST endpoint that allows the switches to download a switch primary or secondary firmware image. 1



NOTE

As per the limitation, up to two switches can download the firmware image simultaneously.

This endpoint is to be used along with REST `/firmware` endpoint to handle the firmware download and installation process.

The **no** form of this command disables the firmware site distribution server.

Examples

Enabling the HTTPS server firmware site distribution:

```
switch(config) # https-server rest firmware-site-distribution
```

Disabling the HTTPS server firmware site distribution:

```
switch(config) # no https-server rest firmware-site-distribution
```

Executing the HTTPS server firmware site distribution:

```
switch(config) # firmware-site-distribution image <primary|secondary> from  
<location> vrf <vrf_name>
```

Here, **location** is the URL source from where the image is going to be downloaded. In FWSD, it's the URL for the seed device with the specified image file (the whole URL path). **primary|secondary** stands for the physical location where the firmware file is going to be downloaded or stored.

Command History

Release	Modification
10.14.1000	Added command for executing server firmware side distribution.
10.10	Command introduced.

Command Information

Platforms	Command context	Authority
All platforms	config	Administrators or local user group members with execution rights for this command.

https-server session close all

Syntax

```
https-server session close all
```

Description

Invalidates and closes all HTTPS sessions. All existing WebUI sessions (including sessions used for Central connections) will be logged out. REST and WebUI users will have to reauthenticate, and all real-time notification feature WebSocket connections are closed and must be resubscribed.

Usage

Typically, a user that has consumed the allowed concurrent HTTPS sessions and is unable to access the session cookie to log out manually must wait for the session idle timeout to start another session. This command is intended as a workaround to waiting for the idle timeout to close an HTTPS session. This command stops and starts the `hpe-restd` service, so using this command affects all existing REST sessions, Web UI sessions, and real-time notification subscriptions.

Example

```
switch# https-server session close all
```

Command History

Release	Modification
10.07 or earlier	- -

Command Information

Platforms	Command context	Authority
All platforms	Manager (#)	Administrators or local user group members with execution rights for this command.

https-server session-timeout

Syntax

```
https-server session-timeout <MINUTES>
```

Description

Configures the timeout, in minutes, for any given HTTPS server session. A value of 0 disables the timeout. This command does not affect sessions used for Central connections.

Parameter	Description
	Specifies the maximum idle time, in minutes for an HTTPS session. Default: 20. Maximum: 480 (8 hours). 0 disables the timeout, but the maximum is still enforced.

Parameter	Description
<MINUTES>	

Example

```
switch(config) # https-server session-timeout 10
```

Command History

Release	Modification
10.08	Command introduced

Command Information

Platforms	Command context	Authority
All platforms	config	Administrators or local user group members with execution rights for this command.

https-server vrf

Syntax

```
https-server vrf <VRF-NAME>
no https-server vrf <VRF-NAME>
```

Description

Configures and starts the HTTPS server on the specified VRF, allowing access to REST and the WebUI from ports assigned to that VRF. This command does not affect access to HPE Aruba Networking Central instances, as this feature has its own dedicated connection channel.

The **no** form of the command stops any HTTPS servers running on the specified VRF and removes the HTTPS server configuration.

Parameter	Description
<code><VRF-NAME></code>	Specifies the VRF name. Required. Length: Up to 32 alpha numeric characters.

Usage

By using this command, you enable access to both the Web UI and to the REST API on the specified VRF. You can enable access on multiple VRFs.

By default, HPE Aruba Networking 8100, 8320, 8325, 8360, 8400, 9300/9300S, 10000, 10040 Switch Series have an HTTPS server enabled on the **mgmt** VRF.

By default, the HPE Aruba Networking 5420, 6200, 6300, and 6400 Switch Series have an HTTPS server enabled on the **mgmt** VRF and on the **default** VRF.

When the HTTPS server is not configured and running, attempts to access the Web UI or REST API result in **404 Not Found** errors.

The VRF you select determines from which network the Web UI and REST API can be accessed.

For example:

- If you want to enable access to the REST API and Web UI through the OOBM port (management IP address), specify the built-in management VRF (**mgmt**).
- If you want to enable access to the REST API and Web UI through the data ports (for "inband management"), specify the built-in default VRF (**default**).
- If you want to enable access to the REST API and Web UI through only a subset of data ports on the switch, specify other VRFs you have created.

HPE Aruba Networking Network Analytics Engine scripts run in the default VRF, but you do not have to enable HTTPS server access on the default VRF for the scripts to run. If the switch has custom HPE Aruba Networking Network Analytics Engine scripts that require access to the Internet, then for those scripts to perform their functions, you must configure a DNS name server on the default VRF.

Examples

Enabling access on all ports on the switch, specify the default VRF:

```
switch(config)# https-server vrf default
```

Enabling access on the OOBM port (management interface IP address), specify the management VRF:

```
switch(config)# https-server vrf mgmt
```

Enabling access on ports that are members of the VRF named `vrfprogs`, specify `vrfprogs`:

```
switch(config)# https-server vrf vrfprogs
```

Enabling access on the management port and ports that are members of the VRF named `vrfprogs`, enter two commands:

```
switch(config)# https-server vrf mgmt
switch(config)# https-server vrf vrfprogs
```

**NOTE**

The HPE Aruba Networking 6200 and 5420 switches support only two VRFs. One management VRF and one default VRF. You cannot add another VRF.

Command History

Release	Modification
10.07 or earlier	- -

Command Information

Platforms	Command context	Authority
All platforms	config	Administrators or local user group members with execution rights for this command.

show https-server

Syntax

```
show https-server [vsx-peer]
```

Description

Shows the status and configuration of the HTTPS server. The REST API and web user interface are accessible only on VRFs that have the HTTPS server features configured.

Parameter	Description
vsx-peer	Shows the output from the VSX peer switch. If the switches do not have the VSX configuration or the ISL is down, the output from the VSX peer switch is not displayed. This parameter is available on switches that support VSX.

Usage

Shows the configuration of the HTTPS server features.

VRF

Shows the VRFs, if any, for which HTTPS server features are configured.

REST Access Mode

Shows the configuration of the REST access mode:

read-write

POST, PUT, and DELETE methods can be called on all configurable elements in the switch database. This is the default value.

read-only

Write access to most switch resources through the REST API is disabled.

Examples

```
switch# show https-server
HTTPS Server Configuration
-----
VRF                : default, mgmt
REST Access Mode   : read-write
Max sessions per user : 6
Session timeout     : 20
```

Command History

Release	Modification
10.07 or earlier	- -

Command Information

Platforms	Command context	Authority
All platforms	Manager (#)	Administrators or local user group members with execution rights for this command.

show https-server authentication

Syntax

```
show https-server authentication
```

Description

Shows the https-server authentication mode status.

Examples

Showing the authentication method with the password mode enabled:

```
switch# show https-server authentication
Authentication Modes Status
-----
Password Status           : enabled
Certificate Status        : disabled
```

Showing the authentication method with the certificate mode enabled:

```
switch# show https-server authentication
Authentication Modes Status
-----
Password Status           : disabled
Certificate Status        : enabled
```

Command History

Release	Modification
10.11	Command Introduced.

Command Information

Platforms	Command context	Authority
All platforms	Operator (>) or Manager (#)	Operators or Administrators or local user group members with execution rights for this command. Operators can execute this command from the operator context (>) only.

```
show https-server 1rest images
```

Syntax

```
show https-server rest images
```

Description

Displays the firmware images location for REST usage.

Parameter	Description
images	Shows firmware image location.

Examples

```
switch# show https-server rest images
REST Software Images Location
-----
primary   : "/fwimages/primary.swi"
secondary : "/fwimages/secondary.swi"
```

Command History

Release	Modification
10.14.1000	Command introduced.

Command Information

Platforms	Command context	Authority
All platforms	Manager (#)	Administrators or local user group members with execution rights for this command.

Accessing the AOS-CX REST API

You can access the REST API using any REST client interface that supports HTTPS requests, and supports obtaining and passing a session cookie.

Examples of client interfaces include the following:

Scripts and programs that support HTTPS requests

A flexible way to access the AOS-CX REST API is to use a programming language that supports HTTPS requests, such as Python, to write programs that automate network management tasks.

The curl command-line interface

You can use curl commands either interactively or within a script to make REST requests. Using curl commands is a way to execute GET requests without writing a script. Using curl commands is a way to test REST requests that you are considering to incorporate into an application.

Browser-based interfaces such as Postman or the AOS-CX REST API Reference

Examples of browser-based interfaces include Postman and the AOS-CX REST API Reference.

The AOS-CX REST API Reference documents the switch resources, parameters, and JSON models for each HTTPS method supported by the resource. Because the AOS-CX REST API Reference is browser-based, it can share the session cookie with a Web UI session active in another browser tab. The AOS-CX REST API Reference is not intended to be used as a configuration tool and is not required for day-to-day operations.

The AOS-CX REST API Reference is one way to execute GET requests without writing a script. The AOS-CX REST API Reference can be used during script coding to help you construct the URIs—with their query parameters—that you use in a script or curl command.

Sessions

To avoid identity theft, sessions are bound to the IP address created by the clients. If a client with a different IP address tries to use the session, the switch will display a 403 Forbidden response, while maintaining the session active for the correct client.



NOTE

The IP address registered for each session is read from a switch perspective. If the login request is redirected through a proxy or tunnel, that address will be used for future validations.

Subtopics

Authenticating REST API sessions

User groups and access authorization

Authenticating REST API sessions

When you start a REST API session, you use the POST method to access the `login` resource of the switch and pass the username and password information as data. Ensure that HTTPS is configured to use port 443. Requests to port 80 are redirected to port 443.

If the credentials are accepted, your authenticated session is started for that username, and the switch returns a cookie containing encoded session information.

In subsequent calls to the API—including to the `logout` resource—the session cookie is passed back to the switch.

The same session cookie is shared across browser tabs, and depending on the browser, multiple browser windows. However, the same session cookie is not shared across devices and scripts. For example, if a user logs into the Web UI from a laptop, again with a tablet, and then uses the same user name in a curl command, that user has three concurrent client sessions.



NOTE

The maximum number of concurrent HTTPS sessions per user per switch is eight. There is an upper limit of 48 total sessions per switch. It is a best practice to log out of HTTPS sessions when you are finished using them.

HTTPS sessions will automatically time out after 20 minutes of inactivity, and have a hard time limit of eight hours, regardless of whether the session is active. You can run the **https-server session close all** command to close all current HTTPS sessions. For more information about using the command, see **https-server session close all**.

Authentication through methods other than the session cookie, such as OAuth or certificates, is not supported. The server uses self-signed certificates.

The procedure to pass the session cookie back and forth from the switch depends on how you access the REST API.

For example:

- If you log in to the REST API using the AOS-CX REST API Reference or using the Web UI and open the API Reference in another browser tab, the browser handles the session cookie for you. You do not have to save or otherwise manage the session cookie.
- If you access the REST API using another method, such as the curl tool, you must do the following:
 - Save the session cookie returned from the login request.
 - Pass that saved cookie to the switch with every subsequent request you make to the REST API, including the `logout` resource.



NOTE

Although it is possible to pass the user name and password information as a query string in the login URL, browser logs or tools outside the switch might save the accessed URL in cleartext in log entries. Instead, Hewlett Packard Enterprise recommends that you pass the credential information as data when using programs such as curl to log in to the switch.

For examples of accessing the REST API using curl, see [Accessing the REST API using curl](#).

User groups and access authorization

For switch resources, the access authorization granted to a user is determined by the group to which the user belongs. Each user group is assigned a number that represents a privilege level. This number is used to represent the user group in logs and in places in which the group name is too long to display.

The following predefined user groups are supported:

User group	Privilege level	Description
<code>operators</code>	1	Authorized for read access to non - sensitive data.
<code>administrators</code>	15	Authorized for read and write access to all switch resources. Write access also requires that the REST API is in read - write access mode.
<code>auditors</code>	19	Authorized for read access to audit log (<code>/logs/audit</code>) and event log (<code>/logs/event</code>) resources only.

All users can access the POST method of the `login` and `logout` resources. However, the login requests fail if the user is not a member of one of the predefined user groups. For example, login attempts fail when attempted by a member of a user-defined local user group.

If a user attempts a request for which they are not authorized, the switch returns an HTTP 403 "Forbidden" error.

If an authorized user attempts a write request but the REST API is in read-only mode, the switch returns an HTTP 404 "Page not found" error.

AOS-CX REST API Reference (UI)

The AOS-CX operating system includes the AOS-CX REST API Reference, which is a web interface based on the Swagger 3.0 UI. For more information about Swagger, see <https://swagger.io/>.

The AOS-CX REST API Reference provides the reference documentation for REST API, including the switch resources, parameters, errors, and JSON models for each HTTPS method supported by the resource. The AOS-CX REST API Reference shows most of the supported read and write methods for all switch resources.

Since the AOS-CX REST API Reference is browser-based, it can share the session cookie with a Web UI session active in another browser tab. The AOS-CX REST API Reference is not intended to be used as a configuration tool and is not required for day-to-day operations.

The AOS-CX REST API Reference is one way to execute HTTP requests like GET, PUT, POST, PATCH, and DELETE, without writing a script. The AOS-CX REST API Reference can be used during script coding to help you construct the URIs and data body (in the case of PATCH, POST, or PUT)—with their query parameters—that you use in a script or curl command.

Subtopics

Accessing the REST API using the AOS-CX REST API Reference

AOS-CX REST API Reference basics

Write methods (POST, PUT, PATCH, and DELETE)

GET method usage and considerations

POST method usage and considerations

PUT method usage and considerations

PATCH method usage and considerations

DELETE method usage and considerations

REST requests and accounting logs

AOS-CX REST API reference summary

Accessing the REST API using the AOS-CX REST API Reference

Although the AOS-CX REST API Reference interacts directly with the REST API, the AOS-CX REST API Reference is not intended as a management or configuration interface. Use caution when using the **Submit** button for POST, PATCH, or PUT methods because this action can result in changes to your current environment.

Prerequisites

- HTTPS server access must be enabled on the VRF from which you are accessing the switch.
- With a few exceptions, using the PUT, POST, PATCH, or DELETE methods require the following conditions to be true:
 - The REST API access mode must be set to `read-write`.
 - The user name you use to log in must be a member of the `administrators` group.

Procedure

1. To view the reference documentation for the REST v10.xx API, access the following URL using a browser: `https://<IP-ADDR>/api/v10.xx/`
<IP-ADDR> is the IP address or hostname of your switch.

For example: **<https://192.0.2.5/api/v10.xx/>**

Subtopics

Logging in and logging out using the AOS-CX REST API Reference

Prerequisites

- Access to the switch REST API must be enabled.
- You must have used a supported browser to access the switch at:

```
https://<IP-ADDR>/api/v10.xx/
```

<IP-ADDR> is the IP address or hostname of your switch.

Procedure

1. Log in to the switch using the **Login** resource:
 - a. Expand the **Login** section.
The POST method for the login resource is displayed.
 - b. Expand the resource by clicking POST or the resource name, /login.
 - c. Click **Try it out**.
 - d. Enter your user name in the **User name** field.
 - e. Enter your password in the **Password** field.
 - f. Click **Execute**.

If the operation is successful, the REST API returns response code 200.

2. When you finish your session, log out by expanding the **Logout** resource and clicking **Execute**.

AOS-CX REST API Reference basics

This section provides information about the different components of the AOS-CX REST API user interface.

Subtopics

[AOS-CX REST API Reference home page](#)

AOS-CX REST API Reference home page

The following is an example of a portion of the AOS-CX REST API Reference home page for a switch running AOS-CX software:

aruba
a Hewlett Packard
Enterprise company

ArubaOS-CX REST API 10.04 OAS3

<https://10.100.114.176/api/v10.04/restd.json>

RESTful Interface for ArubaOS-CX switch software

Servers

- AAA_Accounting_Attributes >
- AAA_Server_Group >
- AAA_Server_Group_Prio >
- ACL >

- The link at the top of the page displays the JSON representation of the RESTful interface.
- The **Servers** drop-down lists the base URL to access the REST API.
- The switch resource URIs are organized in groups. The group names are listed in alphabetical order on the AOS-CX REST API Reference home page.
The group name does not always match the resource collection name. Use the group names as a navigation aid only.
- Group names that are in gray have the URI entries—also called endpoints—collapsed. When you hover over the group name, it turns black. Click the group name to expand it and show the list of methods and URIs in the group.

The following example shows the list of the methods and URIs in the **Subsystem** group:

Subsystem

GET

/system/subsystems

GET

/system/subsystems/{type},{name}

PUT

/system/subsystems/{type},{name}

- The methods that are shown might depend on the REST API access mode. Some methods might not be displayed if the REST API access mode is `read-only`.
- Methods and resources might be displayed that you do not have the authorization to access. For example, users with operator rights are not authorized to make PATCH, PUT, or POST requests to most resources. If you submit a request for which you are not authorized, the switch returns the following error: HTTP error 403 "Forbidden"
- The resource collection name is `subsystems` (not Subsystem).
- Items in braces, such as `{type}` and `{name}`, are path parameters. If you submit a request to a resource URI that includes a path parameter, you are required to supply a value for the parameter.

To show more information about an item on the list, click the URI path. The following example shows a part of the information displayed when GET on `/system/subsystems` is selected:

Subsystem

GET

/system/subsystems

Parameters

Try it out

Name	Description
------	-------------

attributes array[string] (query)	Columns to display. <i>Available values</i> : poe_power, software_images, alias, interfaces, data_plane_connectivity_target_state, product_info, diagnostic_disable, entity_state, usb_status, resource_width_per_feature, macs_remaining, max_power_required, policy_init_status, temp_sensors, capacities, resets_requested, diagnostic_requested, resource_utilization_per_feature, resets_performed, name, storage, selftest, part_number_cfg, thermal_state, poe_persistence_disable, psu_redundancy_set, admin_state, capabilities, selftest_disable, asic_info, state, power_priority, leds, data_plane_target_state, diagnostic_last_run_timestamp, data_plane_state, resource_capacity, data_plane_connectivity_state, resource_utilization, fans, data_plane_error, acl_init_status, resource_reservation_per_feature, psu_redundancy_oper, diagnostic_performed, lossless_pool_percent, power_granted, resource_unreserved, type, power_supplies, power_consumed, next_mac_address, fan_configuration_state, buttons, daemons, diag_test_results
depth integer (query)	Depth to traverse.

You can use the browser scroll bar to navigate to information about the implementation of this method and resource, including the required and optional parameters. You must click **Try it out** to edit the parameters.

- The required parameters are shown with *** required**.

For example, the POST method of the login resource requires a user name and password:

Login

POST

/login User login.

Login to device using username and password.

Parameters

Cancel

Name	Description
------	-------------

username * required
string
(query)

User name

username - User name

password * required
string(\$password)
(query)

Password

password - Password

Execute

- Path parameters, such as `{id}`, are listed as required parameters:

Parameters

Name	Description
id * required string (path)	The ID of this VLAN. 10

- The **Execute** button sends the request. Click **Cancel** to exit the edit mode without sending the request.



CAUTION

Although the AOS-CX REST API Reference interacts directly with the REST API, the AOS-CX REST API Reference is not intended as a management or configuration interface. Use caution when using the **Execute** button for PATCH, POST, or PUT methods because this action can result in changes to your current environment.

In GET requests, there can be multiple attributes and parameters you can use to filter results.

For example:

attributes
 array[string]
 (query)

Columns to display.

--
 mgmd_querier_ip
 aclmac_in_applied
 mgmd_dynamic_group_count

You can select multiple attributes:

- To select a range of attributes, click the first attribute, then press **Shift**, and then click the last attribute in the range you want to select.
- To select attributes that are not adjacent in the list, press **Ctrl**, then click each attribute you want to select.

The JSON model for the resource is described in **Model** and shown with example values in **Example Values** for each method. The following example shows the JSON model and example values for PUT method of the `/system/subsystems/{type}/{name}` resource:

Example Value Model

```
{  
  "admin_state": "diagnostic",  
  "diagnostic_disable": true,  
  "interface_group_speed": {  
    "1": "10g",  
    "2": "10g",  
    "3": "10g",  
    "4": "10g"  
  },  
  "part_number_cfg": "string",  
  "psu_redundancy_set": "none",  
  "selftest_disable": true  
}
```

Edit Value

Model

```
{  
  "admin_state": "diagnostic",  
  "diagnostic_disable": true,  
  "part_number_cfg": "string",  
  "poe_persistency_disable": true,  
  "power_priority": 0,  
  "psu_redundancy_set": "none",  
  "selftest_disable": true  
}
```

After a request is submitted, the AOS-CX REST API Reference shows additional information, including the following:

- The curl command equivalent of the submitted request
- The submitted request URL, including the specified parameters and values.
- The response body returned by the switch
- The response code returned by the switch
- The response headers returned by the switch

The curl command and request URLs are displayed using percent encoding for certain characters in the query string portion of the URL:

Character	Percent - encoded equivalent
, (comma)	%2C
:	%3A
/ (forward slash)	%2F

When you enter curl commands or submit requests through other means, percent encoding is permitted but not required in the query string of the URI.

Write methods (POST, PUT, PATCH, and DELETE)

The supported write methods are POST, PUT, PATCH, and DELETE:

- POST creates a resource.
- PUT replaces a resource.
- PATCH updates a resource.
- DELETE removes a resource.

Not all resources support all write methods. See the AOS-CX REST API Reference for the methods supported by each resource. The REST API must be in read-write mode for the AOS-CX REST API Reference to show all the write methods a resource supports.

Subtopics

Considerations when making configuration changes

Considerations for ports and interfaces

Write methods (POST, PUT) supported in read-only mode

Considerations when making configuration changes

The REST API can access and change every configurable aspect of the switch as modeled in the configuration and state database. However, changing the configuration of a switch through the REST API can be different than changing the configuration through the CLI.

A single configuration change to the switch can require changes to multiple resources in the configuration and state database. Often these changes must be made in a specific order.

The CLI commands have been programmed to work "behind the scenes" to make the correct database changes and to perform data validation checks on the user input. In contrast, when you use the REST API to make a configuration change, you must become familiar with the representational models of the switch resources, the type and format of the data required, and the required order of write operations to various resources.

The REST API is powerful but must be used with extreme caution: No semantic validation is performed on the data you write to the database, and configuration errors can destabilize the switch. Hewlett Packard Enterprise recommends that you refer to the tested examples when using the REST API to make configuration changes.

Considerations for ports and interfaces

The REST API provides the `interfaces` resource to configure and get information about switch ports and interfaces of all types. You do not use the `ports` resource to manage ports.

Subtopics

Hardware (system) interfaces

LAG interfaces

VLAN interfaces

Hardware (system) interfaces

- Hardware interfaces are of type `system`.
- Hardware interfaces are included in the database automatically.
- Interfaces of type `system` cannot be added or deleted.

LAG interfaces

- LAG interfaces are of type `lag`.
- You can use the DELETE method to delete a LAG interface.

Example of creating a LAG interface with member ports 1/1/1 and 1/1/2:

Method and URI:

```
POST "/rest/v10.xx/system/interfaces"
```

Request body:

```
{
  "name": "lag50",
  "vrf": "/rest/v10.xx/system/vrfs/default",
  "type": "lag",
  "interfaces": [
    "/rest/v10.xx/system/interfaces/1%2F1%2F1",
    "/rest/v10.xx/system/interfaces/1%2F1%2F2"
  ]
}
```

VLAN interfaces

- VLAN interfaces are of type `vlan`.
- You can use the DELETE method to delete a VLAN interface.

Example of creating a VLAN interface:

Method and URI:

```
POST "/rest/v10.xx/system/interfaces"
```

Request body:

```
{
  "name": "vlan2",
  "vlan_tag": "/rest/v10.04/system/vlans/2",
  "vrf": "/rest/v10.04/system/vrfs/default",
  "type": "vlan"
}
```

Write methods (POST, PUT) supported in read-only mode

The following switch resources support write methods (POST, PUT, or both) even when the REST API access mode is set to read-only:

- Configuration management: `*/rest/v10.xx/fullconfigs*`
- Firmware: `*/rest/v.10xx/firmware*`
- User login and logout:
 - `*/rest/v.10xx/login`
 - `*/rest/v.10xx/logout`

- Aruba Network Analytics Engine and scripts: `*/rest/v10.xx/system/nae_scripts*`

The * indicates more text to be added in URI path.

GET method usage and considerations

The GET method is a read method that gets the resource specified by the URI. Data is returned in JSON format in the response body.

Using GET on a resource collection results in a list of URIs. Each URI in the list corresponds to a specific resource in the collection.

Using GET on a specific resource returns the attributes of that resource.

Subtopics

GET method parameters

GET method parameters

The GET method supports the following parameters in the query string of the URI:

- `attributes`
- `count`
- `depth`
- `filter`
- `selector`

A path query parameter is specified as a "key=value" pair. When permitted, multiple values are separated by the comma (,) character.

For example:

- `attributes=id,name,type`
- `count=true`
- `depth=2`
- `filter=type:static`
- `selector=writable`

A path query parameter can be used alone or in combination with other parameters. The ampersand (&) character separates each parameter in the string.

For example:

```
GET "https://192.0.2.5/rest/v10.09/system/vlans?
depth=2&attributes=id,name,type"
```

The `count` and `filter` attributes and wildcard character are supported from AOS-CX release 10.05 and later.

Subtopics

Wildcard character support

Attributes parameter

Count parameter

Depth parameter

Filter parameter

Selector parameter

Wildcard character support

When you use the GET method, the URI can contain the asterisk (*) wildcard character instead of a component in URI path. You can use wildcard characters in multiple places in the path. You cannot use a wildcard character as part of the query string.

The wildcard character must replace the entire component in the path. Regular expressions are not supported. For example, you can use a wildcard to specify all VRFs, but you cannot use a regular expression to specify all VRFs that begin with the letter `r`.

By using a wildcard character in place of a component in the path, you can specify that GET return information about multiple resources without requiring you to name each resource instance or to execute multiple GET requests.

For example:

- The following URI specifies all routes regardless of VRF:
"https://192.0.2.5/rest/v10.xx/system/vrfs/*/routes"
- The following URI specifies all ACL entries of type IPv4, regardless of the name of the ACL:
"https://192.0.2.5/rest/v10.xx/system/acls/*,ipv4/cfg_aces"
- The following URI specifies the connection state of all BGP neighbors belonging to all BGP routers in the "red" VRF:
"https://192.0.2.5/rest/v10.xx/system/vrfs/red/bgp_routers/*/bgp_neighbors/*?attributes=status"

Attributes parameter

Syntax

The **attributes** query parameter consists of a comma-separated list of attribute names related to a collection or a specific resource. It allows the user to retrieve specific attributes in the request instead of returning the whole set of attributes for the resource, which is the default behavior if the query parameter is not specified.

Behavior

It can be used in a collection, a specific resource, and in a request including wildcards. If the request for any resource includes an attribute that does not exist, a bad request (HTTP 400) will be returned and the request will fail. There is no limit on the number of attributes that can be specified in the request.

Use case

It allows for more efficient data retrieval and can reduce the amount of data transferred over the network. It reduces CPU and memory consumption, especially in cases with resources that contain many attributes and high amount of data, which also improves the response time to the user.

Examples

Valid specific Interface attribute list

```
GET https://<IP>/rest/<version>/system/interfaces/ubt?
attributes=arp_timeout,ip_mtu,mvrp_enable,interfaces
{
  "arp_timeout": 1800,
  "interfaces": {
    "ubt": "/rest/latest/system/interfaces/ubt"
  },
  "ip_mtu": 1500,
  "mvrp_enable": false
}
```

Valid VLAN collection attribute list

```
GET https://<IP>/rest/<version>/system/vlans/1?attributes=name,type,admin
{
  "admin": "up",
  "name": "DEFAULT_VLAN_1",
  "type": "default"
}
```

Count parameter

Syntax

The **count** parameter indicates the request wants to obtain the total number of entries related to a resource. A successful response contains a JSON format with the **count** key and a number as the value.

It works with requests including wildcard and other query parameters. Omitting it in the request is equivalent to specifying **count=false**. Including it in the request as **count=** is equivalent to specifying **count=true**. A request with **count** to a specific resource is valid and will return 1.

Use Cases

The user gets the number of entries instead of the values of entries. Requests to dynamic or protected resources are not limited.

Limitations

It is not supported for notifications.

Examples

Single resource

A request with a count to a specific resource is valid and will return **1**.

```
GET https://<IP>/rest/<version>/system/vrfs/default?count=true
{
  "count": 1
}
```

Collection

The database has **2** BGP routers under the default VRF. This configuration will return 2.

```
GET https://<IP>/rest/<version>/system/vrfs/default/bgp_routers?count=true
{
  "count": 2
}
```

Wildcard

The database has 2 BGP Routers under the **default** VRF and 1 under **mgmt** VRF. First **2** BGP routers have **2** aggregate addresses, the other one has **1** address. This configuration will return **5**.

```
GET https://<IP>/rest/<version>/system/vrfs/*/bgp_routers/*/
aggregate_addresses/*,*?count=true
{
  "count": 5
}
```

Filter

The database has 2 BGP Routers under the **default** VRF and 1 under **mgmt** VRF. First 2 BGP routers have **trap_enable** set as **true**, the other one as **false**. This configuration will return **1**.

```
GET https://<IP>/rest/<version>/system/vrfs/default/bgp_routers?
count=true&filter=trap_enable:false
{
  "count": 1
}
```

Interfaces

The database has **54** interfaces (**1/1/1-52**), a UBT interface and a LAG member. This configuration will return **54**.

```
GET https://<IP>/rest/<version>/system/interfaces?count=true
{
  "count": 54
}
```

Invalid count value

```
GET https://<IP>/rest/<version>/system/vrfs/*/bgp_routers/*/
aggregate_addresses/*,*?count=invalid
Invalid value for 'count' query parameter. Valid values are 'true' or none,
and 'false'.
```

Depth parameter

Syntax

The **depth** query parameter is used to specify how many levels of URIs should be expanded and replaced with their corresponding JSON representation in the response body. As each level of depth is expanded, the REST API will add one level of URIs into the response body.

Syntax

```
depth=N
```

N is an integer. The value should be greater than or equal to 1.

Behavior

When the **depth** query parameter is not specified, it uses a default value of **1**. Non-expanded references are shown as URIs to the corresponding tables following the Hypermedia as the Engine of Application State (HATEOAS) REST principle.

If a resource can be expanded into **N** layers deep, and the specified **depth** value is **N+M**, the **M** additional expansions will be ignored.

Use case

The **depth** parameter is used to control the amount of referenced objects retrieved. Thus obtaining more information when a given resource has internal references to others.

Limitations

High depth values (≥ 3) are not restricted but can have a significant impact on the CPU and memory usage.

It is important to note that using the depth parameter can result in a large amount of data being returned, especially if there are many items in the list and each item contains a significant amount of JSON data.

To avoid this problem, analyze if instead of using a high depth, the required data can be accessed through a direct REST request to the resource.

Alternatively, consider utilizing a combination of depth and attributes query parameters to filter down the needed columns.

Examples

No depth specified (default : 1)

```
GET https://<IP>/rest/<version>system/users
{
  "admin": "/rest/latest/system/users/admin"
}
```

Depth : 0

```
GET https://<IP>/rest/<version>system/users?depth=0
invalid value for 'depth' query parameter
```

Depth : 1

```
GET https://<IP>/rest/<version>system/users?depth=1
{
  "admin": "/rest/latest/system/users/admin"
}
```

Depth : 2

```
GET https://<IP>/rest/<version>system/users?depth=2
{
  "admin": {
    "authorized_keys": {},
    "current_password": "",
    "is_user_lockedout": false,
    "login_failure_attempts": 0,
    "name": "admin",
    "origin": "built-in",
  }
}
```

```

    "password":
"AQBapbVXeRc4bwfvtySxqiJ0AJ10jiqeV1z9lJUv7zJpzHVMYgAAABSHgjLoTr4B15Nu4hqLN21
jeNpM8i901YDB7n2OLSS/
MhGiPm4PZcrv3tNv38ZiEnQ9Sa6IIrOIbr109NfaO+C3KEVWiG3uSeeGmeq8H0N4KZvx0JTlfaph
JtCvsgLw0zTQ",
    "user_group": {
        "administrators": "/rest/latest/system/user_groups/
administrators"
    }
}

```

Depth: 3

```

GET https://<IP>/rest/<version>system/users?depth=3
{
  "admin": {
    "authorized_keys": {},
    "current_password": "",
    "is_user_lockedout": false,
    "login_failure_attempts": 0,
    "name": "admin",
    "origin": "built-in",
    "password":
"AQBapbVXeRc4bwfvtySxqiJ0AJ10jiqeV1z9lJUv7zJpzHVMYgAAABSHgjLoTr4B15Nu4hqLN21
jeNpM8i901YDB7n2OLSS/
MhGiPm4PZcrv3tNv38ZiEnQ9Sa6IIrOIbr109NfaO+C3KEVWiG3uSeeGmeq8H0N4KZvx0JTlfaph
JtCvsgLw0zTQ",
    "user_group": {
      "administrators": {
        "inherit_rules": null,
        "name": "administrators",
        "origin": "built-in",
        "rbac_rules": "/rest/latest/system/user_groups/
administrators/rbac_rules"
      }
    }
  }
}

```

Depth: 4

```

GET https://<IP>/rest/<version>system/users?depth=4
{
  "admin": {
    "authorized_keys": {},
    "current_password": "",

```

```

        "is_user_lockedout": false,
        "login_failure_attempts": 0,
        "name": "admin",
        "origin": "built-in",
        "password":
"AQBapbVXeRc4bwfvtySxqiJ0AJ10jiqeV1z9lJUv7zJpzHVMYgAAABSHgjLoTr4B15Nu4hqLN21
jeNpM8i90lYDB7n2OLSS/
MhGiPm4PZcrv3tNv38ZiEnQ9Sa6IIrOIbr109NfaO+C3KEVWiG3uSeeGmeq8H0N4KZvx0JTlfaph
JtCvsgLw0zTQ",
        "user_group": {
            "administrators": {
                "inherit_rules": null,
                "name": "administrators",
                "origin": "built-in",
                "rbac_rules": {}
            }
        }
    }
}

```

Depth : 99

```

GET https://<IP>/rest/<version>system/users?depth=99
{
    "admin": {
        "authorized_keys": {},
        "current_password": "",
        "is_user_lockedout": false,
        "login_failure_attempts": 0,
        "name": "admin",
        "origin": "built-in",
        "password":
"AQBapbVXeRc4bwfvtySxqiJ0AJ10jiqeV1z9lJUv7zJpzHVMYgAAABSHgjLoTr4B15Nu4hqLN21
jeNpM8i90lYDB7n2OLSS/
MhGiPm4PZcrv3tNv38ZiEnQ9Sa6IIrOIbr109NfaO+C3KEVWiG3uSeeGmeq8H0N4KZvx0JTlfaph
JtCvsgLw0zTQ",
        "user_group": {
            "administrators": {
                "inherit_rules": null,
                "name": "administrators",
                "origin": "built-in",
                "rbac_rules": {}
            }
        }
    }
}

```

```
}  
}
```

Filter parameter

Syntax

The **filter** parameter provides additional filtering capabilities. This query parameter filters rows and entries depending on the value of one or more attributes.

Syntax

```
filter=type:value
```

: is ascii **%3A** and , is ascii **%2C**

Single filter:

```
...&filter=type:vlan
```

Multiple filters:

```
...&filter=type:vlan,admin_state:up
```



NOTE

REST handles multiple filters with an AND logic. It means only rows which satisfy both filters are retrieved.

Behavior

If a **filter** is specified, the response will only include rows in which the value for the specified attribute matches the specified value. Combining **filter** query parameters with other query parameters such as **depth** or **attribute** is valid.

Use case

This query parameter allows the user to retrieve rows where one or more of its columns matches a specified value. This allows more efficient data retrieval and a reduction of CPU and memory consumption on scenarios where the resource contains a high quantity of rows. Additionally, response time is improved since less data needs to be retrieved and transferred over the network.

Limitations

Filters are optional and support only integer, boolean, and string types.

Examples

No filter

```
GET https://<IP>/rest/<version>/system/vrfs/VRF_1/static_routes/
190.1.1.0%2F24/static_nexthops?depth=2
{
  "0": {
    "bfd_enable": false,
    "distance": 1,
    "id": 0,
    "ip_address": null,
    "port": null,
    "tag": null,
    "type": "nullroute",
    "use_forwarding_address": false
  },
  "1": {
    "bfd_enable": true,
    "distance": 2,
    "id": 1,
    "ip_address": null,
    "port": null,
    "tag": null,
    "type": "nullroute",
    "use_forwarding_address": false
  },
  "2": {
    "bfd_enable": false,
    "distance": 2,
    "id": 2,
    "ip_address": null,
    "port": null,
    "tag": null,
    "type": "forward",
    "use_forwarding_address": false
  },
  "3": {
    "bfd_enable": true,
    "distance": 2,
    "id": 3,
    "ip_address": null,
    "port": null,
    "tag": null,
    "type": "forward",
    "use_forwarding_address": false
  }
}
```

```
}  
}
```

bfd_enable: false

```
GET https://<IP>/rest/<version>/system/vrfs/VRF_1/static_routes/  
190.1.1.0%2F24/static_nextops?depth=2&filter=bfd_enable:false  
{  
  "0": {  
    "bfd_enable": false,  
    "distance": 1,  
    "id": 0,  
    "ip_address": null,  
    "port": null,  
    "tag": null,  
    "type": "nullroute",  
    "use_forwarding_address": false  
  },  
  "2": {  
    "bfd_enable": false,  
    "distance": 2,  
    "id": 2,  
    "ip_address": null,  
    "port": null,  
    "tag": null,  
    "type": "forward",  
    "use_forwarding_address": false  
  }  
}
```

bfd_enable: true

```
GET https://<IP>/rest/<version>/system/vrfs/VRF_1/static_routes/  
190.1.1.0%2F24/static_nextops?depth=2&filter=bfd_enable:false  
{  
  "1": {  
    "bfd_enable": true,  
    "distance": 2,  
    "id": 1,  
    "ip_address": null,  
    "port": null,  
    "tag": null,  
    "type": "nullroute",  
    "use_forwarding_address": false  
  },  
  "3": {  
    "bfd_enable": true,  
    "distance": 2,  
    "id": 3,  
    "ip_address": null,  
    "port": null,  
    "tag": null,  
    "type": "forward",  
    "use_forwarding_address": false  
  }  
}
```

```

        "distance": 2,
        "id": 3,
        "ip_address": null,
        "port": null,
        "tag": null,
        "type": "forward",
        "use_forwarding_address": false
    }
}

```

type: forward

```

GET https://<IP>/rest/<version>/system/vrfs/VRF_1/static_routes/
190.1.1.0%2F24/static_nextops?depth=2&filter=type:forward
{
  "2": {
    "bfd_enable": false,
    "distance": 2,
    "id": 2,
    "ip_address": null,
    "port": null,
    "tag": null,
    "type": "forward",
    "use_forwarding_address": false
  },
  "3": {
    "bfd_enable": true,
    "distance": 2,
    "id": 3,
    "ip_address": null,
    "port": null,
    "tag": null,
    "type": "forward",
    "use_forwarding_address": false
  }
}

```

Multiple filters

```

GET
https://<IP>/rest/<version>/system/vrfs/VRF_1/static_routes/190.1.1.0%2F24/
static_nextops?depth=2&filter=type:forward&filter=bfd_enable:true
OR
GET https://<IP>/rest/<version>/system/vrfs/VRF_1/static_routes/
190.1.1.0%2F24/static_nextops?depth=2&filter=type:forward,bfd_enable:true
{
  "3": {

```

```

        "bfd_enable": true,
        "distance": 2,
        "id": 3,
        "ip_address": null,
        "port": null,
        "tag": null,
        "type": "forward",
        "use_forwarding_address": false
    }
}

```

Unrecognized filter

```

GET https://<IP>/rest/<version>/system/vrfs/VRF_1/static_routes/
190.1.1.0%2F24/static_nexthops?depth=2&filter=id:0
filter 'id' is unknown

```

Non-matching type filter

```

GET https://<IP>/rest/<version>/system/vrfs/VRF_1/static_routes/
190.1.1.0%2F24/static_nexthops?depth=2&filter=tag:null
filter 'tag' value must be of type 'integer'

```

Selector parameter

Syntax

REST API provides the use of selectors to allow filtering over an attributes category.

Syntax

```
selector=selector type
```

The valid selectors are the following:

```
configuration
```

Filters by read/write attributes (category type configuration).

```
statistics
```

Filters by read-only attributes which hold numerical values such as counters (category type statistics).

```
status
```

Filters by non-statistic read-only attributes (category type status).

```
writable
```

Filters by attributes that are mutable and can only be modified in PUT & PATCH operations. This selector is not allowed on collections.



NOTE

- Certain **statistics** fields are not updated in real time, and are instead updated during preset intervals.
- The category of certain attributes may change depending on the configuration of the switch. For example, for different resources of the same type (ex. VLAN1 vs VLAN200) some of its attributes could be either status (which is read-only) or configuration (which is read-write).

Behavior

If a **selector** is specified, the response will exclude attributes whose category does not match the specified selector. Combining selector query parameters with other query parameters such as **depth** or **attribute** is valid. An empty selector value is allowed but ignored; no filtering will occur.

Use case

Selectors allow for more efficient data retrieval and can reduce the amount of data transferred over the network.

Limitations

Selectors are not supported in notifications.

Examples

User Table:

```
"name" : follows -> origin
"origin" : per-value { "built-in" : status, "configuration" :
configuration }
"password" : configuration
"current_password" : volatile
"user_group" : follows -> origin
"login_failure_attempts" : status
"is_user_lockedout" : status
"authorized_keys" : configuration
```

Status Selector:

```
GET https://<IP>/rest/<version>/system/users/admin?selector=status
{
  "is_user_lockedout": false,
```

```

    "login_failure_attempts": 0,
    "name": "admin",
    "origin": "built-in",
    "user_group": {
        "administrators": "/rest/latest/system/user_groups/administrators"
    }
}

```

Configuration Selector:

```

GET https://<IP>/rest/<version>/system/users/admin?selector=status
{
    "authorized_keys": {},
    "current_password": "",
    "password": "AQBapT9vR8JqxjB7VsaMxY2c9Fe0pBR8dzeBm3XkKds6XsxXYgAAAM/
PA8DTxwbZ3+ToBzHyZEA43L5XECFqDltyfWSXPfG5gfh5sfIjRbRTiMjYVABELDhsC0atI3IufvY
DOCQLebsIy5a2+wQ3/r8haPrjuIFidWqca2dPJq+vXQE1B0Db3K0+"
}

```

Statistics Selector:

```

GET https://<IP>/rest/<version>/system/users/admin?selector=statistics
{}

```

Invalid Selector value:

```

GET https://<IP>/rest/<version>/system/users/admin?selector=invalid
invalid value for 'selector' query parameter. Valid values are
'configuration', 'status', 'statistics', 'writable' or none

```

Writable on collections:

```

GET https://<IP>/rest/<version>/system/vrfs?selector=writable
Writable selector is not allowed with resource collections

```

Multiple Selector :

```

GET https://<IP>/rest/<version>/system/vrfs?
selector=configuration&selector=status

```

In 10.12 or earlier versions, first parameter takes priority:

```

{
    "authorized_keys": {},
    "current_password": "",
    "password": "AQBapT9vR8JqxjB7VsaMxY2c9Fe0pBR8dzeBm3XkKds6XsxXYgAAAM/
PA8DTxwbZ3+ToBzHyZEA43L5XECFqDltyfWSXPfG5gfh5sfIjRbRTiMjYVABELDhsC0atI3IufvY
DOCQLebsIy5a2+wQ3/r8haPrjuIFidWqca2dPJq+vXQE1B0Db3K0+"
}

```

In 10.13 or later versions, selector accepts only one value:

```

invalid request: selector accepts just one value

```

POST method usage and considerations

The POST method creates an instance of a resource in the collection specified by the URI:

- Not all resources support the POST method. See the AOS-CX REST API Reference for the methods supported by each resource. The REST API must be in read-write mode to see all the POST methods supported.
- Some resources support the POST method even when the REST API is in read-only mode.
- When you use the POST method, the URI must point to the collection—not to the resource you are creating. The resource you are creating is sent in JSON format in the request body.
 - The JSON representation must include all fields required by the JSON model of that resource.
 - The JSON representation can contain only configuration attributes. It must not contain attributes in the `status` or the `statistics` category.
- You can POST only one resource at a time.
- Most resources have a hierarchical relationship. You must create the parent before you can create the child.

For example, to create an ACL entry:

1. The ACL must be created first by sending the JSON data of the ACL in the request body in a POST request to the URI of the acls collection:
`/system/acls`
2. The entry can then be created by sending the JSON data of the entry in the request body in a POST request to the URI of the ACL:
`/system/acls/<ACL-name>,<ACL-type>/cfg_aces`

The following is an example of using the POST method to create a subinterface instance:

```
POST "https://{mgmt-ip}/rest/v10.11/system/interfaces"
{
  "name": "1/1/3.1026",
  "type": "vlansubint",
  "routing": true,
  "subintf_parent": "/rest/latest/system/interfaces/1%2F1%2F3",
  "subintf_vlan": 1026,
  "ip4_address": "10.5.2.1/24",
```

```
"admin": "up"
}
```

**NOTE**

For more information about subinterfaces, refer to the Fundamentals Guide .

PUT method usage and considerations

The PUT method updates an instance of a resource by replacing the existing resource with the resource provided in the request body.

Configuration attributes that are set at the time a resource is created and that cannot be changed afterward are called **immutable** attributes. Configuration attributes that can be changed after a resource is created are called **mutable** or **writable** attributes. The PUT method is used replace writable attributes only.

- Not all resources support the PUT method. For information about the methods supported for a resource, see the AOS-CX REST API Reference. The REST API must be in `read-write` mode to see all the PUT methods supported.
- The URI must specify a specific resource, not a collection.
- The URI must specify a resource that currently exists.
- For almost all resources, the PUT method is implemented as a strict replace operation. All mutable configuration attributes are replaced. Any mutable attribute that the JSON data in request body does not include is either removed (if there is no default value) or reset to its default value.

Subtopics

PUT request body requirements

PUT behavior

Exceptions to the PUT strict replace behavior

Best practice for building the PUT request body

PUT request body requirements

The JSON data in the request body must include mutable (writable) configuration attributes only.

The JSON model used for the PUT method request body is different from the JSON model used for the GET or the POST method.

The JSON model of a PUT method for a resource contains the mutable attributes only. In contrast, the JSON models for GET and POST methods can include both mutable and immutable attributes.

See the AOS-CX REST API Reference for the JSON model of a PUT method for a resource.

PUT behavior

The PUT operation is a replace operation—not an update operation—because the resource instance in the request body replaces every changeable configuration attribute of the existing resource. To update specific fields, use the PATCH operation.



CAUTION

Any mutable attribute that the JSON data in request body does not include is either removed (if there is no default value) or reset to its default value.

For example:

- If you attempt a PUT operation on the System resource to change the host name, and you supply only the host name, you will destabilize the switch because the other attributes will be reset to their defaults, which might be empty.
- If you attempt to change the name of a VLAN and supply only the name in the PUT request, every other attribute in that VLAN is set to its default of empty.

Exceptions to the PUT strict replace behavior

For Network Analytics Engine agents, the PUT behavior is not a strict replace implementation. You can enable or disable agents without the supplying the entire set of configuration attributes in the PUT request body. For more information about the Network Analytic Engine resources, see the Network Analytics Engine Guide .

Best practice for building the PUT request body

About this task

Hewlett Packard Enterprise recommends the following procedure for building the PUT request body.

Procedure

1. Use the GET method with `selector=writable` to obtain the writable (mutable) configuration attributes for the resource you want to change.

For example:

GET "https://192.0.2.5/rest/v10.xx/system/interfaces/vlan200?selector=writable"

2. Change the values of the attributes to match your wanted configuration. Any attribute you do not include in the request body will be set to its default value, which could be empty.
3. Use the resulting JSON data as the request body for the PUT request.

PATCH method usage and considerations

The PATCH method is an HTTP method that updates values in an existing resource using only the desired values in the request body.

- The PATCH method does not require fetching the current resource in order to send a new request, as the PUT method requires, simplifying automation and reducing bandwidth usage.
- The PATCH method is used to add JSON keys or array elements, but cannot be used to remove JSON keys or array elements.
- This method is only supported by the system and the resources under it. Not all resources support the PATCH method.

The REST API must be in `read-write` mode to see all the PATCH methods supported.



NOTE

PATCH is only available on versions REST v10.04 or later.

DELETE method usage and considerations

The DELETE method deletes an instance of a resource.

- Not all resources support the DELETE method. See the AOS-CX REST API Reference for the methods supported by each resource. The REST API must be in `read-write` mode to see all the DELETE methods supported.
- The URI must specify a specific resource instance. The URI must not specify a collection.
- Child subcollections and resources are deleted when you delete the parent resource. For example, if you delete an ACL, its ACL entries are deleted automatically.
- DELETE requests do not contain a request body.

- DELETE requests do not return a response body.

REST requests and accounting logs

All REST requests—including GET requests—are logged to the accounting (audit) log.

The URI of the REST API resource for accounting logs is the following:

```
/rest/v10.xx/logs/audit
```

In an accounting log entry for a REST request:

- `service=https-server` indicates that the log entry is a result of a REST API request or a Web UI action.
- The string value of `data` identifies the REST API request that was executed.

For more information about accounting log entries, see the description of the `show accounting log` CLI command.

AOS-CX REST API reference summary

The following information is intended as a quick reference for experienced users. Values are not configurable unless noted otherwise.

Subtopics

Switch REST API access default

Switch REST API default access mode

Enabling access to the Web UI and REST API

Setting the REST API access mode to read-write

Showing the REST API access configuration

AOS-CX REST API Reference URL:

REST API versions and switch software versions

Getting REST API version information from a switch

Protocol

Port

Request and response body format

Session idle timeout

Session hard timeout

Authentication

HTTPS client sessions

VSX peer switch access

Switch REST API access default

- **8100, 8320, 8325/8325H/8325P, 8360, 8400, 9300/9300S, 10000 Switch Series:** Enabled on the `mgmt` VRF
- **6200, 6300, 6400 Switch Series:** Enabled on the `mgmt` VRF
- **4100i, 6000, 6100 Switch Series:** Enabled on the `default` VRF

Switch REST API default access mode

Read-write

Enabling access to the Web UI and REST API

CLI command:

```
https-server vrf <VRF-NAME>
```

Example:

```
switch(config)# https-server vrf mgmt
```

Setting the REST API access mode to read-write

CLI command:

```
https-server rest access-mode read-write
```

Example:

```
switch(config)# https-server rest access-mode read-write
```

Showing the REST API access configuration

CLI command:

```
show https-server
```

Example:

```
switch(config)# show https-server
HTTPS Server Configuration
-----
VRF                : default, mgmt
REST Access Mode   : read-write
```

AOS-CX REST API Reference URL:

REST latest API: <https://<IP-ADDR>/api/latest/>

REST v10.16 API: <https://<IP-ADDR>/api/v10.16/>

REST v10.15 API: <https://<IP-ADDR>/api/v10.15/>

REST v10.14 API: <https://<IP-ADDR>/api/v10.14/>

Where <IP-ADDR> is the IP address or hostname of your switch.

Example:

```
https://192.0.2.5/api/v10.04/
```

REST API versions and switch software versions

REST API version	Switch software version
v10.17	AOS-CX 10.17 and later
v10.16	AOS-CX 10.16 and later
v10.15	AOS-CX 10.15 and later
v10.14	AOS-CX 10.14 and later

Getting REST API version information from a switch

Method and URI to get the REST API versions supported on the switch:

```
GET "https://<IP-ADDR>/rest"
```

Where **<IP-ADDR>** is the IP address or hostname of your switch.

Protocol

HTTPS

Port

443

Request and response body format

JSON

Session idle timeout

20 minutes

Session hard timeout

Eight hours, regardless of whether the session is active.

Authentication

Session cookie from successful HTTPS login request.

HTTPS client sessions

- Maximum of 48 sessions per switch.
- Maximum of six concurrent client sessions per user.
- The same session cookie is shared across browser tabs and, depending on the browser, multiple browser windows.
- The same session cookie is not shared across devices and scripts.
For example, if a user logs into the Web UI from a laptop, again with a tablet, and then uses the same user name in a curl command, that user has three concurrent client sessions.

VSX peer switch access

If Virtual Switching Extension (VSX) is enabled on both switches, and the ISL is up, you can access the VSX peer switch from your connected switch. To access the peer VSX switch, insert `/vsx-peer` in the URI path between the server URL and `/rest`. Not supported for login, Web UI, or AOS-CX REST API Reference access. For more information about VSX, see [VSX Peer Switches and REST API Access](#).

For example:

- Accessing a VSX switch:
`https://192.0.2.5/rest/v10.xx/...`
- Accessing its VSX peer switch:
`https://192.0.2.5/vsx-peer/rest/v10.xx/...`

Polling with Event Log Subscriptions

AOS-CX allows customers to use the REST API to get event logs for a set of defined events.

Subtopics

[**Elements of a URI Request Filter**](#)

[**Combining Filters**](#)

[**Event Log Subscription Response messages**](#)

Elements of a URI Request Filter

The URI for these subscription requests include filters comprised of the following elements: **Event ID**, **Priority**, **Interval**, and **Limit**.

Subtopics

Event ID

Priority

Interval

Limit

Event ID

The event ID for a event log filter can be comprised of individual event IDs or a range of event IDs. YOU can add a range of event IDs in the format **<minimum event id>-<maximum event id>**, or add multiple individual event IDs separated by the pipe character (**|**).

For example:

- For a single event: eventid:2001
- For multiple events: eventid:2001|2002|1000
- For a range of events: eventid:2000-2099
- For a filter with both multiple events and a range : eventid:2001|2002|1000|2000-2099|3000

A filter can be comprised of a maximum of 1000 events.

Priority

The priority level in the filter matches the severity level of the event log message.

Value	Severity	Description
0	Emergency	System is unusable
1	Alert	Action must be taken immediately
2	Critical	Critical conditions
3	Error	Error conditions
4	Warning	Warning conditions

Value	Severity	Description
5	Notice	Normal but significant conditions
6	Informational	Informational messages
7	Debug	Debug - level messages

Interval

This value is the time interval in seconds for each notification to be sent. If no value is specified, the default value of 10 seconds is applied, and notifications are sent every 10 seconds. The maximum supported time interval is 600 seconds. A switch can send up to five subscription URLs within the default time range of 10 seconds, and either 225 Subscription URLs (for 4100, 6000 and 6100 Switch series) or 450 Subscription URLs (for all other switch series) during the maximum 600 second interval.

Limit

The limit is the number of message responses allowed within specified time interval. If no limit is included, the filter allows the default maximum of 1000 messages.

Combining Filters

A subscription request can contain multiple filters and multiple values for the same filter. These filters have the following convention:

- A pipe character (|) is used to separate values from the same filter.
- A comma (,) is used to separate the filters from each other.

Examples

The following is an example of a REST API to subscribe to an event log with multiple filters. This REST API will generate an event response for event-ids with 2101 or 2103, or a priority with 3.

```

{
  "type": "subscribe",
  "topics": [
    {

```

```
"name": "/rest/latest/events?filter=eventid:2101|2103,limit:10,priority:3"
}
]
}
...
```

To unsubscribe from event log messages, the unsubscribe message must include the same syntax as the original subscribe message, with just the **"type":"subscribe"** portion changed to **"type":"unsubscribe"**. If the event IDs, priority or interval information in the unsubscribe message is different than the information in the original subscribe message, the unsubscribe operation will fail.

The following example removes the subscription created in the previous example

```
{
  "type": "unsubscribe",
  "topics": [
    {
      "name": "/rest/latest/events?filter=eventid:2101|2103,limit:10,priority:3"
    }
  ]
}
```

The following is an example with interval of 60 seconds:

```
{
  "type": "subscribe",
  "interval": 60,
  "topics": [
    {
      "name": "/rest/latest/events?filter=eventid:8318|8319|8320"
    }
  ]
}
```

This example removes the event subscription in the previous example:

```
{
  "type": "unsubscribe",
  "interval": 60,
  "topics": [
    {
      "name": "/rest/latest/events?filter=eventid:8318|8319|8320"
    }
  ]
}
```

Event Log Subscription Response messages

The notification responses to the event subscription messages include the following parts:

- **type:** Message type.
- **data:** Subscription message information with the initial data of the resource.
- **topicname:** Subscribed topic URI.
- **subscription_id:** Identifier given to the subscription when is created at the subscription event.
- **sequence_number:** Amount of notifications sent for a subscriber and topicname. The initial value is 1.
- **resources:** Contains the resources related to the subscribed URI and their initial data.
- **operation:** This field is always **inserted**.
- **uri:** URI of the resource related to the subscription URI. (The URI can be empty.)
- **values:** key:value pairs of the subscribed resources.
- **subscriber_name:** Identifier given to the subscriber when the WS connection was started.

The following is an example of a response is sent to hpe-restd in json format.

```
{
  "type": "notification",
  "data": [
    {
      "topicname": "/rest/latest/rest/events?filter=eventid:5208,limit:150",
      "resources": [
        {
          "operation": "inserted",
          "uri": "",
          "values": {
            "MODULE_ID": "-",
            "MODULE_ROLE": "AMM",
            "Message": "Event|5208|LOG_ERR|AMM|-|Failed to enable SSH server on VRF
vrfname
10. Admin password is not set.",
            "OPS_EVENT_CATEGORY": "SSH_SERVER",
            "OPS_EVENT_ID": "5208",
            "PRIORITY": "3",
            "_HOSTNAME": "switch",
            "_PID": "3463",
            "_SOURCE_REALTIME_TIMESTAMP": "2024-06-15:22:04:27.305989"
          }
        }
      ]
    }
  ]
}
```

```

},
{
  "operation": "inserted",
  "uri": "",
  "values": {
    "MODULE_ID": "-",
    "MODULE_ROLE": "AMM",
    "Message": "Event|5208|LOG_ERR|AMM|-|Failed to enable SSH server on VRF
vrfname
11. Admin password is not set.",
    "OPS_EVENT_CATEGORY": "SSH_SERVER",
    "OPS_EVENT_ID": "5208",
    "PRIORITY": "3",
    "_HOSTNAME": "switch",
    "_PID": "3463",
    "_SOURCE_REALTIME_TIMESTAMP": "2024-06-15:22:04:47.325953"
  }
},
{
  "operation": "inserted",
  "uri": "",
  "values": {
    "MODULE_ID": "-",
    "MODULE_ROLE": "AMM",
    "Message": "Event|5208|LOG_ERR|AMM|-|Failed to enable SSH server on VRF
vrfname
12. Admin password is not set.",
    "OPS_EVENT_CATEGORY": "SSH_SERVER",
    "OPS_EVENT_ID": "5208",
    "PRIORITY": "3",
    "_HOSTNAME": "switch",
    "_PID": "3463",
    "_SOURCE_REALTIME_TIMESTAMP": "2024-06-15:22:05:07.346010"
  }
}
],
"subscription_id": "2706986211725480710",
"sequence_number": 0
}
]
}

```

Firmware Upgrade

Subtopics

[Introduction](#)

[Limitations](#)

[Check Firmware Status](#)

[Upload firmware image](#)

[Verify upgrade process](#)

[Boot System Image](#)

[Rest Use Case Examples](#)

Introduction

About this task

Firmware upgrade is a feature that allows the client to upload a firmware image to the switch via REST. Two existing approaches can be used:

Procedure

1. Upload the image from an external image server (PUT).
2. Upload the image from a local file (POST).

Results

To use the new firmware image, a system boot to the updated image is required.

Limitations

- Firmware upgrade requires an orchestrator to fully complete its capabilities.
- Devices can handle up to 2 concurrent requests if the requests share the same SSL session.
- The system clock must be up to date to avoid unexpected results.

Check Firmware Status

Check the current firmware status of a device:

```
```http
GET /rest/<version>/firmware HTTP/1.1
```
```

You may receive a response similar to the following:

```
```json
{
 "current_version": "Virtual.10.13.0000-2634-g30b4e6c301825",
 "primary_version": "",
 "secondary_version": "",
 "default_image": "",
 "booted_image": ""
}
```
```

Upload firmware image

Subtopics

[Remote Location](#)

[Local file](#)

Remote Location

Use the following command to enable or disable the Firmware Site Distribution server listening on port 9443:

```
```http
PUT /rest/<version>/firmware?
from=https%3A%2F%2F10.0.0.1%2FTL_10_17_0001.swi&image=secondary&vrf=mgmt
HTTP/1.1
```
```

Subtopics

[Parameters](#)

Parameters

| Parameter | Description |
|-----------|--|
| from | URI where the image is located. Must include the image filename, for example:

<code>https://10.0.0.1/TL_10_11_0001.swi` or<br/>`https://image.server/TL_10_11_0001.swi</code> |
| image | Slot in which the image will be installed, either `primary` or `secondary`. |
| vrf | Routing namespace used to establish the connection, for example: `mgmt` or `default`. |



NOTE

The upgrade process and the verification process are distinct operations. Each must be initiated independently.

In firmware site distribution scenarios, the port must be omitted because it is automatically added to the source URL.

Local file

Upload a firmware image from a local file:

```
````http
POST /rest/<version>/firmware?image=secondary
Content-Type: multipart/form-data
fileupload=@GL_10_17_0001.swi
````
```

Subtopics

Parameters

Parameters

| Parameter | Description |
|-----------|---|
| image | Slot in which the image will be installed, either `primary` or `secondary`. |

Verify upgrade process

Check the status of the upgrade process after the upload starts:

```
```http
GET /rest/<version>/firmware/status HTTP/1.1
```
```

You may receive a similar example response:

```
```json
{
 "date": "1673392180",
 "reason": "none",
 "status": "success"
}
```
```



NOTE

The upgrade process and the verification process are distinct operations. Each must be initiated independently.

Boot System Image

Boot system to the new firmware image:

```
```http
POST /rest/<version>/boot?image=secondary
```
```

Subtopics

Parameters

Parameters

| Parameter | Description |
|-----------|---|
| image | Slot in which the image will be installed, either `primary` or `secondary`. |

Rest Use Case Examples

The firmware update table contains columns to initiate firmware updates and upload firmware images of the switch. The following columns are used to start these requests:

- src_url
- dst_url
- vrf
- options
- start_request

Supported URL formats

Refer to this section to learn about the supported URL formats.

Download options

- src_url
 - http://SERVER[:PORT]/FILENAME
 - https://SERVER[:PORT]/FILENAME
 - tftp://SERVER[:PORT][:blocksize=<512-65464>]/FILENAME
- dst_url
 - flash:/primary
 - flash:/secondary

Upload options

- src_url

- flash:/primary
- flash:/secondary
- dst_url
 - tftp://SERVER[:PORT][;blocksize=<512-65464>]/FILENAME

Local image transfer options

- src_url
 - flash:/primary
 - flash:/secondary
 - usb:/file
- dst_url
 - flash:/primary
 - flash:/secondary
 - usb:/file



NOTE

If src_url and dst_url hold the same value, an error will be reported and the request will fail.

Firmware Update

The https_ca_trust_anchor option, specifies the certificate authority trust store to use when validating the server's certificate. This option must be configured when using HTTPS, and ignored for other protocols. Possible values for the https_ca_trust_anchor option include:

- user (not currently supported)
- switch
- hpe

While a firmware transfer is in progress, its status can be monitored on the **status** column:

- status
 - not_started
 - starting
 - transferring

- verifying
- complete
- failed
- aborted

The **statistics** column can also be monitored to learn about the update in progress:

- statistics
 - percent_complete
 - bytes_total
 - bytes_transferred
 - time_elapsed
 - transfer_rate
 - start_time
 - finish_time
 - estimated_completion_time
 - retry_num

Once a firmware update has completed, the **status** column will either indicate **complete**, **failed**, or **aborted**. In case of a failure, the **failure reason** column will be updated to display one of the following reasons for failure:

- **failure_reason.number** indicates a numeric value corresponding to the failure reason
- **failure_reason.description** indicates a textual description for the failure

At any time during a firmware update, the process may be aborted by writing within the **abort_request** column. An abort is then triggered. To abort a process:

1. Perform a **PATCH** operation to update the following columns:

- src_url
- dst_url
- vrf
- options

- start_request



NOTE

a new firmware update will be triggered on any integer value change in the **start_request** column. A requestor may need to read this value first to make sure that the **start_request** value being used is different than the current value.

2. Monitor the **status** and **statistics** columns during the firmware update transfer, using the **GET** operation. Statistics are updated at 1-second intervals.
3. Monitor the **status** column until status indicates **completed**, **failed**, or **aborted**. If status shows as **failed**, read the **failure_reason** to determine the cause.



NOTE

Firmware update requests are processed only one at a time. If a new request is received while another is in progress, the new request will be ignored. The **processed_request** column is updated to show the current request in progress. This column will match the value of the **start_request** used to initiate the update.

Subtopics

Use Case 1

Use Case 2

Use Case 3

Use Case 4

Use Case 5

Use Case 1

Prerequisites

Complete the following steps to perform a TFTP download:

Procedure

1. Download the TFTP to the primary image location using the management VRF:

```
curl -X PATCH "https://1.1.1.2/rest/v10.17/system/firmware_update" \  
-H "accept: */*" \  
-H "Content-Type: application/json" \  
-H "x-csrf-token: MnIMxZb2DHWf6VM8PAgcBw==" \  

```

```
-d '{
  "src_url": "tftp://1.1.1.10/NL_10_17_0001.swi",
  "dst_url": "flash:/primary",
  "start_request": 1,
  "vrf": "/rest/v10.17/system/vrfs/mgmt"
}'
```

2. Monitor the **status**, **statistics**, and **processed_request** until the **status** shows as **complete** or **failed**.

```
curl -X GET "https://1.1.1.2/rest/v10.17/system/firmware_update?
attributes=status,processed_request,statistics" \
-H "accept: application/json" \
-H "x-csrf-token: MnIMxZb2DHWf6VM8PAgcBw=="
```

The following information will populate to show the **statistics** and **status**:

```
{
  "processed_request": 1,
  "statistics": {
    "bytes_total": 686059690,
    "bytes_transferred": 26765824,
    "estimated_completion_time": 325,
    "finish_time": 0,
    "percent_complete": 2,
    "retry_num": 0,
    "start_time": 1752786982,
    "time_elapsed": 16,
    "transfer_rate": 2230270
  },
  "status": "transferring"
}
```

3. Use the **GET** operation to monitor the status:

```
curl -X GET "https://1.1.1.2/rest/v10.17/system/firmware_update?
attributes=status,processed_request,statistics" \
-H "accept: application/json" \
-H "x-csrf-token: MnIMxZb2DHWf6VM8PAgcBw=="
```

4. The status of the update should show as complete:

```
"processed_request": 1, "statistics": { "bytes_total": 686059690,
"bytes_transferred": 686059690, "estimated_completion_time": 0, "f
inish_time": 1752787287, "percent_complete": 100, "retry_num": 0,
"start_time": 1752786982, "time_elapsed": 305, "transfer_rate": 22
75936 }, "status": "complete" }
```

Use Case 2

Prerequisites

Complete the following steps to update the secondary firmware image from another switch, using firmware site distribution.

1. Configure the **source** switch for firmware site distribution. See the [firmware site distribution](#) section for information on how to configure this feature.
2. Start an HTTPS download to the secondary image location using the **switch https_ca_trust_anchor**.

```
curl -X PATCH "https://1.1.1.2/rest/v10.17/system/firmware_update" \
-H "accept: */*" \
-H "Content-Type: application/json" \
-H "x-csrf-token: MnIMxZb2DHWf6VM8PAgcBw==" \
-d '{
  "src_url": "https://1.1.1.30:9443/fwimages/primary.swi",
  "dst_url": "flash:/secondary",
  "start_request": 500,
  "vrf": "/rest/v10.17/system/vrfs/mgmt"
  "options": {
    "https_ca_trust_anchor": "switch"
  },
}'
```

3. Monitor the **status**, **statistics**, and **processed_request** until the **status** shows as **complete** or **failed**.

```
curl -X GET "https://1.1.1.2/rest/v10.17/system/firmware_update?
attributes=status,processed_request,statistics" \
-H "accept: application/json" \
-H "x-csrf-token: MnIMxZb2DHWf6VM8PAgcBw=="
```

The status of the update should show as **complete**:

```
{
  "processed_request": 500,
  "statistics": {
    "bytes_total": 686059690,
    "bytes_transferred": 686059690,
    "estimated_completion_time": 0,
    "finish_time": 1752787287,
    "percent_complete": 100,
    "retry_num": 0,
    "start_time": 1752786982,
    "time_elapsed": 305,
```

```

    "transfer_rate": 2275936
  },
  "status": "complete"
}

```

Use Case 3

Prerequisites

Complete the following steps to initiate and abort an HTTP transfer:

1. Begin an HTTP download to the secondary image location using the management VRF.

```

curl -X PATCH "https://1.1.1.2/rest/v10.17/system/firmware_update" \
-H "accept: */*" \
-H "Content-Type: application/json" \
-H "x-csrf-token: MnIMxZb2DHWf6VM8PAgcBw==" \
-d '{
  "src_url": "http://1.1.1.20/NL_10_17_0001.swi",
  "dst_url": "flash:/secondary",
  "start_request": 100,
  "vrf": "/rest/v10.17/system/vrfs/mgmt"
}

```

2. Abort the update using the **abort_request**.

```

curl -X PATCH "https://1.1.1.2/rest/v10.17/system/firmware_update" \
-H "accept: */*" \
-H "Content-Type: application/json" \
-H "x-csrf-token: Ejq2OIa3z3lSGvmK3gyWag==" \
-d '{
  "abort_request": 100
}

```

3. Read the **status** and **failure_reason** to confirm the aborted update.

```

curl -X GET "https://1.1.1.2/rest/v10.17/system/firmware_update?
attributes=status,failure_reason" \
-H "accept: application/json" \
-H "x-csrf-token: MnIMxZb2DHWf6VM8PAgcBw=="

```

The status of the update should show as **aborted**:

```

{
  "failure_reason": {
    "description": "Firmware update operation was aborted by user
request",

```

```
"number": 3
},
"status": "aborted",
```

Use Case 4

Prerequisites

Complete the following steps to begin a transfer from the primary to secondary flash image:

1. Begin the firmware update from the primary to secondary image location.

```
curl -X PATCH "https://1.1.1.2/rest/v10.17/system/firmware_update" \
-H "accept: */*" \
-H "Content-Type: application/json" \
-H "x-csrf-token: MnIMxZb2DHWf6VM8PAgcBw==" \
-d '{
  "src_url": "flash:/primary",
  "dst_url": "flash:/secondary",
  "start_request": 105,
}'
```

2. Monitor the **status**, **statistics**, and **processed_request** until the **status** shows as **complete** or **failed**.

```
curl -X GET "https://1.1.1.2/rest/v10.17/system/firmware_update?
attributes=status,processed_request,statistics" \
-H "accept: application/json" \
-H "x-csrf-token: MnIMxZb2DHWf6VM8PAgcBw=="
```

The status of the update should show as **complete**.

```
{
  "processed_request": 105,
  "statistics": {
    "bytes_total": 686059690,
    "bytes_transferred": 686059690,
    "estimated_completion_time": 0,
    "finish_time": 1752787287,
    "percent_complete": 100,
    "retry_num": 0,
    "start_time": 1752786982,
    "time_elapsed": 305,
    "transfer_rate": 2275936
  },
  "status": "complete"
}
```

Use Case 5

Prerequisites

Complete the following steps to perform a USB-to-flash image transfer:

1. Start the firmware update from an already inserted and mounted USB flash drive to the primary image location.

```
curl -X PATCH "https://1.1.1.2/rest/v10.17/system/firmware_update" \
-H "accept: */*" \
-H "Content-Type: application/json" \
-H "x-csrf-token: MnIMxZb2DHWf6VM8PAgcBw==" \
-d '{
  "src_url": "usb:/NL_10_17_0001.swi",
  "dst_url": "flash:/primary",
  "start_request": 222,
}'
```

2. Monitor the **status**, **statistics**, and **processed_request** until the **status** shows as **complete** or **failed**.

```
curl -X GET "https://1.1.1.2/rest/v10.17/system/firmware_update?
attributes=status,processed_request,statistics" \
-H "accept: application/json" \
-H "x-csrf-token: MnIMxZb2DHWf6VM8PAgcBw=="
```

The status of the update should show as **complete**.

```
{
  "processed_request": 222,
  "statistics": {
    "bytes_total": 686059690,
    "bytes_transferred": 686059690,
    "estimated_completion_time": 0,
    "finish_time": 1752787287,
    "percent_complete": 100,
    "retry_num": 0,
    "start_time": 1752786982,
    "time_elapsed": 305,
    "transfer_rate": 2275936
  },
  "status": "complete"
}
```

Firmware Site Distribution

Subtopics

Introduction

Requirements

Limitations

Rest Management Interface

CLI Management Interface

Introduction

Firmware Site Distribution (FWSD) is an efficient feature designed to allow the client to upgrade multiple devices in a network simultaneously. The orchestrator coordinates the entire upgrade process, providing better control and monitoring. The feature can handle a large number of devices, making it suitable for networks of various sizes. Devices in the network can retrieve firmware images from seed devices, minimizing the load on external sources.

This feature can be used by a CLI command or a REST request. It's possible to execute the firmware site distribution via CLI and the REST management interface request at the same time. The switch can handle up to 2 concurrent requests if the requests share the same SSL session. Although it is possible, it is recommended to use only one to avoid any inconvenience.

FWSD is particularly suitable for environments with a large number of devices which are locally co-located.

FWSD operates as follows:

1. The orchestrator determines which devices require upgrades.
2. The orchestrator sends upgrade commands to the pending devices.
3. Each device individually requests the upgrade, and the orchestrator commands and coordinates the process.

Requirements

To use Firmware Site Distribution:

- all devices must support Firmware Site Distribution.
- ensure that the server is enabled on the seed devices.

Limitations

- HTTPS is the only supported protocol in the location URL.
- Firmware Site Distribution shares the same limitations as [Firmware Upgrade](#).

Rest Management Interface

The rest management interface includes:

- [Seed initialization](#)
- Starting the Upgrade process

Subtopics

[Seed Initialization](#)

[Start Upgrade Process](#)

Seed Initialization

Complete the steps in this section to initialize a seed for Firmware Site Distribution.

Step One

Enable the https server on the server switch.



NOTE

Most platforms have the https server enabled on the VRF "default". If not, execute the following command with the preferred VRF name.

To enable access to the HTTPS server in a given VRF:

```
PATCH /rest/{version}/system/vrfs/{VRF_NAME} HTTP/1.1
Content-Type: application/json
{
  "http_server": {"enable": true}
}
```

Step Two

Enable the image distribution server on the server switch.

To enable the image distribution server:

```
PATCH /rest/{version}/system/rest_config HTTP/1.1
Content-Type: application/json
{
  "firmware_site_distribution_enabled": true
}
```

Step Three

Verify that the server is enabled using a REST request. Use **rest_config** and **firmware_site_distribution_status**.

Example request:

```
GET /rest/<version>/system/rest_config?
attributes=firmware_site_distribution_status HTTP/1.1
```

Example response:

```
{
  "firmware_site_distribution_status": {
    "status": "enabled"
  }
}
```

Start Upgrade Process

To start the upgrade process in Firmware Site Distribution scenarios, you must specify a seed device as the location source in the request sent to the firmware resource.

To receive the location URL for the firmware upgrade from the seed device, the client can make a request to the following resource:

```
```http
GET /rest/<version>/system?attributes=sw_images
```
```

You will receive a JSON response noting the locations for the firmware images:

```
{
  "sw_images": {
    "primary": {
      "location": "/fwimages/primary.swi"
    },
    "secondary": {
      "location": "/fwimages/secondary.swi"
    }
  }
}
```

```
}  
}
```

Use the correct location URL from the response to configure the seed device's location source for the upgrade.



NOTE

The location URL should follow the 'https://<switch>/fwimages/<image>' format. The 'fwimages' prefix is required.

Use the following command to request a download from the Firmware Site Distribution server listening port 9443:

```
```http  
PUT /rest/<version>/firmware?
from=https%3A%2F%2F10.0.0.2%2Ffwimages%2FTL_10_11_0001.swi&image=secondary&v
rf=mgmt HTTP/1.1
```
```



NOTE

When upgrading from one device to another using Firmware Site Distribution, make sure that a properly configured seed device is used as the location source.

CLI Management Interface

Subtopics

[Seed Initialization](#)

[Start Upgrade Process](#)

[Verify Upgrade Process](#)

Seed Initialization

Perform the steps in this section to begin a seed for Firmware Site Distribution.

Subtopics

[Step One](#)

[Step Two](#)

[Step Three](#)

[Step Four](#)

Step One

Enable the https server on the server switch.

Most platforms have the https server enabled on the VRF "default". If not, execute the following command with the preferred VRF name:

```
```bash
switch(config)#
https-server vrf <vrf_name>

```
```

Step Two

Enable the image distribution server on the server switch:

```
```bash
switch(config)# https-server rest firmware-site-distribution

```
```

Step Three

Verify that the server is enabled using the CLI:

```
```bash
switch# show https-server rest firmware-site-distribution
Firmware Site Distribution Configuration

Status: enabled

```
```

Step Four

Upload a firmware image to the seed device using the steps described in the [Firmware Upgrade](#) section.

Start Upgrade Process

To start the upgrade process in Firmware Site Distribution scenarios, you must specify a seed device as the location source.

For CLI management, use the following CLI command:

```
```bash
switch# show https-server rest images
REST Software Images Location

Bank Location

primary "/fwimages/primary.swi"
secondary "/fwimages/secondary.swi"
```
```

Use the correct location URL from the response to configure the seed device's location source for the upgrade.



NOTE

The location URL should follow the 'https://<switch>/fwimages/<image>' format. The 'fwimages' prefix is required.

Include the seed device's address in the from parameter of the request, as shown in the following example:

```
```bash
switch# firmware-site-distribution image secondary from https://<switch_ip>/
fwimages/primary.swi vrf default
```
```



NOTE

When upgrading from one device to another using Firmware Site Distribution, make sure that a properly configured seed device is used as the location source.

Verify Upgrade Process

During the upgrade process, CLI commands will remain blocked until reaching completion.

Subtopics

Interaction between REST management interface and CLI

Interaction between REST management interface and CLI

It's possible to execute the firmware site distribution via CLI and the REST management interface request at the same time.



NOTE

To avoid any inconvenience, it is recommended to use only 2 concurrent requests that share the same SSL session.

Using Curl Commands

There are several tools available for accessing RESTful web service APIs, one of which is curl. The curl tool, created by the cURL project, is a command-line application for transferring data using URL syntax.

For details on installing the curl application, see <https://curl.haxx.se/download.html>.

The curl application has many options, which are described in detail in the curl manual (run `curl --manual`) and at <https://curl.haxx.se/docs/manpage.html>.

Subtopics

[About the curl command examples](#)

[Getting the REST API versions on the switch](#)

[Accessing the REST API using curl](#)

[Examples](#)

About the curl command examples

In the curl examples, the workstation is running a Linux-based operating system and curl version 7.35 is installed.

The curl examples generated by the AOS-CX REST API Reference might use different options than in other examples, and do not include cookie file handling because the cookie is handled by the browser.

Many examples of curl commands are formatted in multiple lines for readability. The backslash (`\`) continuation character at the end of the line indicates that the command continues on the next line.

The curl command examples in this document use minimal options. The following options are commonly used in the curl command examples:

```
-b<cookie-file>
```

Specifies that the file `<cookie-file>` , which contains the session cookie, be passed with the request. `<cookie-file>` specifies the path and name of the cookie file.

When you use curl, you log in at the beginning of your session and log out at the end of the session. When you log in, you must save the cookie returned from the login request. You must provide the cookie with every subsequent curl command.

`-k`

Specifies that the curl program not attempt to verify the server certificate against the list of certificate authorities included with the curl software.

The switch uses self-signed certificates. By default, the curl program attempts to verify certificates against its list of certificate authorities, and attempts to verify self-signed certificates will fail. Therefore you must use the `-k` option to disable attempts to verify self-signed certificates against a certificate authority.

`--no-proxy`

Specifies that a web proxy is not required. The `--no-proxy` option is appropriate where execution of curl commands does not need a proxy to access the applications.

If your network is configured to require a proxy to access applications, use the `--proxy` option instead of the `--no-proxy` option.

`-d '<string>'`

Specifies that curl send the data in `<string>` in a POST request using the content-type application/x-www-form-urlencoded.

`-X`

Specifies a method that curl would not use by default. Typically used with PUT, DELETE, and POST methods only.

`-H` or `--header <header>`

Specifies an extra header in the HTTP request.

`-D`

Specifies that curl write the returned protocol headers to the standard output file. Used for debugging.

More options can be used to customize your experience for your environment. For more information about curl options, see:

<https://curl.haxx.se/docs/manpage.html>

Getting the REST API versions on the switch

To get information about the latest and all available REST API versions on a switch, execute a GET request to the following URI:

```
"https://<IP-ADDR>/rest"
```

<IP-ADDR> is the IP address or hostname of your switch.

Example method and URI:

```
GET "https://192.0.2.5/rest"
```

Example curl command:

```
$ curl -k GET \  
-b /tmp/auth_cookie \  
"https://192.0.2.5/rest"
```

Example response body:

```
{  
  "latest": {  
    "version": "v10.09",  
    "prefix": "/rest/v10.09",  
    "doc": "/api/v10.09"  
  },  
  "v10.09": {  
    "version": "v10.09",  
    "prefix": "/rest/v10.09",  
    "doc": "/api/v10.09"  
  },  
  "v10.04": {  
    "version": "v10.04",  
    "prefix": "/rest/v10.04",  
    "doc": "/api/v10.04"  
  }  
}
```

Accessing the REST API using curl

When you use curl, you log in at the beginning of your session and log out at the end of the session. When you log in, you must save the cookie returned from the login request so that you can pass that same cookie value to the switch in subsequent curl commands.

Prerequisites

- Access to the switch REST API must be enabled.

Procedure

1. To access the AOS-CX REST API using curl, use curl version 7.35 or later. The examples provided in this document are tested with version 7.35.

2. For all curl commands, use the `-k` option to disable certificate validation.
The switch uses self-signed certificates. By default, the curl program attempts to verify certificates against its list of certificate authorities, and attempts to verify self-signed certificates fail. Therefore you must use the `-k` option to disable attempts to verify self-signed certificates against a certificate authority.
3. Start your session by logging in. When you log in, save the cookie file by specifying the `-c` option with a file name.
4. In all subsequent curl commands—including logging out—pass the cookie value back to the switch by specifying the `-b` option with the same file name.
5. At the end of the session, log out of the switch using curl.

**NOTE**

Logging out at the end of the session is important because the number of concurrent HTTPS sessions per client and per switch are limited, and session cookies are not shared across devices and scripts.

Subtopics

Logging in using curl

Passing the cookie back to the switch

Logging Out Using Curl

Logging in using curl

Prerequisites

Access to the switch REST API must be enabled.

**CAUTION**

Credential information (user name, password, domain, and authentication tokens) used in curl commands entered at a command-line prompt might be saved in the command history. For security reasons, Hewlett Packard Enterprise recommends that you disable command history before executing commands containing credential information.

Procedure

Use the following curl command to access the `login` resource of the switch and provide your user name and password as data:

Syntax:

```
curl [--noproxy <IP-ADDR>] -k -X POST
-c <COOKIE-FILE>=
-H 'Content-Type: multipart/form-data'
'https://<IP-ADDR>/rest/v10.xx/login'-F 'username=<USER-NAME>' -F
'password=<PASSWORD>'
```

Options:

-k

Specifies that the curl program not attempt to verify the server certificate against the list of certificate authorities included with the curl software.

The switch uses self-signed certificates. By default, the curl program attempts to verify certificates against its list of certificate authorities, and attempts to verify self-signed certificates fail. Therefore you must use the **-k** option to disable attempts to verify self-signed certificates against a certificate authority.

-X

Specifies a method that curl would not use by default. Typically, used only with POST, PUT, PATCH, or DELETE methods.

--noproxy *IP-ADDR*

Optional. The **--noproxy** option is appropriate where execution of curl commands does not need a proxy to access the applications. If your network is configured to require a proxy to access applications, use the **--proxy** option. *<IP-ADDR>* specifies the IP address or hostname of the switch.

-C *<COOKIE-FILE>*

Specifies the file in which to store the session cookie. This session cookie is required when you execute subsequent curl commands.

-H or **--header** *<header>*

Specifies an extra header in the HTTP request.

-F

Specifies that the curl command will emulate a filled-in form in which a user has pressed the submit button for the HTTP protocol family. This causes curl to POST data using the Content-Type multipart/form-data.

<USER-NAME>

Specifies the user name.

<PASSWORD>

Specifies the password for the user.



NOTE

Although it is possible to pass the user name and password information as a query string in the login URI, system logs save the accessed URI in cleartext in log entries. Hewlett Packard Enterprise recommends that you pass the credential information as data instead of in the URI when using programs such as curl to log in to the switch.

Example:

```
$ curl --noproxy "192.0.2.5" -k -X POST \  
-c /tmp/auth_cookie -H 'Content-Type: multipart/form-data' \"https://  
192.0.2.5/rest/v10.09/login\" \  
-F 'username=test' -F 'password=test'
```

Passing the cookie back to the switch

Prerequisites

Start a session by logging in to the REST API and save the cookie file.

Procedure

Use the following curl command to pass the cookie file back to the switch using the -b option.

Syntax:

```
curl [--noproxy <IP-ADDR>] -k GET  
-b <COOKIE-FILE>  
-H 'Content-Type:application/json'  
-H 'Accept: application/json'  
"https://<IP-ADDR>/rest/v10.xx/system"
```

Options:

```
--noproxy <IP-ADDR>
```

Optional. The --noproxy option is appropriate where execution of curl commands does not need a proxy to access the applications. If your network is configured to require a proxy to access applications, use the --proxy option. <IP-ADDR> specifies the IP address or hostname of the switch.

```
-k
```

Specifies that the curl program not attempt to verify the server certificate against the list of certificate authorities included with the curl software.

The switch uses self-signed certificates. By default, the curl program attempts to verify certificates against its list of certificate authorities, and attempts to verify self-signed certificates fail. Therefore you must use the -k option to disable attempts to verify self-signed certificates against a certificate authority.

```
-b <COOKIE-FILE>
```

Specifies that the file <cookie-file>, which contains the session cookie, be passed with the request. The <cookie-file> specifies the path and name of the cookie file.

When you use curl, you log in at the beginning of your session and log out at the end of the session. When you log in, you must save the cookie returned from the login request. You must provide the cookie with every subsequent curl command.

```
-H or --header <header>
```

Specifies an extra header in the HTTP request.

Example:

```
$ curl --noproxy -k GET
-b /tmp/auth_cookie \
-H 'Content-Type:application/json' \
-H 'Accept: application/json' \
"https://192.0.2.5/rest//system"
```

Logging Out Using Curl

Use the following curl command to access the

```
logout
```

resource of the switch:

Syntax:

```
curl [--noproxy <IP-ADDR>] -k -X POST
-b <COOKIE-FILE>
"https://<IP-ADDR>/rest/v10.xx/logout"
```

Options:

```
-k
```

Specifies that the curl program not attempt to verify the server certificate against the list of certificate authorities included with the curl software.

The switch uses self-signed certificates. By default, the curl program attempts to verify certificates against its list of certificate authorities, and attempts to verify self-signed certificates fail. Therefore you must use the `-k` option to disable attempts to verify self-signed certificates against a certificate authority.

```
--noproxy<IP-ADDR>
```

Optional. The `--noproxy` option is appropriate where execution of curl commands does not need a proxy to access the applications. If your network is configured to require a proxy to access applications, use the `--proxy` option. <IP-ADDR> specifies the IP address or hostname of the switch.

```
-b <COOKIE-FILE>
```

Specifies the file that contains the session cookie.



NOTE

When you use curl, you log in at the beginning of your session and log out at the end of the session. When you log in, you must save the cookie returned from the login request so that you can pass that same cookie value to the switch in subsequent curl commands. When you log in, save the cookie file by specifying the `-c` option with a file name.

In subsequent curl commands, pass the cookie value back to the switch by specifying the `-b` option with the same file name.

`-X`

Specifies a method that curl would not use by default. Typically, used only with POST, PUT, or DELETE methods.

Example:

```
$ curl --no-proxy "192.0.2.5" -k -X POST \  
-b /tmp/auth_cookie \  
"https://192.0.2.5/rest/v10.09/logout"
```

Examples

The following examples show how you can use curl with AOS-CX REST API. The response body might vary based on the AOS-CX switch series. For example, the 8320 and 6400 switches show VSX information, whereas the 6300 and 6200 switches show VSF and PoE information.

As a best practice, before you perform a GET, PUT, PATCH, POST, or DELETE operation, you must login to create a session and save the cookie file by specifying the `-c` option with a file name. After you perform the operation, you must logout to end the session and pass the cookie file back to the switch by specifying the `-b` option with the same file name. The following examples assume that you are performing the step to login before performing the operations in the example and logout after performing the operations. For more information, see [Accessing the REST API using curl](#).



NOTE

The request and response body in the following examples contain a truncated part of the actual data. The data that you see after running the curl commands might vary. Ellipses (...) in the response body represent data not shown in the example.

Subtopics

Example: GET method

Example: Getting and deleting certificates using REST APIs

Example: Generating a self-signed certificate using REST APIs

Example: Getting and installing a signed leaf certificate using REST APIs

Example: Associating a leaf certificate with a switch feature using REST APIs

Example: Configuration management using REST APIs

Example: Log operations using REST APIs

Example: Ping operations using REST APIs

Example: Traceroute operations using REST APIs

Example: User management using REST APIs

Example: Creating an ACL with an interface using REST APIs

Example: Creating a VLAN and a VLAN interface using REST APIs

Example: Enabling routing on an interface

Example: PATCH Method

Example: GET method

Instructions and examples in this document use an IP address that is reserved for documentation, 192.0.2.5, as an example of the IP address for the switch. To access your switch, you must use the IP address or hostname of that switch.

- Get the list of all VLANs:
GET "https://192.0.2.5/rest/v10.xx/system/vlans"
- Expand the list of URIs in the `vlans` collection by one level, which replaces the URI for the VLAN with the JSON data for that VLAN.
GET "https://192.0.2.5/rest/v10.xx/system/vlans?depth=2"
- Get the administrative state of interface 1/1/3:
https://192.0.2.5/rest/v10.xx/system/interfaces/1%2F1%2F3?attributes=admin
- Use the `attributes` parameter to get all interfaces but show only the attributes `name` and `ipv4_address`:
GET "https://192.0.2.5/rest/v10.xx/system/interfaces?depth=2&attributes=name,ipv4_address"
- Use the `selector` parameter to get all the writable configuration attributes of VLAN 100:
GET "https://192.0.2.5/rest/v10.xx/system/vlans/100?selector=writable"
- Use the `selector` parameter to get all the system attributes that are in the categories configuration and status:
GET "https://192.0.2.5/rest/v10.xx/system?selector=configuration,status"

Example: Getting and deleting certificates using REST APIs

Subtopics

[Getting a list of all certificates](#)

[Getting a certificate](#)

[Deleting a certificate](#)

Getting a list of all certificates



NOTE

It is important to note that the certificate resources do not support the use of internationalized strings. Since UTF8 is the only supported encoding, a Subject Alternative Name (SAN) must be used instead.

Example method and URI:

```
GET "https://192.0.2.5/rest/v10.xx/certificates"
```

Example curl command:

```
$ curl --noproxy 192.0.2.5 -k GET \  
-b /tmp/auth_cookie \  
"https://192.0.2.5/rest/v10.09/certificates"
```

On successful completion, the switch returns response code 200 OK and a response body containing the certificate resource URLs indexed by the certificate name. For example:

```
{  
  "my-cert-1": "/rest/v10.xx/certificates/my-cert-1",  
  "my-cert-2": "/rest/v10.xx/certificates/my-cert-2"  
}
```

Getting a certificate

Example method and URI:

```
GET "https://192.0.2.5/rest/v10.xx/certificates/my-cert-2"
```

Example curl command:

```
$ curl --noproxy 192.0.2.5 -k GET \  
-b /tmp/auth_cookie \  
"https://192.0.2.5/rest/v10.09/certificates/my-cert-2"
```

```
"https://192.0.2.5/rest/v10.09/certificates/my-cert-2"
```

On successful completion, the switch returns response code 200 OK and a response body containing the certificate.

For example:

```
{
  "cert_name": "my-cert-2",
  "cert_type": "regular"
  "cert_status": "csr_pending",
  "key_type": "RSA",
  "key_size": 2048,
  "subject": {
    "common_name": "CX-8400",
    "country": "US",
    "locality": "el camino",
    "state": "CA",
    "org": "HPE",
    "org_unit": "Aruba"
  },
  "certificate": "<certificate-in-PEM-format>"
}
```

Deleting a certificate

Example method and URI:

```
DELETE "https://192.0.2.5/rest/v10.xx/certificates/my-cert-3"
```

Example curl command:

```
$ curl --no-proxy 192.0.2.5 -k -X DELETE \
-b /tmp/auth_cookie \
"https://192.0.2.5/rest/v10.09/certificates/my-cert-3"
```

On successful completion, the switch returns response code 204.

Example: Generating a self-signed certificate using REST APIs

The following example generates a self-signed certificate.

Example method and URI:

```
POST "https://192.0.2.5/rest/v10.xx/certificates"
```

Example request body:

```
{
...
  "certificate_name": "my-cert-1",
  "subject": {
    "country": "US",
    "state": "CA",
    "org": "HPE",
    "org_unit": "Aruba",
    "common_name": "CX-8400"},
  "key_type": "RSA",
  "key_size": 2048,
  "cert_type": "self-signed"
...
}
```

Example curl command:

```
$ curl --noproxy 192.0.2.5 -k -X POST \
-b /tmp/auth_cookie \
"https://192.0.2.5/rest/v10.09/certificates"
-d '{
...
  "certificate_name": "my-cert-1",
  "subject": {
    "country": "US",
    "state": "CA",
    "org": "HPE",
    "org_unit": "Aruba",
    "common_name": "CX-8400"},
  "key_type": "RSA",
  "key_size": 2048,
  "cert_type": "self-signed"
...
}'
```

On successful completion, the switch returns response code 201 Created.

Example: Getting and installing a signed leaf certificate using REST APIs

About this task

This example includes the step to create a trust anchor (TA) profile. If the TA profile had previously been configured, that step of the example would be skipped. The TA profile is used to validate the signed certificate when you import the certificate as part of the PUT request.

For more information about certificates and certificate management, see the Security Guide .

Procedure

1. Create a TA Profile

- a. From the certificate authority (CA), get a copy of the certificate against which you will validate leaf certificates. The certificate you validate leaf certificates against can be a root certificate or an intermediate certificate.

The steps to get the leaf certificate depend on the CA and the operating system you use.

- b. Create a JSON object with a `certificate` key and a `name` key.

For example:

```
{ "name": "<profile-name>", "certificate": "<root-ca-cert>" }
```

- For the value of the name key, replace <profile-name> with the name of the TA profile you want to create.
- For the value of the certificate key, replace <root-ca-cert> by pasting the copied certificate.
- After pasting, edit the text to ensure proper loading as a JSON object by doing the following:
 - Ensure the certificate headers and footers are treated as separate lines by adding `\n` characters after the header and before the footer.
The following example shows the `\n` characters in bold.

```
{  
  
  "name": "myta", "certificate": "-----BEGIN CERTIFICATE-----\nMIIF2DCCA8CgAwIBAgIICnL  
MA0GCSqGSIb3DQEBCwUAMHkxCzAJBgNVBAYTAkdCMRAwDgYDVQQIDAdFbmdsYW5kM  
PKj0FmJ1+Qzw9Bcm6HiPTyxOVozMeRQzSQhTZVlh3OvBw/cUwTlqFJCe/  
afNQCqa9XnvTpJvP/Q3z ... S4L9srxk/i3hKB88\n-----END CERTIFICATE-----"  
  
}
```

- Ensure that any private key headers and footers are treated as separate lines by adding `\n` characters before and after them as needed.

For example:

```
\n-----BEGIN PRIVATE KEY-----\n  
MIIFDjBABGkqhkiG9wBBQ0wMzAbBgqkw0QwwDQIpJMN7sVGwCAgGA
```

...

```
iKnXnUMpVPfLc74ty2S41DtH0X9gf6aa1jStg+7cND9XfGtjaV2CA
```

```
\n-----END PRIVATE KEY-----\n
```

```
\n-----BEGIN ENCRYPTED PRIVATE KEY-----\n
```

```
IJ6L/UhEtH523nUkdV6gvAgoYaD83PswToAGv5VS8OMFTPttrn5/K ...
```

```
OgSecqZsG6arbx0ESaYBir1c/6rPspcbx283iD1MWOpes2aEmOX=
```

```
\n-----END ENCRYPTED PRIVATE KEY-----\n
```

- c. Use the POST method to create the TA profile with the copied certificate. Include the JSON object in the request body: Example method and URI:

POST "https://192.0.2.5/rest/v10.xx/system/pki_ta_profiles"

Example curl commands:

```
$ curl --noproxy 192.0.2.5 -k -X POST \ -b /tmp/primary_auth_cookie \ -H 'Content-Type:application/json' "https://192.0.2.5/rest/v10.04/system/pki_ta_profiles" -d '{ "name": "myta", "certificate": "-----BEGIN CERTIFICATE-----\nMIIF2DCCA8CgAwIBAgIJANKWgud1lCnLMA0GCSqSgSlb3DQEBChUAMHkxCzAJBgNVBAYTAkdCMRAwDgYDVQQIDAdFbmdsYW5kMRIwEAYPKj0FmJ1+Qzw9Bcm6HiPTyxOVozMeRQzSQhTZVlh3OvBw/cUwTlqFJCe/afNQCqa9XnvTpJvP/Q3ze6 S4L9srxk/i3hKB88\n-----END CERTIFICATE-----" }
```

' On successful completion, the switch returns response code 201 Created.

2. Create a certificate with a pending certificate signing request (CSR). For information about the required and optional items in the request body, see the JSON model for the certificates resource in the AOS-CX REST API Reference.

Example method and URI:

POST "https://192.0.2.5/rest/v10.xx/certificates"

Example request body:

```
{ "certificate_name": "my-cert-name", "subject": { "common_name": "CX-8400" "country": "US", "locality": "el camino", "state": "CA", "org": "HPE", "org_unit": "Aruba", }, "key_type": "RSA", "key_size": 2048, "cert_type": "regular" }
```

Example curl command:

```
$ curl --noproxy 192.0.2.5 -k -X POST \ -b /tmp/primary_auth_cookie \ -d '{ "certificate_name": "my-cert-name", "subject": { "common_name": "CX-8400" "country": "US", "locality": "el camino", "state": "CA", "org": "HPE", "org_unit": "Aruba", }, "key_type": "RSA", "key_size": 2048, "cert_type": "regular" }' "https://192.0.2.5/rest/v10.09/certificates"
```

On successful completion, the switch returns response code 201 Created.

3. Get the certificate you created in the previous step. Example method and URI:

GET "https://192.0.2.5/rest/v10.xx/certificates/my-cert-name"

Example curl command:

```
$ curl --noproxy 192.0.2.5 -k GET \ -b /tmp/primary_auth_cookie \ "https://192.0.2.5/rest/v10.09/certificates/my-cert-name"
```

On successful completion, the switch returns response code 200 OK and a response body containing the CSR in PEM format.

4. Send the CSR to the CA for signing. The steps to send the CSR depend on the CA and the operating system you use.

The CA returns the signed certificate in PEM format.

5. Import the signed certificate by using a PUT request to update the `my-cert-name` certificate with the signed certificate you received from the CA.

The imported certificate data must include all the intermediate CA certificates in the certificate chain leading to the certificate that was imported into the specified TA profile.

If you copy and paste the certificate into a JSON object, you must ensure that the certificate and private key headers and footers are processed as separate lines by editing the text to add `\n` characters as needed.

As part of the PUT request, the switch attempts to validate the certificate against the pool of all TA profiles installed on the switch. The certificate is accepted if it is validated with one of the TA profiles.

Example method and URI:

PUT "https://192.0.2.5/rest/v10.xx/certificates/my-cert-name"

Example request body:

```
{ "certificate": "-----BEGIN CERTIFICATE-----\nMIIFRDCCAyggAwIBAgQP8nS2Vp15u0xXMdkDJzANBgkqhkiG9w0Bv ... 1NGNm3NG03GqPScs/\nTF9bVyFA5BOS5lmmkfRYK8D/kMTfRreSdxis YQ1u1NqShps= \n-----END\nCERTIFICATE-----\n\n-----BEGIN ENCRYPTED PRIVATE\nKEY-----\nMIIFDjBABgkqhkiG9wBBQ0wMzAbBgqkw0QwwDQlpJMN7sVGwCAggA ... \niKnXnUMpVPfLc74ty2S41DtH0X9gf6aa1jStg+7cND9XfGtjaV2+/ cb4= \n-----END ENCRYPTED\nPRIVATE KEY-----" }
```

Example curl commands:

```
$ curl --noproxy 192.0.2.5 -k -X PUT \ -b /tmp/primary_auth_cookie \ -d '{ "certificate": "-----
BEGIN CERTIFICATE-----\n MIIFRDCCAYygAwIBAgQP8nS2Vp15u0xXMdkDJzANBgkqhkiG9w0Bv ...
1NGNm3NG03GqPScs/TF9bVyFA5BOS5lmmkfRYK8D/kMTfRreSdxis YQ1u1NqShps= \n-----
END CERTIFICATE-----\n \n-----BEGIN ENCRYPTED PRIVATE
KEY-----\n MIIFDjBABgkqhkiG9wBBQ0wMzAbBgqkw0QwwDQlpJMN7sVGwCAggA ...
iKnXnUMpVPfLc74ty2S41DtH0X9gf6aa1jStg+7cND9XfGtjaV2+/ cb4= \n-----END ENCRYPTED
PRIVATE KEY-----" }' "https://192.0.2.5/rest/v10.09/certificates/my-cert-name"
```

On successful completion, the switch returns response code 200 OK.

The certificate is installed and ready to be associated with switch features.

Example: Associating a leaf certificate with a switch feature using REST APIs

The following example associates the signed certificate `my-cert-name` with the HTTPS server switch feature. For complete information about the switch features to which you can associate a leaf certificate, see the AOS-CX Security Guide .

Procedure

1. Get the configuration attributes of the `system` resource:

Example method and URI:

```
GET "https://192.0.2.5/rest/v10.xx/system?selector=configuration"
```

Example curl command:

```
$ curl --noproxy 192.0.2.5 -k GET \
-b /tmp/primary_auth_cookie \
"https://192.0.2.5/rest/v10.09/system?selector=configuration"
```

On successful completion, the switch returns response code 200 and a JSON object containing the configuration attributes.

2. In the portion of the response body that defines the certificate name for the HTTPS server, change the value to: `my-cert-name` .

The certificate name associated with the HTTPS server is the value assigned to the **https-server** key, which is under the `certificate_association` key. By default, the certificate name is: `local-cert`

The request body of a PUT request is permitted to include only the mutable configuration attributes. In the AOS-CX software releases to which this example applies, all the configuration attributes for the

system resource are mutable attributes, so you do not need to edit the JSON object to remove the immutable attributes.

3. Using a PUT request, update the system resource with the edited JSON data as the request body.
Example method and URI:

PUT "https://192.0.2.5/rest/v10.xx/system"

Example request body:

```
{ "aaa": { ... }, ... "certificate_association": { "https-server": "my-cert-name", "syslog-client": "local-cert" }, ... }
```

Example curl command:

```
$ curl --noproxy 192.0.2.5 -k -X PUT \  
-b /tmp/primary_auth_cookie \  
-d '{  
    "aaa": {  
    ...  
    },  
    ...  
    "certificate_association": {  
        "https-server": "my-cert-name",  
        "syslog-client": "local-cert"  
    },  
    ...  
}'  
"https://192.0.2.5/rest/v10.09/system"
```

On successful completion, the switch returns response code 200 OK.

Example: Configuration management using REST APIs

Subtopics

Downloading a configuration

Downloading the startup configuration:

Uploading a configuration

Copying a configuration

Downloading a configuration

Downloading the current configuration:

- Example method and URI:
GET "https://192.0.2.5/rest/v10.xx/fullconfigs/running-config"

- Example curl command:

```
$ curl --noproxy 192.0.2.5 -k GET \  
-b /tmp/primary_auth_cookie \  
"https://192.0.2.5/rest/v10.09/fullconfigs/running-config"
```

Downloading the startup configuration:

- Example method and URI:
GET "https://192.0.2.5/rest/v10.xx/fullconfigs/startup-config"

- Example curl command:

```
$ curl --noproxy 192.0.2.5 -k GET \  
-b /tmp/primary_auth_cookie \  
"https://192.0.2.5/rest/v10.09/fullconfigs/startup-config"
```

On successful completion, the switch returns response code 200 OK and a response body containing the entire configuration in JSON format.

Uploading a configuration

The following example shows uploading a configuration to become the running configuration. The running configuration is the only configuration that can be updated using this technique, however, you can copy other configurations. For more information about copying configurations, see [Copying a configuration](#).

- Example method and URI:
PUT "https://192.0.2.5/rest/v10.xx/fullconfigs/running-config"

The request body must contain the configuration—in JSON format—to be uploaded.

- Example curl command:

```
$ curl --noproxy 192.0.2.5 -k -X PUT \  
-b /tmp/auth_cookie \  
"https://192.0.2.5/rest/v10.09/fullconfigs/running-config" \  

```

```
-d '{  
...  
'
```

The configuration being uploaded—represented as ellipsis but not shown in this example—is in JSON format in the body of the command (enclosed in braces).

On successful completion, the switch returns response code 200 OK.

Copying a configuration

To replace an existing configuration with another, use a REST PUT request to the destination configuration. Use the `from` query string parameter to specify the source configuration.

- At least one of the source or the destination configuration must be either `running-config` or `startup-config`. You cannot copy a checkpoint to a different checkpoint.
- If you specify a destination checkpoint that exists, an error is returned. You cannot overwrite an existing checkpoint.

The syntax of the method and URI is as follows:

```
PUT "https://<IPADDR>/rest/v10.xx/fullconfigs/<destination-config>?  
from=/rest/v10.xx/fullconfigs/<source-config>"
```

Copying the running configuration to the startup configuration:

- Example method and URI:
PUT "https://192.0.2.5/rest/v10.xx/fullconfigs/startup-config? from=/rest/v10.xx/fullconfigs/running-config"
- Example curl command:

```
$ curl --no-proxy 192.0.2.5 -k -X PUT \  
-b /tmp/auth_cookie -D-  
"https://192.0.2.5/rest/v10.09/fullconfigs/startup-config?  
from=/rest/v10.09/fullconfigs/running-config"
```

Copying the startup configuration to the running configuration:

- Example method and URI:
PUT "https://192.0.2.5/rest/v10.xx/fullconfigs/running-config? from=/rest/v10.xx/fullconfigs/startup-config"
- Example curl command:

```
$ curl --noproxy 192.0.2.5 -k -X PUT \
-b /tmp/auth_cookie -D-
"https://192.0.2.5/rest/v10.09/fullconfigs/running-config?
from=/rest/v10.09/fullconfigs/startup-config"
```

Copying a checkpoint to the running configuration:

- Example method and URI:
PUT "https://192.0.2.5/rest/v10.xx/fullconfigs/running-config? from=/rest/v10.xx/fullconfigs/MyCheckpoint"

- Example curl command:

```
$ curl --noproxy 192.0.2.5 -k -X PUT \
-b /tmp/auth_cookie -D-
"https://192.0.2.5/rest/v10.09/fullconfigs/running-config?
from=/rest/v10.09/fullconfigs/MyCheckpoint"
```

Copying the running configuration to a checkpoint:

- Example method and URI:
PUT "https://192.0.2.5/rest/v10.xx/fullconfigs/MyCheckpoint? from=/rest/v10.xx/fullconfigs/running-config"

- Example curl command:

```
$ curl --noproxy 192.0.2.5 -k -X PUT \
-b /tmp/auth_cookie -D-
"https://192.0.2.5/rest/v10.09/fullconfigs/MyCheckpoint?
from=/rest/v10.09/fullconfigs/running-config"
```

Copying a configuration to the running-config using a JSON config file:

The syntax of the method and URI is as follows:

```
PUT "https://{IP}/rest/{version}/configs/running-config"
```



CAUTION

The JSON configuration file is provided as the HTTP request payload.

- Example method and URI:
PUT "https://192.0.2.5/rest/v10.09/configs/running-config"

- Example curl command:

```
$ curl -k --noproxy 192.0.2.5 -X PUT -b /tmp/cookies \
"https://192.0.2.5/rest/v10.09/configs/running-config" --header "Content-
Type: application/json" --data-binary @config.json
```

Copying a configuration to the running-config using a Base64-encoded CLI config file:

The syntax of the method and URI is as follows:

```
PUT "https://{IP}/rest/{version}/configs/running-config?operation={replace|merge}"
```



CAUTION

The Base64-encoded CLI is provided as the HTTP request payload.

- Example method and URI:
PUT "https://192.0.2.5/rest/v10.09/configs/running-config?operation=merge"

- Example curl command:

```
$ curl -k --noproxy 192.0.2.5 -X PUT -b /tmp/cookies \  
"https://192.0.2.5/rest/v10.09/configs/running-config?operation=merge"  
--header "Content-Type: text/plain" --data-binary @config.b64
```

Example: Log operations using REST APIs

Subtopics

Event logs

Accounting (audit) logs

Event logs

A GET request to `/rest/v10.xx/logs/event` URI returns all entries from all the event logs on the switch, including logs from internal processes.

The information returned by this request was not optimized for human readability. If you want to examine the log entries, Hewlett Packard Enterprise recommends that you use the Web UI. The Web UI also provides a method to export log entries.

In the following example, the `MESSAGE_ID` parameter filters the output to include event log messages only:

- `50c0fa81c2a545ec982a54293f1b1945` identifies event log messages from the active management module.
- `73d7a43eaf714f97bbdf2b251b21cade` identifies event log messages from the standby management module. Not all switches have a standby management module.

Example method and URI:

```
GET "https://192.0.2.5/rest/v10.xx/logs/event?
limit=1000&
priority=4&
since=24%20hour%20ago&
MESSAGE_ID=50c0fa81c2a545ec982a54293f1b1945,73d7a43eaf714f97bbdf2b251b21cade
"
```

Example curl command:

```
$ curl --noproxy 192.0.2.5 -k GET \
-b /tmp/primary_auth_cookie \
"https://192.0.2.5/rest/v10.09/logs/event?
limit=1000&
priority=4&
since=24%20hour%20ago&
MESSAGE_ID=50c0fa81c2a545ec982a54293f1b1945,73d7a43eaf714f97bbdf2b251b21cade
"
```

Accounting (audit) logs

A GET request to the `/rest/v10.xx/logs/audit` URI returns all entries from the accounting logs on the switch.

For a list of supported query parameters, see the AOS-CX REST API Reference.

Example method and URI:

```
GET "https://192.0.2.5/rest/v10.xx/logs/audit?
since=24%20hour%20ago&
usergroup=administrators&
session=CLI"
```

Example curl command:

```
$ curl --noproxy 192.0.2.5 -k GET \
-b /tmp/primary_auth_cookie \
"https://192.0.2.5/rest/v10.09/logs/audit?
since=24%20hour%20ago&
usergroup=administrators&
session=CLI"
```

Example: Ping operations using REST APIs

This example gets ping statistics for the ping target.

Example method and URI:

```
GET "https://192.0.2.5/rest/v10.xx/ping?
ping_target=192.0.2.10&
is_ipv4=true&
ping_data_size=100&
ping_time_out=2&
ping_repetitions=1&
ping_type_of_service=0&
include_time_stamp=false&
include_time_stamp_address=false&
record_route=false&
mgmt=false"
```

Example curl command:

```
$ curl --noproxy 192.0.2.5 -k GET \
-b /tmp/primary_auth_cookie \
"https://192.0.2.5/rest/v10.09/ping?
ping_target=192.0.2.10&
is_ipv4=true&
ping_data_size=100&
ping_time_out=2&
ping_repetitions=1&
ping_type_of_service=0&
include_time_stamp=false&
include_time_stamp_address=false&
record_route=false&
mgmt=false"
```

On successful completion, the switch returns response code 200 OK and a response body containing the output string produced by the ping operation.

Example: Traceroute operations using REST APIs

Example method and URI:

```
GET "https://192.0.2.5/rest/v10.xx/traceroute?
ip=192.0.2.10&
is_ipv4=true&
timeout=3&
```

```
destination_port=33434&
max_ttl=30&
min_ttl=1&
probes=3&
mgmt=false"
```

Example curl command:

```
$ curl --noproxy 192.0.2.5 -k GET \
-b /tmp/primary_auth_cookie \
"https://192.0.2.5/rest/v10.09/traceroute?
ip=192.0.2.10&
is_ipv4=true&
timeout=3&
destination_port=33434&
max_ttl=30&
min_ttl=1&
probes=3&
mgmt=false"
```

On successful completion, the switch returns response code 200 OK and a response body containing the output string produced by the traceroute operation.

Example: User management using REST APIs

Subtopics

[Creating a user](#)

[Changing a password](#)

[Deleting a user](#)

Creating a user

Example method and URI:

```
POST "https://192.0.2.5/rest/v10.xx/system/users"
```

Example request body:

```
{
...
  "name": "myadmin",
  "password": "P@ssw0rd",
  "user_group": "/rest/v10.xx/system/user_groups/administrators",
```

```
  "origin": "configuration"
  ...
}
```

Example curl command:

```
$ curl --noproxy -k -X POST \
-b /tmp/auth_cookie \
"https://192.0.2.5/rest/v10.09/system/users"
-d '{
  ...
  "name": "myadmin",
  "password": "P@ssw0rd",
  "user_group": "/rest/v10.09/system/user_groups/administrators",
  "origin": "configuration"
  ...
}'
```

On successful completion, the switch returns response code 201 Created.

Changing a password

Example method and URI:

```
PUT "https://192.0.2.5/rest/v10.xx/system/users/myadmin"
```

Example request body:

```
{
  "password": "P@ssw0rd2g"
  "current_password": "current_password"
}'
```

Example curl command:

```
$ curl --noproxy -k -X PUT \
-b /tmp/auth_cookie \
"https://192.0.2.5/rest/v10.09/system/users/myadmin"
-d '{
  "password": "P@ssw0rd2g"
  "current_password": "current_password"
}'
```

On successful completion, the switch returns response code 200 OK.

Deleting a user

Example method and URI:

```
DELETE "https://192.0.2.5/rest/v10.xx/system/users/myadmin"
```

Example curl command:

```
$ curl --noproxy -k -X DELETE \  
-b /tmp/auth_cookie \  
"https://192.0.2.5/rest/v10.09/system/users/myadmin"
```

On successful completion, the switch returns response code 204 No Content.

Example: Creating an ACL with an interface using REST APIs

About this task

This example shows creating the following ACL and interface configuration on a switch at IP address 192.0.2.5:

```
access-list ip ACLv4  
    10 permit tcp 10.0.100.101 eq 80 10.0.100.102 eq 8000  
interface 1/1/2  
    no shutdown  
    apply access-list ip ACLv4 out
```

Procedure

1. Creating the ACL.

```
$ curl --noproxy 192.0.2.5 -k -X POST \  
-b /tmp/auth_cookie -d '{  
"cfg_version": 0,  
"list_type": "ipv4",  
"name": "ACLv4"}'  
"https://192.0.2.5/rest/v10.09/system/acls"
```

2. Creating an ACL entry.

```
$ curl --noproxy 192.0.2.5 -k -X POST \  
-b /tmp/auth_cookie -d '{  
"action": "permit",  
"dst_ip": "10.0.100.102/255.255.255.255",  
"dst_l4_port_max": 8000,  
"dst_l4_port_min": 8000,  
"protocol": 6,
```

```
"sequence_number": 10,
"src_ip": "10.0.100.101/255.255.255.255",
"src_l4_port_max": 80,
"src_l4_port_min": 80}'
"https://192.0.2.5/rest/v10.09/system/acls/ACLv4,ipv4/cfg_aces"
```

3. Updating the ACL configuration using the return body received from the GET request performed in the previous step. Any writable attributes you do not include in the PUT request body are set to their defaults, which could be empty.

The following example shows the request to update the ACL configuration:

```
$ curl --no-proxy 192.0.2.5 -k -X PUT \
-b /tmp/auth_cookie -d '{
"cfg_aces":{"10":"/rest/v10.09/system/acls/ACLv4,ipv4/cfg_aces/10"},
"cfg_version":1}' \
"https://192.0.2.5/rest/v10.09/system/acls/ACLv4,ipv4"
```

4. Getting the writable attributes of an interface. The GET response body includes only the configuration attributes that have been set.

```
$ curl --no-proxy 192.0.2.5 -k GET \
-b /tmp/auth_cookie \
"https://192.0.2.5/rest/v10.09/system/interfaces/1%2F1%2F2?
selector=writable"
```

Response body:

```
{ ... "cdp_disable": false, "description": null, "lldp_med_loc_civic_ca_info": {}, "lldp_med_loc_civic_info":
null, "lldp_med_loc_elin_info": null, "options": {}, "other_config": { "lldp_dot3_macphy_disable":
false, "lldp_med_capability_disable": false, "lldp_med_network_policy_disable": false,
"lldp_med_topology_notification_disable": false }, "pfc_priorities_config": null, "selftest_disable":
false, "udld_arubaos_compatibility_mode": "forward_then_verify", "udld_compatibility": "aruba_os",
"udld_enable": false, "udld_interval": 7000, "udld_retries": 4, "udld_rfc5171_compatibility_mode": "normal",
"user_config": { "admin": "down" ... }
```

5. Enabling the interface and adding the ACL information to the interface by using the return body received from the GET request performed in the previous step. The modified values are shown in the following example.

```
$ curl --no-proxy 192.0.2.5 -k -X PUT \
-b /tmp/auth_cookie -d
'{
...
"user_config": { "admin": "up" },
"aclv4_out_cfg":"/rest/v10.09/system/acls/ACLv4,ipv4",
"aclv4_out_cfg_version":1,
```

```
...
}' -D- \
"https://192.0.2.5/rest/v10.09/system/interfaces/1%2F1%2F2"
```

Example: Creating a VLAN and a VLAN interface using REST APIs

About this task

This example shows creating the following VLAN and interface configuration on a switch at IP address 192.0.2.5:

```
vlan 2
    no shutdown
interface vlan 2
```

Procedure

1. Creating the VLAN.

```
$ curl --no-proxy 192.0.2.5 -k -X POST \
-b /tmp/auth_cookie -d '{
"name":"vlan2",
"id":2,
"type":"static",
"admin":"up"
"user_config":{"admin":"up"}}' \
"https://192.0.2.5/rest/v10.09/system/vlans"
```

2. Creating an interface with VLAN information

```
$ curl --no-proxy 192.0.2.5 -k -X POST \
-b /tmp/auth_cookie -d '{
"vrf": "/rest/v10.09/system/vrfs/default",
"vlan_tag":"/rest/v10.09/system/vlans/2",
"name":"vlan2",
"type":"vlan"}' \
-D- "https://192.0.2.5/rest/v10.09/system/interfaces"
```

Example: Enabling routing on an interface

About this task

The following example shows how to enable routing on an interface.

```
interface 1/1/1
    routing
```

Procedure

1. Getting the writable configuration information for the interface.

```
$ curl --noproxy 192.0.2.5 -k GET \
-b /tmp/auth_cookie \
-H 'Content-Type:application/json'
-H 'Accept: application/json'
"https://192.0.2.5/rest/v10.09/system/interfaces/1%2F1%2F1?
selector=writable"
```

Response body:

The GET response body includes only the configuration attributes that have been set.

```
{
...
  "routing": false,
  "udld_arubaos_compatibility_mode": "forward_then_verify",
  "udld_compatibility": "aruba_os",
  "udld_enable": false,
  "udld_interval": 7000,
  "udld_retries": 4,
  "udld_rfc5171_compatibility_mode": "normal",
  "user_config": {}
  "vlan_mode": null,
  "vlan_tag": null,
  "vlan_translations": {},
  "vlan_trunks": [],
  "vlans_per_protocol": {},
  "vrf": null,
...
}
```

2. Update the interface using the return body received from the GET request, modifying the `routing` attribute to be: `"routing": true`.

Any writable attributes you do not include in the PUT request body are set to their defaults, which could be empty.

```
$ curl --noproxy -X PUT \
-b /tmp/auth_cookie \
-H 'Content-Type: application/json'
-H 'Accept: application/json'
-d '{
...
"routing":true,
...
}'
"https://192.0.2.5/rest/v10.09/system/interfaces/1%2F1%2F1"
```

Example: PATCH Method

Subtopics

Enabling a VLAN

Enabling Central

Changing the Source IP of a VRF

Using GET and PATCH to Update the admin state of a VLAN

Using PATCH to Update a Non-configurable attribute

Enabling a VLAN

Example curl command:

```
$
curl -k --noproxy <ip> -X PATCH -b /tmp/cookies -d '{"admin":"up"}' -D- "ht
tps://<ip>/rest/<version>/system/vlans/100"
```

Enabling Central

Example curl command:

```
$
curl -k --noproxy <ip> -b /tmp/cookies -X PATCH -d '{"disable":false}' -D- "
https://<ip>/rest/<version>/system/aruba_central"
```

Changing the Source IP of a VRF

Example curl command:

```
$  
curl -k --noproxy <ip> -b /tmp/cookies -X PATCH -d '{"source_ip":{"all":"10.1.1.1"}}' -D- "https://<ip>/rest/<version>/system/vrfs/mgmt"
```

Using GET and PATCH to Update the admin state of a VLAN

Example GET curl command:

```
$ curl --noproxy -k -X GET "https://192.0.2.5/rest/v10.09/system/vlans/100"  
-H "accept: */*" -d "" -b /tmp/auth_cookie  
HTTP/1.1 200 OK  
Server: nginx  
Date: Wed, 03 Nov 2021 22:53:02 GMT  
Content-Type: application/json; charset=utf-8  
Transfer-Encoding: chunked  
Connection: keep-alive  
Etag: be17a56d01ca6eba8fc1901a4f5d2fd6  
X-Frame-Options: SAMEORIGIN  
X-Content-Type-Options: nosniff  
X-XSS-Protection: 1; mode=block  
Strict-Transport-Security: max-age=31536000; includeSubdomains;  
Content-Security-Policy: script-src 'self' 'unsafe-inline'; object-src  
'none'; font-src *; media-src 'none'; form-action 'self';
```

On successful completion, the switch returns the following:

```
{  
...  
  "admin": "up",  
  "clear_ip_bindings": {},  
  "delete_macs_rejected": {},  
  "delete_macs_requested": {},  
  "description": null,
```

```

    "flood_enabled_subsystems": {},
    "id": 100,
    "internal_usage": {},
    "macs": "/rest/v10.09/system/vlans/100/macs",
    "macs_invalid": null,
    ...
}

```

Example PATCH curl command:

```

$ curl --noproxy -k -X PATCH "https://192.0.2.5/rest/v10.09/system/
vlans"/100 -H "accept: */*" -d '{"admin": "down"}' -b /tmp/auth_cookie
HTTP/1.1 204 No Content
Server: nginx
Date: Wed, 03 Nov 2021 22:54:43 GMT
Connection: keep-alive
X-Frame-Options: SAMEORIGIN
X-Content-Type-Options: nosniff
X-XSS-Protection: 1; mode=block
Strict-Transport-Security: max-age=31536000; includeSubdomains;
Content-Security-Policy: script-src 'self' 'unsafe-inline'; object-src
'none'; font-src *; media-src 'none'; form-action 'self';

```

Example GET curl command:

```

$ curl --noproxy -k -X GET "https://192.0.2.5/rest/v10.09/system/vlans/100
-H "accept: */*" -d "" -b /tmp/auth_cookie
HTTP/1.1 200 OK
Server: nginx
Date: Wed, 03 Nov 2021 22:53:02 GMT
Content-Type: application/json; charset=utf-8
Transfer-Encoding: chunked
Connection: keep-alive
Etag: be17a56d01ca6eba8fc1901a4f5d2fd6
X-Frame-Options: SAMEORIGIN
X-Content-Type-Options: nosniff
X-XSS-Protection: 1; mode=block
Strict-Transport-Security: max-age=31536000; includeSubdomains;
Content-Security-Policy: script-src 'self' 'unsafe-inline'; object-src
'none'; font-src *; media-src 'none'; form-action 'self';
$ curl --noproxy -k -X GET "https://192.0.2.5/rest/v10.09/system/vlans"/100
-H "accept: */*" -d "" -b /tmp/auth_cookie
HTTP/1.1 200 OK
Server: nginx
Date: Wed, 03 Nov 2021 22:54:52 GMT
Content-Type: application/json; charset=utf-8

```

```
Transfer-Encoding: chunked
Connection: keep-alive
Etag: d54a643eb48515e94ea94bd501d9d2c8
X-Frame-Options: SAMEORIGIN
X-Content-Type-Options: nosniff
X-XSS-Protection: 1; mode=block
Strict-Transport-Security: max-age=31536000; includeSubdomains;
Content-Security-Policy: script-src 'self' 'unsafe-inline'; object-src
'none'; font-src *; media-src 'none'; form-action 'self';
```

On successful completion, the switch returns the following:

```
{
...
  "admin": "down",
  "clear_ip_bindings": {},
  "delete_macs_rejected": {},
  "delete_macs_requested": {},
  "description": null,
  "flood_enabled_subsystems": {},
  "id": 100,
  "internal_usage": {},
  "macs": "/rest/v10.09/system/vlans/100/macs",
  "macs_invalid": null,
...
}
```

Using PATCH to Update a Non-configurable attribute

Example PATCH curl command:

```
$ curl --no-proxy -k -X PATCH "https://192.0.2.5/rest/v10.09/system/
vlans/100 -H "accept: */*" -d '{"oper_state": "up"}' -b /tmp/auth_cookie
HTTP/1.1 400 Bad Request
Server: nginx
Date: Wed, 03 Nov 2021 23:08:02 GMT
Content-Type: text/plain; charset=utf-8
Content-Length: 73
Connection: keep-alive
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
X-Content-Type-Options: nosniff
X-XSS-Protection: 1; mode=block
```

```
Strict-Transport-Security: max-age=31536000; includeSubdomains;  
Content-Security-Policy: script-src 'self' 'unsafe-inline'; object-src  
'none'; font-src *; media-src 'none'; form-action 'self';  
cannot modify the attribute: oper_state, reason: Non-configurable column  
$
```

AnyCLI

AnyCLI is a custom REST resource that allows for a predefined list of troubleshooting CLI commands to be executed via REST API. Only one command is permitted per request and no concurrent requests are allowed. The resource also provides an endpoint to retrieve the full list of available commands.



NOTE

AnyCLI does not support CLI configuration commands. Only a subset of read-only show commands are available.

URI

- The URI to execute a CLI command: `/rest/{version}/cli`
- The URI to retrieve allowed CLI commands: `/rest/{version}/cli/commands`

Limitations

- Valid versions: v10.04 and higher
- Invalid version: v1

Security

Allowed roles: Administrator and operator.

Prohibited: Auditor

Response Time

If the response time exceeds the http server timeout (10min) the command will be canceled and with a 408 Request Timeout generated.

Concurrent Requests

One command is allowed per request.

Subtopics

[Commands available per platform](#)

[CLI operations](#)

[CLI commands operations](#)

[Swagger](#)

Full URI

CURL example

Error codes

Allowed commands

Full example

Commands available per platform

The mechanism exposed via REST mirrors the same level of support the CLI has for a predefined list of CLI commands. If a command is available but has a limitation, then the REST CLI feature will reflect the same. For details in the description, usability, and health of any particular command, it is recommended to review the functionality guide where the feature that command belongs to is documented.

Not all show commands are available on every platform. As per the Error Code section, an "invalid command" response is expected whenever the CLI doesn't support the command. However, without executing the command through CLI REST has no visibility into whether the command is available on any given platform. Since the `/cli/commands` endpoint does not interact with CLI, it does not filter unsupported commands.

The Allowed Commands section lists the complete set of commands available through REST CLI. The commands from that list (which are not present in the Commands Available Per Platform table) are available in every platform.

To determine if a command is available on a specific switch platform, refer to the .

CLI operations

Obtains the vtysh output from executing a command.

POST

Body of the request must contain a `cmd` key with the corresponding command from the allowed list of commands as its value. Response body will contain the plain/text output of vtysh if successful.

Request body

```
{  
  "cmd": <cmd>  
}
```

Response body

```
<vtysh_output>
```

CLI commands operations

Obtains the list of allowed commands.

GET

Response body will contain a JSON objects commands with the list of allowed commands.

Response body

```
{
  "commands": <allowed_commands>
}
```

Swagger

<https://{IP}/api/{version}/cli>

Full URI

- Full URI: <https://{IP}/rest/{version}/cli/>
- Full URI: <https://{IP}/rest/{version}/cli/commands/>

CURL example

```
curl -k --no-proxy "{IP}" -b cookies \
-X POST "https://{IP}/rest/cli" \
-H "content-type: application/json" \
-d '{"cmd": "show uptime"}'
curl -k --no-proxy "{IP}" -b cookies \
-X GET "https://{IP}/rest/cli/commands" \
-H "accept: application/json"
```

Error codes

| HTTP Code | Error | Response |
|-----------------------|--|---|
| 200 OK | None | VTYSH output |
| 400 Bad Request | Invalid Input, Command incomplete or Ambiguous command | Invalid command. |
| 400 Bad Request | Invalid JSON Payload missing key | Invalid JSON input. '{}' required parameter missing |
| 401 Unauthorized | Authorization error | 401 Authorization Required |
| 403 Forbidden | Command not allowed (non in preset - list) | Command '{}' not allowed |
| 403 Forbidden | Command not allowed on CLI | Command not allowed |
| 429 Too Many Requests | Multiple Concurrent Requests Error | 429 Too Many Requests |
| 502 Bad Gateway | Any other CLI error | Command execution failed |

Allowed commands

- show aaa accounting
- show aaa authentication port-access interface all client-status
- show aaa authentication
- show aaa authorization
- show aaa server-groups
- show active-gateway
- show arp all-vrfs
- show arp
- show bgp all
- show bgp all community
- show bgp all extcommunity

- show bgp all flap-statistics
- show bgp all neighbors
- show bgp all paths
- show bgp all summary
- show bgp all-vrf all
- show bgp all-vrf all neighbors
- show bgp all-vrf all paths
- show bgp all-vrf all summary
- show bgp ipv4 unicast
- show bgp ipv4 unicast community
- show bgp ipv4 unicast extcommunity
- show bgp ipv4 unicast flap-statistics
- show bgp ipv4 unicast neighbors
- show bgp ipv4 unicast paths
- show bgp ipv4 unicast summary
- show bgp l2vpn evpn
- show bgp l2vpn evpn extcommunity
- show bgp l2vpn evpn neighbors
- show bgp l2vpn evpn paths
- show bgp l2vpn evpn summary
- show boot-history
- show capacities
- show capacities-status
- show class ip
- show class ipv6
- show clock
- show copp-policy statistics

- show dhcp-relay
- show dhcp-server
- show dhcpv4-snooping binding
- show dhcpv4-snooping
- show dhcpv6-relay
- show dhcpv6-server
- show dhcpv6-snooping binding
- show dhcpv6-snooping
- show environment
- show evpn evi detail
- show evpn evi
- show evpn mac-ip
- show evpn vtep-neighbor all-vrfs
- show gbp role-mapping
- show interface brief
- show interface error-statistics
- show interface lag
- show interface physical
- show interface qos
- show interface queues
- show interface statistics
- show interface tunnel
- show interface utilization
- show interface vxlan vni vteps
- show interface vxlan vni
- show interface vxlan vteps detail
- show interface vxlan vteps

- show interface vxlan
- show interface
- show ip dns
- show ip helper-address
- show ip igmp
- show ip mroute
- show ip multicast summary
- show ip ospf all-vrfs
- show ip ospf border-routers all-vrfs
- show ip ospf interface all-vrfs
- show ip pim
- show ip route all-vrfs
- show ipv6 helper-address
- show ipv6 mld
- show ipv6 mroute
- show ipv6 neighbors all-vrfs
- show ipv6 neighbors
- show ipv6 ospfv3 all-vrfs
- show ipv6 ospfv3 border-routers all-vrfs
- show ipv6 ospfv3 interface all-vrfs
- show ipv6 pim6
- show lacp aggregates
- show lacp interfaces
- show lldp local
- show lldp neighbor
- show loop-protect
- show mac-address-table

- show module
- show nd-snooping binding
- show nd-snooping prefix-list
- show nd-snooping statistics
- show nd-snooping
- show ntp associations
- show ntp servers
- show ntp status
- show port-access clients onboarding-method device-profile
- show port-access clients onboarding-method dot1x
- show port-access clients onboarding-method mac-auth
- show port-access clients onboarding-method port-security
- show port-access clients
- show port-access gbp
- show port-access policy
- show port-access port-security interface all client-status
- show port-access port-security interface all port-statistics
- show port-access role local
- show port-access role radius
- show port-access port-security violation client-limit-exceeded interface all
- show power-over-ethernet
- show qos dscp-map
- show qos queue-profile
- show qos schedule-profile
- show qos trust
- show radius dyn-authorization
- show radius-server

- show resources
- show secure-mode
- show spanning-tree detail
- show spanning-tree mst detail
- show system inventory
- show system resource-utilization
- show tacacs-server
- show ubt brief
- show ubt information
- show uptime
- show version
- show vlan
- show vrf
- show vsf detail
- show vsf link detail
- show vsf link error-detail
- show vsf topology
- show vsf
- show ztp information

Full example

Get the full list of available commands:

- **URI:** /rest/{version}/cli/commands
- **Valid versions:** v10.04 and higher
- **Operation:** GET
- **Query parameters:** empty

- **Request body:** empty
- **Response body:** snippet

```
curl -X GET -b /tmp/cookies \
"https://{IP}/rest/{version}/cli/commands" \
-H "accept: application/json"
```

- **Response body:** snippet

```
{
  "commands": [
    ...
    "show lldp local",
    "show lldp neighbor",
    ...
  ]
}
```

Then, retrieve the vtysh output from the command with a POST:

- **URI:** /rest/{version}/cli
- **Operation:** POST
- **Query parameters:** empty
- **Request body:** snippet

```
{
  "cmd": "show uptime"
}
curl -X POST -b /tmp/cookies "https://{IP}/rest/{version}/cli" \
-H "content-type: application/json" -d '{"cmd": "show uptime"}'
```

- **Response body:** snippet

```
System has been up 11 minutes
```

Secure Mode

Secure mode provides a method of setting the security mode of the device. A zeroization is required before switching between enhanced and standard secure modes.

Setting secure mode requires populating writable attribute `SYSTEM::secure_mode` with the following value:

```
true
```

Limitations

- Valid versions: v10.04 and higher
- Invalid version: v1

Security

Allowed roles: Administrator and operator.

Prohibited: Auditor

Example

The "secure_mode" variable has two values, as follows:

- false: secure mode flag is not set and the device continues to operate in its current state
- true: secure mode flag is set and the device reboots immediately, performs a zeroization, and boots to Secure mode



NOTE

The "secure_mode" flag always returns **false** as the device reboots immediately after this value is set, and after booting to any mode, the flag is reset to **false**.

Get the secure mode writable attributes:

```
* __URI__: `/REST/{VERSION}/system`
* __Valid versions__: `v10.04` and higher
* __Operation__: `GET`
* __Query parameters__:
* `selector=writable`
* __Request body__: empty
* __Response body__:
```json
{
...
"secure_mode":null
...
}
```
* __Description__: Get writable secure mode attributes to use them for a
`PUT`
* __Swagger__: `https://{IP}/api/{version}/#/System/get_system`
* __Full URI__: `https://{IP}/rest/{version}/system?selector=writable`
* __Curl example__:
```bash
curl -X GET -b /tmp/cookies \
```

```
"https://{IP}/rest/{version}/system?selector=writable" \
-H "accept: application/json"
...

"secure_mode":null
* __CLI equivalent commands__: N/A
```

Or, get relevant secure mode parameters:

```
* __URI__: `/REST/{VERSION}/system`
* __Valid versions__: `v10.04` and higher
* __Operation__: `GET`
* __Query parameters__:
* `selector=writable`
* __Request body__: empty
* __Response body__:
```json
{
...
"secure_mode":null
...
}
...

* __Description__: Get writable secure mode attributes to use them for a
`PUT`
* __Swagger__: `https://{IP}/api/{version}/#/System/get_system`
* __Full URI__: `https://{IP}/rest/{version}/system?selector=writable`
* __Curl example__:
```bash
curl -X GET -b /tmp/cookies \
"https://{IP}/rest/{version}/system?attributes=secure_mode" \
-H "accept: application/json"
...

<HTTP/1/1 200 OK
< Server: nginx
< Date: Mon, 27 Mar 2023 22:35:54 GMT
< Content-Type: application/json; charset=utf-8
< Content-Length: 62
< Connection: keep-alive
< Etag: 52c2158e26ef504acd5b65a6e00b4a2b
< X-Frame-Options: SAMEORIGIN
< X-Content-Type-Options: nosniff
< X-XSS-Protection: 1; mode=block
< Strict-Transport-Security: max-age=31536000; includeSubdomains;
< Content-Security-Policy: script-src 'self' 'unsafe-inline'; object-src
'none'; font-src *; media-src 'none'; form-action 'self';
```

```
<
{
 "secure_mode": null,
}
```

Then, use the returned JSON to do a **PATCH**:

```
* __URI__: `/REST/{VERSION}/system`
* __Valid versions__: `v10.10` and higher
* __Operation__: `PATCH`
* __Query parameters__: empty
* __Request body__:
  ```json
  {
    "secure_mode":true
  }
  ```
* __Description__: Initiate a zeroization followed by setting secure mode
* __Swagger__: `https://{IP}/api/{version}/#/System/patch_system`
* __Full URI__: `https://{IP}/rest/{version}/system`
* __Curl example__:
  ```bash
  curl {IP} -X PATCH -b /tmp/cookies \
  "https://{IP}/rest/{version}/system" \
  -H "accept: */*" \
  -H "Content-Type: application/json" \
  -d \
  "{
  \"secure_mode\":true
  }"
  ```
* __CLI equivalent commands__:
* [secure-mode enhanced] (#secure-mode-command)
```

Or, use the returned JSON to do a **PUT**:

```
* __URI__: `/REST/{VERSION}/system`
* __Valid versions__: `v10.10` and higher
* __Operation__: `PATCH`
* __Query parameters__: empty
* __Request body__:
  ```json
  {
    "secure_mode":true
  }
  ```
```

```

* __Description__: Initiate a zeroization followed by setting secure mode
* __Swagger__: `https://{IP}/api/{version}/#/System/patch_system`
* __Full URI__: `https://{IP}/rest/{version}/system`
* __Curl example__:
```bash
curl {IP} -X PUT -b /tmp/cookies \
"https://{IP}/rest/{version}/system" \
-H "accept: */*" \
-H "Content-Type: application/json" \
-d \
"{
  \"secure_mode\":true
}"

```

Showing the secure mode setting is read from the `SYSTEM::secure_mode_status` attribute:

```

* __URI__: `/REST/{VERSION}/system`
* __Valid versions__: `v10.10` and higher
* __Operation__: `GET`
* __Query parameters__: empty
* __Request body__: empty
* __Response body__:
```json
{
 ...
 "secure_mode_status":"enhanced"
 ...
}
```

* __Description__: Get the current status of secure mode.
* __Swagger__: `https://{IP}/api/{version}/#/System/get_system`
* __Full URI__: `https://{IP}/rest/{version}/system`
* __Curl example__:
```bash
curl -X GET -b /tmp/cookies \
"https://{IP}/rest/{version}/system" \
-H "accept: application/json"

```

Or, showing the relevant secure mode parameters:

```

* __URI__: `/REST/{VERSION}/system`
* __Valid versions__: `v10.10` and higher
* __Operation__: `GET`
* __Query parameters__: empty
* __Request body__: empty
* __Response body__:
```json

```

```

{
...
"secure_mode_status":"enhanced"
...
}
...

* __Description__: Get the current status of secure mode.
* __Swagger__: `https://{IP}/api/{version}/#/System/get_system`
* __Full URI__: `https://{IP}/rest/{version}/system`
* __Curl example__:
```bash
curl -X GET -b /tmp/cookies \
"https://{IP}/rest/{version}/system?attributes=secure_mode_status" \
-H "accept: application/json"
< HTTP/1.1 200 OK
< Server: nginx
< Date: Mon, 27 Mar 2023 22:35:54 GMT
< Content-Type: application/json; charset=utf-8
< Content-Length: 62
< Connection: keep-alive
< Etag: 52c2158e26ef504acd5b65a6e00b4a2b
< X-Frame-Options: SAMEORIGIN
< X-Content-Type-Options: nosniff
< X-XSS-Protection: 1; mode=block
< Strict-Transport-Security: max-age=31536000; includeSubdomains;
< Content-Security-Policy: script-src 'self' 'unsafe-inline'; object-src
'none'; font-src *; media-src 'none'; form-action 'self';
<
{
"secure_mode_status": "enhanced"
}

```

## Subtopics

### Commands available per platform

## Commands available per platform

The mechanism exposed via REST mirrors the same level of support the CLI has for a predefined list of CLI commands. If a command is available but has a limitation, then the REST CLI feature will reflect the same. For details in the description, usability, and health of any particular command, it is recommended to review the functionality guide where the feature that command belongs to is documented.

Not all show commands are available on every platform. As per the Error Code section, an "invalid command" response is expected whenever the CLI doesn't support the command. However, without executing the command through CLI REST has no visibility into whether the command is available on any given platform. Since the `/cli/commands` endpoint does not interact with CLI, it does not filter unsupported commands.

The Allowed Commands section lists the complete set of commands available through REST CLI. The commands from that list (which are not present in the Commands Available Per Platform table) are available in every platform.

To determine if a command is available on a specific switch platform, refer to the .

## VSX peer switches and REST API access

If Virtual Switching Extension (VSX) is enabled, you can access the REST API of a peer switch without having to separately log into or manage a session cookie from that peer switch.

To access a peer REST API from your connected switch, insert `/vsx-peer` in the URI path after the server URL and before the REST API and version identifier.

For example:

```
https://192.0.2.5/vsx-peer/rest/v10.xx/...
```



### NOTE

The following uses of `/vsx-peer` in the URI path are not supported:

- You cannot specify the `login` resource. Requests to `/vsx-peer/rest/v10.04/login` are not required because logging in to one device automatically gives you access to the peer device.
- You cannot access the Web UI of a VSX peer switch. Setting the browser address to `https://<connected_switch_ip>/vsx-peer` is not supported.
- You cannot specify a VSX peer switch in the URIs in topic subscription messages in the real-time notifications framework. However, you can access the real-time notifications framework on the VSX peer switch by setting the connection address to the following:

```
wss://<connected_switch_ip>/vsx-peer/rest/v10.xx/notification
```

Please note the following points when using REST API with VSX.

- VSX must be enabled on both switches, and the interswitch link (ISL) must be up.
- REST API access must be enabled on the switch to which you are connected.

- For write access, the REST API access mode must be set to read-write on the switch to which you are connected.
- You must be logged in to the switch to which you are connected. For example, if you are connected to the primary VSX switch, you must be logged in to the primary switch.
- When configuration synchronization is enabled, supported configuration changes on the primary VSX switch are replicated on the secondary VSX switch. Changing the configuration of a secondary VSX switch might cause the configurations to be out of synchronization.
- Audit messages are logged on the peer switch, with the user information from the switch to which the user is connected.

## Subtopics

### Examples of curl commands

#### Example: Interacting with a VSX peer switch

## Examples of curl commands

- Getting the VSX status of the secondary VSX switch while connected to the primary VSX switch at IP address 192.0.2.5:

```
$ curl --no-proxy "192.0.2.5" -k GET \
-b /tmp/primary_auth_cookie \
"https://192.0.2.5/vsx-peer/rest/v10.09/system/vsx?
attributes=oper_status"
```

- Getting the VSX status of the primary VSX switch while connected to the secondary VSX switch at IP address 192.0.2.6:

```
$ curl --no-proxy "192.0.2.6" -k GET \
-b /tmp/secondary_auth_cookie \
"https://192.0.2.6/vsx-peer/rest/v10.09/system/vsx?
attributes=oper_status"
```

- Getting the names and IP addresses of interfaces on secondary VSX switch while connected to the primary VSX switch at IP address 192.0.2.5:

```
$ curl --no-proxy "192.0.2.5" -k GET \
-b /tmp/primary_auth_cookie \
"https://192.0.2.5/vsx-peer/rest/v10.09/system/interfaces?
depth=2&attributes=name,ipv4_address"
```

For more information about VSX, see the Virtual Switching Extension (VSX) Guide .

## Example: Interacting with a VSX peer switch

In the following examples, Virtual Switching Extension (VSX) is enabled, the primary VSX switch IP address is 192.0.2.5, and the secondary VSX switch IP address is 192.0.2.6.

Getting the list of all VLANs on the connected switch at IP address 192.0.2.5:

- Example method and URI:  
GET "https://192.0.2.5/rest/v10.xx/system/vlans"

- Example curl command:

```
$ curl --no-proxy 192.0.2.5 -k GET \
-b /tmp/primary_auth_cookie \
"https://192.0.2.5/rest/v10.09/system/vlans"
```

Getting the list of all VLANs on the peer VSX switch:

- Example method and URI:  
GET "https://192.0.2.5/vsx-peer/rest/v10.xx/system/vlans"

- Example curl command:

```
$ curl --no-proxy 192.0.2.5 -k GET \
-b /tmp/primary_auth_cookie \
"https://192.0.2.5/vsx-peer/rest/v10.09/system/vlans"
```

Getting the VSX status of the secondary VSX switch while connected to the primary VSX switch at IP address 192.0.2.5:

- Example method and URI:  
GET "https://192.0.2.5/vsx-peer/rest/v10.xx/system/vsx?attributes?oper\_status"

- Example curl command:

```
$ curl --no-proxy 192.0.2.5 -k GET \
-b /tmp/primary_auth_cookie \
"https://192.0.2.5/vsx-peer/rest/v10.09/system/vsx?attributes?oper_status"
```

You can also get the VSX status of the primary VSX switch while connected to the secondary VSX switch.

## AOS-CX real-time notifications subsystem

The AOS-CX REST API, combined with built-in databases that provide configuration, state, statistical data, and time-series data for the features and protocols running in the switch, provides a flexible means for switch programmability. Each resource or collection of resources inside the switch is uniquely identified by its URI.

Clients can use the REST API to request information about resources. However, this polling ability does not address the specific use cases in which network management systems need to receive live data or real-time events from the switch. There is a need to have a live notification subsystem that provides the remote network management system with real-time information about any changes that occur in the switch. Timely information about changes is important for troubleshooting and statistical data analyses, as well as for the immediate reaction to real-time events.

The AOS-CX real-time notifications subsystem enables external clients to connect to the switch through a secure WebSocket Protocol connection and to receive real-time notifications about the switch resources, configuration changes, state changes, and statistical information of their interest.

The WebSocket Protocol was selected based on latency, throughput, resource utilization, network overhead, and security requirements. The handshake part of the WebSocket protocol uses HTTPS, so there is no need to open a new port on the switch side, and there is no need to provide a new authentication mechanism. Multiple clients and connections are supported.

AOS-CX notification messages use JSON encoding. The JSON encoding was designed to align with REST payloads, which enable clients to use combined REST and notification solutions.

The ability to subscribe to these push notifications about a variety of types of information about the switch, combined with the structured nature of the JSON data reported by the switch database, enables a form of network monitoring commonly called telemetry streaming.

Interested clients, known as subscribers, might include the following:

- Web clients such as the AOS-CX Web UI
- Network management systems
- Monitoring scripts

## **Subtopics**

**Secure WebSocket Protocol connections for notifications**

**Enabling the notifications subsystem on a switch**

**Establishing a secure WebSocket connection through a web browser**

**Establishing a secure WebSocket connection using a script**

**Subscribing to topics**

**Unsubscribing from topics**

**Subscription throttling**

**Parts of a subscribe message**

**Parts of a subscription success message**

**Parts of a notification message**

**Disconnection scenarios**

**Example: Browser-based WebSocket connection**

**Example: Getting information about current subscribers**

**Live Switch Events**

## Secure WebSocket Protocol connections for notifications

You subscribe to and receive notifications from the switch through a secure WebSocket Protocol ( `wss://` ) connection.

A secure WebSocket Protocol connection is a secure, persistent, and full-duplex connection between a client and a server. Either the client or the server can send data in the form of messages at any time.

The handshake part of the WebSocket Protocol uses HTTPS, so there is no need to open a new port on the switch side, and there is no need to provide a new authentication mechanism. When you connect to a switch through a secure WebSocket Protocol connection, you pass the session cookie received from logging in to the REST API. Secure WebSocket Protocol connections to switches running AOS-CX software remain active until the connection is closed, even after the session cookie expires. Multiple clients and connections are supported.

For more information about the WebSocket Protocol see RFC 6455: The WebSocket Protocol at: <https://tools.ietf.org/html/rfc6455>

### Subtopics

**Notification topics as switch resource URIs**

**Rules for topic URIs**

**Notification security features**

**AOS-CX real-time notifications subsystem reference summary**

## Notification topics as switch resource URIs

When you subscribe to notifications, you subscribe to notifications about specific topics. A topic is the URI of a specific switch resource. That URI can contain a query string that specifies particular attributes of that resource.

For example, specifying the following URI as a topic results in notifications being sent when the administrative state or link state of any interface changes, but not when some other attribute of an interface changes:

```
/rest/v10.09/system/interfaces?depth=2&attributes=admin_state,link_state
```

The AOS-CX REST API Reference lists all the switch resources. You can use the GET method of the resource in the AOS-CX REST API Reference to determine the URI for that switch resource, including the query string to specify an attribute or list of attributes.

## Rules for topic URIs

A topic is the URI of a switch resource:

Not all switch resource URIs are supported as notification topics.

The **Implementation Notes** section of the GET method of the resource in the AOS-CX REST API Reference indicates if the resource is not supported by the notifications subsystem.

- Wildcard characters ( `*` ) are supported. In this example, you can subscribe to all VLANs:

```
``data/rest/v10.04/system/vlans/*
```
```

- Specifying a resource on a peer VSX switch, by including `/vsx-peer` in the URIs for topic subscription messages, is not supported.

To specify a peer switch, include `/vsx-peer` in the URL of the WSS connection. For example, to get notifications about VLANs on a peer, first open a connection to `wss://192.0.2.5/vsx-peer/rest/v10.xx/notification` and then subscribe to `/rest/v10.04/system/vlans` as the topic name.

You can specify a specific resource instance or a collection of resources. Examples of specific resource instances:

- `/rest/v10.xx/system/vrfs/default`
- `/rest/v10.xx/system/vlans/1`

Examples of resource collections:

- `/rest/v10.xx/system/vrfs/default/bgp_routers`
- `/rest/v10.xx/system/vlans`

The `depth` query parameter is supported, with a maximum value of `2`, only with resource collections. For example:

- Correct: `/rest/v10.xx/system/vlans?depth=1`
- Incorrect: `/rest/v10.xx/system/vlans?depth=3.`

The `attributes` query parameter is supported. You can specify a comma-separated list of attribute names in the query string for either resource collections or resource instances. If attributes are specified, then the subscriber receives notification messages only when the value of one of the specified attributes changes. For example:

The following URI specifies the administrative state and link state of all interfaces on the switch:

```
/rest/v10.xx/system/interfaces?attributes=admin_state,link_state
```

The following URI specifies the names of the VLANs:

```
/rest/v10.xx/system/vlans?depth=2&attributes=name
```

The names of the attributes must match the names as documented in the AOS-CX REST API Reference for the GET method of the resource.

Notification security features

The notification feature uses secure WebSocket connections based on the TLS v1.2 protocol (Transport Layer Security version 1.2), which is the same protocol used for the REST HTTPS connections.

The switch uses self-signed certificates. To avoid certificate verification errors, disable certificate verification when establishing the connection.

AOS-CX real-time notifications subsystem reference summary

The following information is intended as a quick reference for experienced users. Values are not configurable unless noted otherwise.

Subtopics

Connection protocol

Port

Message format

Message types

Authorization

Notification resource URI

Session idle timeout

Session hard timeout

Subscription persistence

Configuration maximums

Connection protocol

WebSocket secure (`wss://`)

Port

443

Message format

JSON

Message types

The following are the supported message types:

- `subscribe`
- `unsubscribe`
- `success`
- `error`
- `notification`

Authorization

Session cookie from successful HTTPS login request

Notification resource URI

```
wss://<IP-ADDR>/rest/v10.xx/notification
```

<IP-ADDR> is the IP address of the switch.

For example:

```
wss://192.0.2.5/rest/v10.xx/notification
```

Session idle timeout

None

Session hard timeout

None

Subscription persistence

Subscriptions are active only while the WebSocket secure connection is open.

Configuration maximums

- Maximum number of subscribers per switch: 50
- Maximum number of topics in one subscription message: 2000

Enabling the notifications subsystem on a switch

The AOS-CX real-time notifications subsystem relies on the REST API, so the REST API must be enabled on the switch and VRF from which you want to receive notifications.

HTTPS server must be enabled on the specified VRF. The VRF you specify determines from which network the HTTPS server can be accessed. You can enable access on multiple VRFs, including user-defined VRFs.

Establishing a secure WebSocket connection through a web browser

Prerequisites

- Access to the switch REST API must be enabled. The REST API access mode can be either read-only or read/write.
- The web browser you use must support the secure WebSocket Protocol.

Procedure

1. Open a web browser page and log in to the switch Web UI or the REST API.
The session cookie is managed by the browser and is shared among browser tabs.
2. From a different tab in the same browser, open the page that contains the WebSocket interface.

For example, many browsers have a plugin for secure WebSocket connections.

3. Connect to the switch at the following URL:

`wss://<IP-ADDR>/rest/v10.xx/notification`

<IP-ADDR> is the IP address of the switch.

For example:

`wss://192.0.2.5/rest/v10.xx/notification`

After the connection is established, you can use the interface to send subscribe or unsubscribe messages and to view the responses and notification messages.

Establishing a secure WebSocket connection using a script

Access to the switch REST API must be enabled. The REST API access mode can be either read-only or read/write.

- If you are using a script, you must include the actions to log in, get the session cookie, store the session cookie, and pass the session cookie with the secure WebSocket connection request.

When you create the secure WebSocket connection, use the following URL:

`wss://<IP-ADDR>/rest/v10.xx/notification`

<IP-ADDR> is the IP address of the switch.

For example:

`wss://192.0.2.5/rest/v10.xx/notification`

- The exact methods to use to create connections and handle notification messages depend on the scripting language and library module you choose.

Subscribing to topics

Prerequisites

- You must have a secure WebSocket connection to the switch.
- Access to the switch REST API must be enabled. The REST API access mode can be either read-only or read/write.

Procedure

Using the WebSocket secure connection, send a subscribe message that contains the topics to which you want to subscribe.

Some resource attributes—typically in the statistics category—are not populated until a client requests the information.

For example:

```
{
  "type": "subscribe",
  "topics": [
    {
      "name": "/rest/v10.04/system/vrfs"
    },
    {
      "name": "/rest/v10.04/system/vlans/1?
attributes=admin,oper_state_reason"
    }
  ]
}
```

If there is an error in the syntax of the subscribe message, an error message is sent back to the client with the description of the error. For example, for the following incorrect subscribe message:

```
...
{"topics":[{"name":"/rest/v10.04/system/vrfssss"}],"type":"subscribe"}
...
```

The corresponding error message is sent:

```
{
  "type":"error",
  "message": "resource or attribute vrfssss not found",
  "data": null
}
```

If the subscriber already has a subscription to the specified topic, the following error is returned:

```
{
  "type":"error",
  "message":"The topic or combination of topics have been already
subscribed."
}
```

Example of a message returned by a successful subscription attempt:

```
{
  "type": "success",
  "data": [
    {
      "topicname": "/rest/<version>/system/vrfs",
```

```

    "sequence_number": 1,
    "resources": [
      {
        "operation": "",
        "uri": "/rest/<version>/system/vrfs/default",
        "values": {}
      },
      {
        "operation": "",
        "uri": "/rest/<version>/system/vrfs/mgmt",
        "values": {}
      }
    ],
    "subscription_id": "3530620397081182061"
  },
  {
    "topicname": "/rest/<version>/system/vlans/1?
attributes=admin,oper_state_reason",
    "sequence_number": 1,
    "resources": [
      {
        "operation": "",
        "uri": "/rest/<version>/system/vlans/1",
        "values": {
          "admin": "up",
          "oper_state_reason": "no_member_port"
        }
      }
    ]
  },

```

Unsubscribing from topics

Prerequisites

- You must have a secure WebSocket connection to the switch.
- The switch must have REST API access enabled. The REST API access mode can be either read-only or read/write.

Procedure

Use the secure WebSocket connection to send an unsubscribe message that specifies the topic or topics from which you no longer want notifications.

Use a comma to separate topics in a list of topics.

You must be connected as the same subscriber that subscribed to the topic. For example, you must be using the same web browser session or be passing the same session cookie with the request.

For example, to unsubscribe notifications about the default VRF, send the following message through the WebSocket secure connection:

```
{
  "type": "unsubscribe",
  "topics": [
    {
      "name": "/rest/v10.04/system/vrfs/default"
    }
  ]
}
```

If the subscriber does not have a subscription to that topic, the following message is returned:

```
{
  "type": "error",
  "message": "subscription /rest/v10.04/system/vrfs doesn't exist"
  "data": null
}
```

The error can indicate that you have already unsubscribed, the connection was lost, or you attempted to unsubscribe from a different subscriber.

If the request is successful, the following message is returned:

```
{
  "type": "success",
  "message": "Successfully unsubscribe."
}
```

Subscription throttling

Throttling is an optional parameter that can be passed in the subscription message to specify the notification interval in seconds. Notifications are only sent if there are any changes during the specified interval of time.

Showing a subscription for all VLANs with an interval of 5 seconds:

```
{
  "type": "subscribe",
  "interval": 5,
  "topics": [
    {
```

```
"name": "/rest/v10.04/system/vlans?depth=2"
}
]
}
```

With the throttling above, the system will send notifications for all VLANs every 5 seconds if there are any changes to the VLANs.

In [Figure 1](#), the system sent a notification because a change was made to the description.

Figure 1. Subscription throttling notifications

```
{
  "type": "notification",
  "data": [
    {
      "topicname": "/rest/latest/system/vlans?depth=2",
      "resources": [
        {
          "operation": "modified",
          "uri": "/rest/latest/system/vlans/100",
          "values": {
            "description": "hello"
          }
        }
      ],
      "subscription_id": "17922661235713176305",
      "sequence_number": 3
    }
  ]
}
```

Subscription throttling can also be used to handle notifications for resource attributes that only provide interval-based notifications, known as on-demand attributes.

Showing a subscription for an on-demand attributes with an interval of 10 seconds:

```
{
  "type": "subscribe",
  "interval": 10,
  "topics": [
    {
      "name": "/rest/v10.04/system/interfaces/1%2F14?attributes=aclv4_out_
statistics,policy_out_statistics"
    }
  ]
}
```

In [Figure 2](#), the system sent a notification for the on-demand attributes during the interval specified in the example above.

Figure 2. Subscription throttling notifications for on-demand attributes

```
{
  "type": "notification",
  "data": [
    {
      "topicname": "/rest/latest/system/interfaces/1%2F1%2F14?attributes=aclv4_out_statistics,policy_out_statistics",
      "resources": [
        {
          "operation": "modified",
          "uri": "/rest/latest/system/interfaces/1%2F1%2F14",
          "values": {
            "aclv4_out_statistics": {
              "10": 179793815
            },
            "policy_out_statistics": {
              "1": 179298641
            }
          }
        }
      ]
    },
    "subscription_id": "14179622499199942155",
    "sequence number": 2
  ]
}
```

Parts of a subscribe message

A subscribe message is the message sent when a subscriber requests a subscription to a topic on a switch. The subscribe message is in JSON format.

Subtopics

Subscribe message example

Components of a subscribe message

Subscribe message example

```
{
  "type": "subscribe",
  "topics": [
    {
      "name": "/rest/v10.04/system/vrfs"
```

```

    },
    {
      "name": "/rest/v10.04/system/vlans/1?
attributes=admin,oper_state_reason"
    }
  ]
}

```

Components of a subscribe message

type

Required. For a subscribe message, you must specify the following value: `subscribe`

topics

Required. The value is a comma-separated list of one or more topics in JSON key-value format. A topic includes one component:

name

Required. The name of the topic, identified by the URI of the switch resource, including the optional query string.

Parts of a subscription success message

When a subscription request is successful, a subscription success message is returned. The subscription success message is in JSON format.

Subtopics

Example success message

Components of subscription success message

Components of a topic

Example success message

```

{
  "type": "success",
  "data": [
    {
      "topicname": "/rest/<version>/system/vrfs",
      "sequence_number": 1,

```

```

    "resources": [
      {
        "operation": "",
        "uri": "/rest/<version>/system/vrfs/default",
        "values": {}
      },
      {
        "operation": "",
        "uri": "/rest/<version>/system/vrfs/mgmt",
        "values": {}
      }
    ],
    "subscription_id": "3530620397081182061"
  },
  {
    "topicname": "/rest/<version>/system/vlans/1?
attributes=admin,oper_state_reason",
    "sequence_number": 1,
    "resources": [
      {
        "operation": "",
        "uri": "/rest/<version>/system/vlans/1",
        "values": {
          "admin": "up",
          "oper_state_reason": "no_member_port"
        }
      }
    ],
    "subscription_id": "3530620397081182061"
  }
],
"subscriber_name": "6z6w4ckkw9f0"
}

```

Components of subscription success message

`type`

Identifies the type of message. Success messages have the type: `success`

`subscriber_name`

Contains a unique identifier that represents the name of the subscriber.

`data`

Contains a comma-separated list of one or more topics in JSON format.

Components of a topic

In a subscription success message, each topic in the data contains the following components:

`topicname`

Contains the name of the topic, identified by the URI of the switch resource, including the optional query string.

`resources`

Contains a comma-separated list of one or more resources in JSON format. When the URI of a topic is a resource collection, a topic includes multiple resources. In the example message, the `vrfs` resource includes two VRF instances: `default` and `mgmt`.

`subscription_id`

The string identifier for a group of topics and interval.

`sequence_number`

The number of notifications sent for a subscriber and topic name. The initial value is 1; then, for each notification, the sequence number increases by one. If one of the disconnection scenarios happens, the value of the current sequence number will be reset once the subscriber resubscribes.

`operation`

The value of `operation` is empty for success messages. This component is used for notification messages only.

`uri`

Contains the URI of the resource instance within the resource collection. If the `topicname` is a resource instance instead of a collection, `uri` matches the path portion of the URI in `topicname`.

`values`

Contains the names and current values of the attributes that were specified in the query string of `topicname`.

Parts of a notification message

A notification message is the message sent to the subscriber when there is a change to a switch resource that is the topic of a subscription. The notification message is in JSON format.

The content of a notification message depends on the type of change that occurred.

Subtopics

Notification message examples

Components of a notification message

Components of a topic

Notification message examples

For the following examples, assume that the following subscribe message was used:

```
{
  "type": "subscribe",
  "topics": [
    {
      "name": "/rest/v10.04/system/vlans?depth=2&attributes=name"
    }
  ]
}
```

The subscriber receives a notification when the name of any VLAN changes:

In the following example, VLAN7 has been added to the switch configuration:

```
{
  "type": "notification",
  "data": [
    {
      "topicname": "/rest/<version>/system/vlans?depth=2&attributes=name,oper_state",
      "sequence_number": 2,
      "resources": [
        {
          "operation": "inserted",
          "uri": "/rest/<version>/system/vlans/VLAN7",
          "values": {
            "name": "VLAN2",
            "oper_state": "down"
          }
        }
      ],
      "subscription_id": "7091641273267809475"
    }
  ]
}
```

```
]
}
```

In the following example, VLAN7 has been deleted from the configuration:

```
{
  "type": "notification",
  "data": [
    {
      "topicname": "/rest/<version>/system/vlans?
depth=2&attributes=name,oper_state",
      "sequence_number": 4,
      "resources": [
        {
          "operation": "deleted",
          "uri": "/rest/<version>/system/vlans/VLAN7",
          "values": {}
        }
      ],
      "subscription_id": "7091641273267809475"
    }
  ]
}
```

In the following example, the subscriber has subscribed to the following topic:

```
/rest/v10.xx/system/interfaces/1%2F1%2F2?attributes=name,admin_state
```

If either the name or the administrative state of VLAN **2** changes, a notification message is sent. If attributes other than name or administrative state changes, no notification message is sent.

In the following example, the administrative state of the interface changed to `up`.

```
{
  "type": "notification",
  "data": [
    {
      "topicname": "/rest/<version>/system/vlans?
depth=2&attributes=name,oper_state",
      "sequence_number": 3,
      "resources": [
        {
          "operation": "modified",
          "uri": "/rest/<version>/system/vlans/2",
          "values": {
            "oper_state": "up"
          }
        }
      ],
    }
  ],
}
```

```
    "subscription_id": "7091641273267809475"
  }
]
}
```

Components of a notification message

`type`

Identifies the type of message. Notification messages have the type: `notification`

`data`

Contains a comma-separated list of one or more topics in JSON format.

Components of a topic

In a notification message, each topic in the data contains the following components:

`topicname`

Contains the name of the topic, identified by the URI of the switch resource, including the optional query string.

`resources`

Contains a comma-separated list of one or more resources in JSON format. When the URI of a topic is a resource collection, a topic includes multiple resources. Each resource includes the following components:

`operation`

For notification messages, operation is one of the following values:

`inserted`

The resource or resource attribute was added to the configuration of the switch.

`deleted`

The resource or resource attribute was deleted from the switch.

`modified`

The resource or resource attribute changed.

`subscription_id`

The string identifier for a group of topics and interval.

`sequence_number`

The number of notifications sent for a subscriber and topic name. The initial value is 1; then, for each notification, the sequence number increases by one. If one of the disconnection scenarios happens, the value of the current sequence number will be reset once the subscriber resubscribes.

`uri`

Contains the URI of the resource instance within the resource collection. If the `topicname` is a resource instance instead of a collection, `uri` matches the path portion of the URI in `topicname`.

`values`

The content of `values` depends on the operation:

- When the `operation` value is `deleted`, `values` is empty.
- When the `operation` value is `inserted`, `values` contains the current names and values of the attributes specified in the query portion of the `topicname`. If no query string was included in `topicname`, all attributes and values for that resource are included.
- When the `operation` value is `modified`, `values` contains the name and current value of the attribute in the query string that changed value:
 - If no query string was included in `topicname`, all attributes and values for that resource are included.
 - If multiple attributes are included in the query string of a topic and only some of those attribute values changed, only the changed attributes are included.
 - If an attribute that was not included in the query string changes, no notification message is sent because that attribute is not part of the subscription.

Disconnection scenarios

The notification framework does not have a timeout mechanism like the API. But, there are some scenarios where all subscriptions for a subscriber or all subscribers could be lost.

AOS-CX will eliminate all subscriptions tied to a subscriber when there is a disconnection with the WebSocket channel.

All subscribers will need to recreate their subscriptions when:

- The switch is rebooted
- The REST daemon restarts
- A new version of firmware is installed
- A High Availability event occurs (Example, Failover)

- A hot patch is applied to the REST daemon

Example: Browser-based WebSocket connection

Subtopics

[About the example](#)

[Example screen](#)

[Example HTML source](#)

About the example

The following example, `websocket-client.html`, uses HTML and Javascript to create a webpage that you can use to establish a WSS connection and send and receive notification messages.

- Access to the switch REST API must be enabled on the VRF through which this browser will connect to the switch.
- Before you can use the HTML page, you must log in to the switch Web UI or REST API from a separate tab in the same web browser session. The browser shares the session cookie between tabs.
- When the browser page is open, in **Server Location**, substitute the switch IP address for `{IPAddresses}` in `wss://{IPAddresses}/rest/v10.xx/notification`, then click **Connect**.
- Enter the subscription message in **Request** and click **Send**.
- Responses and notifications are shown in **Response**.

Example screen

| | |
|-----------------|--|
| Server Location | |
| Connect | Disconnect wss://{IPAddress}/rest/{RestVersion}/notification |
| Request | |
| Send | |
| Response | |
| | |

Example HTML source

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Web Socket Client Example</title>
  <script type="text/javascript">
    window.onload = function () {
      var conn;
      var log = document.getElementById("log");
      var msg = document.getElementById("msg");
      function appendLog(item) {
        var doScroll = log.scrollTop === log.scrollHeight -
log.clientHeight;
        log.appendChild(item);
        if (doScroll) {
          log.scrollTop = log.scrollHeight - log.clientHeight;
        }
      }
      document.getElementById("connect").onclick = function () {
        var server = document.getElementById("wsURL");
        conn = new WebSocket(server.value);
        if (window["WebSocket"]) {
          if (conn) {
            conn.onopen = function (evt) {
              document.getElementById("disconnect").disabled
= false
              document.getElementById("sendMsg").disabled =
false
              document.getElementById("connect").disabled =
true
            }
          }
        }
      }
    }
  </script>
</head>
<body>
  <div>
    <div>Server Location</div>
    <div>
      <button>Connect</button>
      <input type="text" value="wss://{IPAddress}/rest/{RestVersion}/notification"/>
    </div>
    <div>Request</div>
    <div>
      <button>Send</button>
      <div></div>
    </div>
    <div>Response</div>
    <div></div>
  </div>
</body>
</html>
```

```

        document.getElementById("status").innerHTML =
"Connection opened"
    }
    conn.onclose = function (evt) {
        document.getElementById("status").innerHTML =
"Connection closed"
        document.getElementById("connect").disabled =
false

    };
    conn.onmessage = function (evt) {
        var messages = evt.data.split('\n');
        for (var i = 0; i < messages.length; i++) {
            var item = document.createElement("pre");
            item.innerText = messages[i];
            appendLog(item);
        }
    }
} else {
    var item = document.createElement("pre");
    item.innerHTML = "<b>Your browser does not support
WebSockets.</b>";
    appendLog(item);
}
};
document.getElementById("disconnect").onclick = function () {
    conn.close()
    document.getElementById("sendMsg").disabled = true
    document.getElementById("connect").disabled = false
    document.getElementById("disconnect").disabled = true
    document.getElementById("status").innerHTML = "Connection
closed"
};
document.getElementById("form").onsubmit = function () {
    if (!conn) {
        return false;
    }
    if (!msg.value) {
        return false;
    }
    conn.send(msg.value);
    var item = document.createElement("pre");
    item.classList.add("subscribeMsg");

```

```

        item.innerHTML = msg.value;
        appendLog(item);
        return false;
    };
};
</script>
<style type="text/css">
    html {
        overflow: hidden;
    }
    body {
        overflow: hidden;
        padding: 0;
        margin: 0;
        width: 100%;
        height: 100%;
        background: gray;
    }
    #log {
        background: white;
        margin: 0;
        padding: 0.5em 0.5em 0.5em 0.5em;
        top: 1.5em;
        left: 0.5em;
        right: 0.5em;
        bottom: 3em;
        overflow: auto;
        position: absolute;
        height: 530px;
    }
    #form {
        padding: 0 0.5em 0 0.5em;
        margin: 0;
        position: absolute;
        bottom: 3em;
        top: 5em;
        left: 8px;
        width: 100%;
        overflow: hidden;
    }
    #serverLocation {
        padding-top: 0.3em;
    }

```

```

        #requestSection {
            height: 38px;
        }
        #responseMsgSection {
            height: 570px;
            position: relative;
        }
    </style>
</head>
<body>
<fieldset>
    <legend>Server Location</legend>
    <div>
        <input type="button" value="Connect"/>
        <input type="button" value="Disconnect" disabled/>
        <input type="text" value="wss://{IPAddress}/rest/v10.04/
notification" size="64">
        <span></span>
    </div>
</fieldset>
<fieldset>
    <legend>Request</legend>
    <form>
        <input type="submit" value="Send" ; disabled/>
        <input type="text" size="80"/>
    </form>
</fieldset>
<fieldset>
    <legend>Response</legend>
    <div></div>
</fieldset>
</body>
</html>

```

Example: Getting information about current subscribers

To get information about the subscribers receiving notifications from a switch, you must use the REST API.

Instructions and examples in this document use an IP address that is reserved for documentation, 192.0.2.5, as an example of the IP address for the switch. To access your switch, you must use the IP address or hostname of that switch.

Prerequisites

You must be logged in to the switch REST API.

Procedure

To get the list of current subscribers, send a GET request to the `notification_subscribers` resource.

For example:

```
GET "https://192.0.2.5/rest/v10.xx/system/notification_subscribers"
```

The response body is a list of URIs. The identifier at the end of the URI string is the subscriber name.

For example:

```
[  
  "rest/v10.xx/system/notification_subscribers/z6901beisjgf",  
  "rest/v10.xx/system/notification_subscribers/1819g87erb42"  
]
```

Live Switch Events

Message-based notifications provide an additional data source available on the system. They are intended to be used through subscriptions instead of polling them at fixed intervals, improving resource utilization (CPU and RAM). This allows subscriptions to data coming through the messaging subsystem. This data source typically does not overlap with the resources found under the **/system** URI prefix.

These notifications work the same way as regular notifications. After establishing a websocket connection, a client can send a subscribe message through the websocket channel. In order to stop receiving notification messages, clients should send an unsubscribe message for the topic(s) they were subscribed to.



NOTE

Currently supported resources: `/rest/\<version\>/events`

Examples

After establishing a websocket connection, a client can send a "subscribe" message through the websocket channel. A valid subscribe message contains the following keys:

"type": represents the type of message being sent (subscribe or unsubscribe).

"topics": list of topics in which the client is interested in. Each object contains a name key with the URI path and query parameters that represents the topic.

This an example of a valid **subscribe** message:

```
{  
  "type": "subscribe",  
  "topics": [  
    {  
      "name": "/rest/v10.xx/system/notification_subscribers/z6901beisjgf"  
    },  
    {  
      "name": "/rest/v10.xx/system/notification_subscribers/1819g87erb42"  
    }  
  ]  
}
```

```

    "topics": [
      {
        "name": "/rest/latest/events?filter=eventid:112|111|102|101|5208|5209,priority:2,limit:1000"
      }
    ],
    "interval": 10
  }

```

This is an example of a success message response for the previous **subscribe** message:

```

{
  "type": "success",
  "data": [
    {
      "topicname": "/rest/events?filter=eventid:112|111|102|101|5208|5209,priority:2,limit:1000",
      "resources": [],
      "subscription_id": "15346667474612266703",
      "sequence_number": 1
    }
  ],
  "subscriber_name": "gisbittk9eh2",
  "code": 201
}

```

This is an example of an invalid **subscribe** message (notice the interval value):

```

{
  "type": "subscribe",
  "topics": [
    {
      "name": "/rest/latest/events?filter=eventid:112|111|102|101|5208|5209,priority:2,limit:1000"
    }
  ],
  "interval": -10
}

```

This is an example of an error message response for the previous invalid **subscribe** message:

```

{
  "type": "error",
  "message": "Invalid interval value: -10",
  "data": [
    {
      "topicname": "",
      "resources": null,
    }
  ]
}

```

```

        "subscription_id": "",
        "sequence_number": 0
    }
],
"code": 400
}

```

Subtopics

Notification Messages

Unsubscription

Limitations

Notification Messages

After the client subscribes to a topic, it starts receiving notification messages. These are all treated as **inserted** operations.

The following example shows the message format for those operations:

```

{
  "type": "notification",
  "data": [
    {
      "topicname": "/rest/latest/events?filter=eventid:112|111|102|101|5208|5209,priority:2,limit:1000",
      "resources": [
        {
          "operation": "inserted",
          "uri": "",
          "values": {
            "MODULE_ID": "-",
            "MODULE_ROLE": "AMM",
            "Message": "Event|5208|LOG_ERR|AMM|-|Failed to enable SSH server on VRF Bharath. Admin password is not set.",
            "OPS_EVENT_CATEGORY": "LLDP",
            "OPS_EVENT_ID": "5208",
            "PRIORITY": "6",
            "_HOSTNAME": "switch",
            "_PID": "2084",
            "__REALTIME_TIMESTAMP": ""
          }
        }
      ],
    },
  ],
}

```

```

{
  "operation": "inserted",
  "uri": "",
  "values": {
    "MODULE_ID": "-",
    "MODULE_ROLE": "AMM",
    "Message": "Event|101|LOG_INFO|AMM|-|LLDP Enabled",
    "OPS_EVENT_CATEGORY": "LLDP",
    "OPS_EVENT_ID": "5208",
    "PRIORITY": "6",
    "_HOSTNAME": "switch",
    "_PID": "2084",
    "__REALTIME_TIMESTAMP": ""
  }
},
{
  "operation": "inserted",
  "uri": "",
  "values": {
    "MODULE_ID": "-",
    "MODULE_ROLE": "AMM",
    "Message": "Event|102|LOG_INFO|AMM|-|LLDP Disabled",
    "OPS_EVENT_CATEGORY": "LLDP",
    "OPS_EVENT_ID": "5208",
    "PRIORITY": "6",
    "_HOSTNAME": "switch",
    "_PID": "2084",
    "__REALTIME_TIMESTAMP": ""
  }
},
{
  "operation": "inserted",
  "uri": "",
  "values": {
    "MODULE_ID": "-",
    "MODULE_ROLE": "AMM",
    "Message": "Event|111|LOG_INFO|AMM|-|LLDP statistics cleared",
    "OPS_EVENT_CATEGORY": "LLDP",
    "OPS_EVENT_ID": "5208",
    "PRIORITY": "6",
    "_HOSTNAME": "switch",
    "_PID": "2084",
    "__REALTIME_TIMESTAMP": ""
  }
}

```

```

        }
    }
    ],
    "subscription_id": "13024811290814909966",
    "sequence_number": 1
}
],
"subscriber_name": "7z2ggpajqle7",
"code": 200,
}

```

Unsubscription

In order to stop receiving notification messages, the client should send an unsubscribe message for the topic(s) he was subscribed before.

Here is an example of **unsubscribe** message:

```

{
  "type": "unsubscribe",
  "topics": [
    {
      "name": "/rest/latest/events?filter=eventid:112|111|102|101|5208|5209,priority:2,limit:1000"
    }
  ]
}

```

On successful unsubscription, the client receives following message:

```

{
  "type": "success",
  "code": 200,
  "message": "Successfully unsubscribed.",
  "data": [
    {
      "topicname": "",
      "resources": null,
      "subscription_id": "",
      "sequence_number": 0
    }
  ]
}

```

Limitations

- There are no real time (zero interval) event notifications.
- There is no data coalescing in REST, meaning data will not be grouped together and all changes for a given resource will be notified separately.
- Just as regular notification subscriptions, all data will be lost in case of a daemon restart. This includes operations as VSF switchover, hot patching, ISSU or a REST daemon crash.
- Different data backends on the same subscription request are not allowed. For example: **/rest/v10.04/system** and **/rest/v10.04/events**.
- These subscriptions are not dynamic, that means they are not subscriptions with auto-cancellation.
- Even though there is no versioning for these endpoints, a version needs to be added to the topic when subscribing.
- There are no segmented notifications for these resources.
- The user can subscribe to only one topic at a time per subscription request.

Firewall Mode Strict



NOTE

This feature is only supported by 10000 Switch series. For CLI configuration details, please refer to the Security Guide.

Subtopics

Overview

Configuring System Firewall Mode

Configure VLAN Firewall Mode Strict Bypass

Show Firewall Mode Status

Overview

In order to drop traffic users are required to configure policies. This feature enables users to force all VLAN or VRF traffic to be implicitly dropped, and can bypass the **strict** firewall mode with a bypass action on a given VLAN/VRF. When enabled, traffic that is not explicitly permitted by policy rules is dropped. When not enabled, traffic is forwarded unless explicitly blocked by a policy. Strict mode is disabled by default and

applies to all IPv4, IPv6, unicast, and multicast traffic on all VLANs and VRFs. Exceptions can be configured for specific VLANs and VRFs used for network underlay or other traffic that is allowed to bypass strict mode policy.

The unicast handling for the Drop/Permit is handled via service redirect. The Unicast Handling will be toggled via the **firewall_mode** columns in the System/VLAN/VRF tables. Once this is set, the action taken on the unicast traffic will be handled via the **service_redirect** column.



NOTE

Route-only ports, subinterfaces, and the **mgmt** VRF are not subject to strict mode and are always in bypass mode. VLAN 1 and the default VRF that are frequently used for network underlay traffic. They are subject to strict mode by default and will drop all traffic if not configured for **strict-bypass** or with a policy that permits underlay not configured for strict-bypass or with a policy that permits underlay traffic.



NOTE

The **show Firewall mode status** command may not reflect the unicast traffic action, since this applies multicast traffic. For the unicast based traffic, please review the **service_redirect** column for the status and/or action.

Configuring System Firewall Mode

Request URL: `https://<ip>/rest/{version}/system/firewall_config`

Enable firewall mode strict

```
curl -k --no-proxy <ip> -X PUT --header 'Content-Type: application/json' --
header 'Accept: application/json' -d '{
"firewall_mode": "strict"
}' 'https://<ip>/rest/{version}/system/firewall_config'
```

Disable firewall mode strict

```
curl -k --no-proxy <ip> -X PUT --header 'Content-Type: application/json' --
header 'Accept: application/json' -d '{
"firewall_mode": "disabled"
}' 'https://<ip>/rest/{version}/system/firewall_config'
```

Example Response:

```
{
  "10": {
    "id": 10,
    "name": "VLAN10",
    "firewall_mode": "strict-bypass",
```

```

    "firewall_mode_status": "enabled"
  },
  "20": {
    "id": 20,
    "name": "VLAN20",
    "firewall_mode": "none",
    "firewall_mode_status": "disabled"
  }
}

```

Configure VLAN Firewall Mode Strict Bypass

Request URL: https://<ip>/rest/{version}/system/vlans/{vlan_id}

Enable strict-bypass on VLAN 20

```

curl -k --no-proxy <ip> -X PUT --header 'Content-Type: application/json' --
header 'Accept: application/json' -d '{
"firewall_mode": "strict-bypass"
}' 'https://<ip>/rest/{version}/system/vlans/20'

```

Disable strict-bypass on VLAN 20

```

curl -k --no-proxy <ip> -X PUT --header 'Content-Type: application/json' --
header 'Accept: application/json' -d '{
"firewall_mode": "none"
}' 'https://<ip>/rest/{version}/system/vlans/20'

```

Example Response:

```

{
  "vrf1": {
    "name": "vrf1",
    "firewall_mode": "strict-bypass",
    "firewall_mode_status": "enabled"
  },
  "vrf2": {
    "name": "vrf2",
    "firewall_mode": "none",
    "firewall_mode_status": "disabled"
  }
}

```

Show Firewall Mode Status

Request URLs:

- https://<ip>/rest/{version}/system/firewall_config
- https://<ip>/rest/{version}/system/vlans?attributes=id,name,firewall_mode,firewall_mode_status
- https://<ip>/rest/{version}/system/vrfs?attributes=name,firewall_mode,firewall_mode_status

Get system firewall mode configuration

```
curl -X GET --header 'Accept: application/json' 'https://<ip>/rest/{version}/system/firewall_mode'
```

Get VLANs with firewall strict bypass configuration

```
curl -X GET --header 'Accept: application/json' 'https://<ip>/rest/{version}/system/vlans?attributes=id,name,firewall_mode,firewall_mode_status'
```

Get VRFs with firewall strict bypass configuration

```
curl -X GET --header 'Accept: application/json' 'https://<ip>/rest/{version}/system/vrfs?attributes=name,firewall_mode,firewall_mode_status'
```

Troubleshooting

Subtopics

General troubleshooting tips

REST API response codes

Error "'admin' password is not set"

Error "certificate verify failed" returned from curl command

HTTP 400 error "Invalid Operation"

HTTP 400 error "Value is not configurable" or "Bad Request"

HTTP 401 error "Authorization Required"

HTTP 401 error "Login failed: session limit reached"

HTTP 403 error "Forbidden" on a write request

HTTP 403 error "Forbidden" on a GET request

HTTP 404 error "Page not found" when accessing the switch URL

HTTP 404 error "Object not found" on object with "ports/" or "interfaces/" in URI Path

HTTP 404 error "Object not found" returned from a switch that supports multiple REST API versions (10.04 and later)

HTTP 404 error "Object not found" when using a write method

HTTP 404 error "Page not found" when using a write method

Logout Fails

General troubleshooting tips

Subtopics

Connectivity

Resources, attributes, and behaviors

GET, PUT, PATCH, POST, and DELETE methods

Hardware and other features

Connectivity

Connectivity is often the first issue you encounter. Ensure that you have enabled https-server on the VRF you are trying to use.

- To connect to the REST API through the management (OOBM) port, REST API access must be enabled on the management VRF.
- To connect to the REST API through a data port, REST API access must be enabled on the default VRF or a user-created VRF that includes that data port.

Resources, attributes, and behaviors

Resources, attributes, and behaviors might differ between different versions of the switch software.

If you are getting errors when making requests to switches with different software versions, use the AOS-CX REST API Reference on each switch to compare the URI paths and attributes for the resource. You might need to alter your code to handle the different software versions.

Resources, attributes, and behaviors might differ between different versions of the REST API, and the switch supports access through multiple versions of the REST API.

OSPF and BGP routing information updates frequently due to the nature of these resources. Best practices is to avoid using the REST API to view information about these resources, as this will trigger a large number of insert event notifications.



NOTE

The REST API supports Aruba clients such as Central, NetEdit, the AOS-CX WebUI and Network Analytics Engine (NAE), and Aruba Fabric Composer (AFC). Each of these clients makes use of REST through polling or subscriptions, where different requests are made to show the updated data to the user. Although it is possible to use one or more of these clients simultaneously, it is not recommended to have more than one of the clients connected at the same time, this can will cause high CPU and memory usage.

GET, PUT, PATCH, POST, and DELETE methods

Most resources do not allow POST, PATCH, PUT, or DELETE methods and do not display those methods in the AOS-CX REST API Reference unless the REST access mode is set to `read-write`.

The JSON model of a resource can vary by method used. The JSON data you receive from the GET method is not the same as the JSON data you can or must provide with the POST or PUT methods:

- The GET method model contains all the attributes.
- The POST method model contains only the configuration attributes.
- The PATCH method updates values in an existing resource using only the desired values in the request body.
- The PUT method model contains only the **mutable** (changeable) configuration attributes. If you do not provide all the mutable attributes in the request body of the PUT request, those attributes you do not provide are set to their defaults, which could be empty. If you attempt to provide an immutable attribute in a PUT request, an error is returned.

Use the GET method with the `selector=configuration` parameter to get only the configuration attributes of a resource. Using the REST v10.04 API and later, you can also use the GET method with the `selector=writable` parameter to get only the mutable configuration attributes of a resource.

You can use the AOS-CX REST API Reference to view information about the supported methods and resource models. You can obtain additional platform-specific information through GET requests for product information attributes or subsystem collections.

HPE Aruba Networking 8400 switch examples:

Example request:

```
GET "https://192.0.2.5/rest/v10.xx/system/subsystems"
```

Example response body:

```
{
  "chassis,1": "/rest/v10.04/system/subsystems/chassis,1",
  "line_card,1/3": "/rest/v10.04/system/subsystems/line_card,1%2F3",
  "management_module,1/5": "/rest/v10.04/system/subsystems/
management_module,1%2F5"
}
```

Example request:

```
GET "https://192.0.2.5/rest/v10.xx/system/subsystems/chassis,1?
attributes=product_info"
```

Example response body:

```
{
  "product_info": {
    "base_mac_address": "00:00:5E:00:53:00",
    "device_version": "",
    "instance": "1",
    "number_of_macs": "512",
    "part_number": "JL375A",
    "product_description": "8400 8-slot Chassis/3xFan Trays/18xFans/Cable
Manager/X462 Bundle",
    "product_name": "8400 Base Chassis/3xFT/18xFans/Cbl Mgr/X462 Bundle",
    "serial_number": "SG00A2A00A",
    "vendor": "Aruba"
  }
}
```

HPE Aruba Networking 8320 switch examples:

Example request:

```
GET "https://192.0.2.5/rest/v10.xx/system/subsystems"
```

Example response body:

```
{
  "chassis,1": "/rest/v10.04/system/subsystems/chassis,1",
  "line_card,1/1": "/rest/v10.04/system/subsystems/line_card,1%2F1 ",
  "management_module,1/1": "/rest/v10.04/system/subsystems/
management_module,1%2F1"
}
```

Example request:

```
GET "https://192.0.2.5/rest/v10.xx/system/subsystems/chassis,1?
attributes=product_info"
```

Example response body:

```
{
  "product_info": {
```

```

    "base_mac_address": "00:00:5E:00:53:01",
    "device_version": "",
    "instance": "1",
    "number_of_macs": "74",
    "part_number": "JL479A",
    "product_description": "8320",
    "product_name": "8320",
    "serial_number": "TW00000000",
    "vendor": "Aruba"
  }
}

```

Hardware and other features

Different switches have different hardware and features. For example, the management module resource ID is 1/1 for some switches, and 1/4 or 1/5 for other switches. To get information about the switch model, use the GET method request with the URI for the **platform_name** system attribute.

For example:

```
GET "https://192.0.2.5/rest/v10.xx/system?attributes=platform_name"
```

The following is an example of a response body for an HPE Aruba Networking 8320 switch:

```

{
  "platform_name": "8320"
}

```

The following is an example of a response body for an Aruba 8400 switch:

```

{
  "platform_name": "8400X"
}

```

The words "port" and "interface" have meanings that are different from other network operating systems. In the AOS-CX operating system:

- A port is the logical representation of a port.
- An interface is the hardware representation of a port.

You can enable debugging logs by using the **debug** command. The module name is **rest**. You can specify all severity log levels or a minimum severity log level.

Example specifying all severity log levels:

```
switch# debug rest all
```

Example specifying a minimum severity log level of **error**:

```
switch# debug rest all severity error
```

REST API response codes

The following table describes the different categories of the response codes.

Category	Description
2xx	Indicates that the request was accepted successfully.
4xx	Returns the client - side error response with the error message.
5xx	Returns the server - side error response with the error message.

The following are some response codes that you will see in the REST API.

Response code	Status	Description
200	OK	Returned from GET and PUT operations, and non - configuration API calls such as Login or Logout when the request is successfully completed.
201	Created	Returned from POST operations when a new resource was successfully created.
204	No Content	Returned from a PUT, POST PATCH, or DELETE operation when the request was successfully processed and there is no content to return.
400	Bad request	A problem with the request body, such as invalid syntax, incorrectly formatted JSON, or data violating a database constraint.
401	Unauthorized	No active session for this client (the login API has not been called) or

Response code	Status	Description
		too many sessions already created from this client.
403	Forbidden	The client session is valid, but does not have permissions to access the requested resource.
404	Not found	The resource does not exist, or the URI is incorrect for the desired resource. Can also occur when accessing the POST, PUT, PATCH, or DELETE API while the REST access - mode is set to read - only.
500	Internal server error	An unexpected error has occurred in processing the request. View the logs on the device for details.
503	Service unavailable	The device is receiving more requests than it can process and is defensively rejecting requests to protect resources.

Error "'admin' password is not set"

Symptom

An attempt to enable the HTTPS server using the **https-server vrf** command fails and the following error is returned:

```
Failed to enable https-server on VRF <VRF>. 'admin' password is not set
```

Cause

The switch is shipped from the factory with a default user named `admin` without a password. The `admin` user must set a valid password before HTTPS servers can be enabled.

Action

From the global configuration context, set a valid password for the `admin` user.

For example:

```
switch(config)# user admin password
Changing password for user admin
```

```
Enter password:*****
Confirm password:*****
```

Error "certificate verify failed" returned from curl command

Symptom

A curl command to the switch URL fails with an error similar to the following:

```
SSL3_GET_SERVER_CERTIFICATE:certificate verify failed
```

Cause

The curl program could not verify the switch server certificate against the CA certificate bundle that comes with the curl installation, and you did not include the `-k` option in the curl command.

Action

Retry the command with the `-k` option included.

The switch HTTPS server uses self-signed certificates, which cannot be verified against a certificate authority. The `-k` option disables curl certificate validation.

For example:

```
$ curl -k --noproxy "192.0.2.5" GET /tmp/auth_cookie \
"https://192.0.2.5/rest/v10.09/system/vlans"
```

HTTP 400 error "Invalid Operation"

Symptom

A REST request returns response code 400 and the response body contains the following text string:

```
Invalid operation
```

Cause

The method used for this REST request is not supported for the specified resource. For example, the `Invalid operation` response is returned if you attempt a DELETE request on the `system` resource.

Action

Use a method supported by the resource.

The AOS-CX REST API Reference displays the methods supported by each resource.

HTTP 400 error "Value is not configurable" or "Bad Request"

Symptom

A PUT, PATCH, or POST request returns response code 400 and the response body contains the following text string:

```
Value <value> is not configurable
```

Cause

The JSON data in the POST, PATCH, or PUT request body contains non-configuration or immutable attributes.

Action

Retry the request with the correct JSON resource model for that PUT, PATCH, or POST method.

To determine the configuration attributes of a resource, you can send a GET request with the `selector=configuration` query parameter to the resource. Using the REST v10.04 API or later, you can also use the GET method with the `selector=writable` parameter to get only the mutable configuration attributes of the resource.

You can also use the AOS-CX REST API Reference to verify the JSON model of the PUT, PATCH, or POST method of the resource.

The category an attribute belongs to can depend on whether that instance of the resource is owned by the system or owned by a user. Configuration attributes can become status attributes in resource instances that are owned by the system. Status attributes can not be modified by users.

In addition, some configuration attributes cannot be changed after a resource is created. These immutable attributes cannot be included in a PUT request.

HTTP 401 error "Authorization Required"

Symptom

A REST request returns response code 401 and the response body contains the following text string:

```
Authorization Required
```

This response means that no valid session was found for the session token passed to the API.

Subtopics

Solution 1

Solution 2

Solution 1

Cause

The user attempting the request is not logged into the REST API for one of the following reasons:

- The user has not yet logged in.
- The user logged in but the session has expired.

Action

Log in to the REST API.

Solution 2

Cause

The user attempting the request is not logged in to the REST API because the user did not pass the correct session cookie to the API. Typically, incorrect session cookies are not a cause when accessing the REST API through a browser because the browser automatically handles the session cookie.

Action

1. Ensure that you save the session cookie returned from the login request.
2. Ensure that you pass the same cookie back to the switch with every REST API request, including the request to log out.

HTTP 401 error "Login failed: session limit reached"

Symptom

A REST request or Web UI login attempt returns response code 401 and the response body contains the following text string:

```
Login failed: session limit reached
```

Cause

A user attempted to log into the REST API or the Web UI, but that user already has the maximum number of concurrent sessions running.

Action

1. Log out from one of the existing sessions.
Browsers share a single session cookie across multiple tabs or even windows. However, scripts that POST to the login resource and later do not POST to the logout resource can easily create the maximum number of concurrent sessions.
2. If the session cookie is lost and it is not possible to log out of the session, then wait for the session idle time limit to expire.
When the session idle timeout expires, the session is terminated automatically.
3. If it is required to stop all HTTPS sessions on the switch instead of waiting for the session idle time limit to expire, you can stop all HTTPS sessions using the **https-server session close all** command.
This command stops and starts the hpe-restd service, so using this command affects all existing REST sessions and Web UI sessions.

HTTP 403 error "Forbidden" on a write request

Symptom

A POST, PUT, PATCH, or DELETE REST request returns response code 403 and the response body contains the following text string:

```
Forbidden
```

Cause

The user attempting the request is not a member of the `administrators` group.

Action

Log in to the REST API with a user name that has administrator rights as part of the `administrators` group.

The user must be a member of the predefined `administrators` group. POST requests to the `login` resource fail for members of a user-defined local user group.

HTTP 403 error "Forbidden" on a GET request

Symptom

A GET REST request returns response code 403 and the response body contains the following text string:

```
Forbidden
```

Cause

The user attempting the request is a member of the Auditors group, and the GET request specified a switch resource that users with auditor rights are not permitted to access.

Action

Log in to the REST API with a user name that has operator or administrator rights.

HTTP 404 error "Page not found" when accessing the switch URL

Symptom

The switch is operational and you are using the correct URL for the switch, but attempts to access the REST API or Web UI result in an HTTP 404 "Page not found" error.

Cause

REST API access is not enabled on the VRF that corresponds to the access port you are using. For example, you are attempting to access the REST API or Web UI from the management (OOBM) port, and access is not enabled on the `mgmt` VRF.

Action

Use the **`https-server vrf`** command to enable REST API access on the specified VRF.

For example:

```
switch(config) # https-server vrf mgmt
```

HTTP 404 error "Object not found" on object with "ports/" or "interfaces/" in URI Path

Symptom

A request was made with an URI that contains `rest/v10.xx/` and `ports/` or `interfaces/` in the URI path, and the request returns response code 404 and the response body contains the following text string:

```
Object not found
```

Cause

The resource does not exist in the system. The URI in the request is incorrect.

The `ports` collection does not exist in the REST v10.04 or later API schema.

Action

Change the request to a request that is valid for the REST v10.04 or later API.

HTTP 404 error "Object not found" returned from a switch that supports multiple REST API versions (10.04 and later)

Symptom

A switch that supports multiple REST API versions returns response code 404 and the response body contains the following text string:

```
Object not found
```

Cause

The resource does not exist in the system. The URI in the request is incorrect for the version of the REST API specified in the request.

Action

Verify the URI of the resource and retry the request.

HTTP 404 error "Object not found" when using a write method

Symptom

A PUT or DELETE request returns response code 404 and the response body contains the following text string:

```
Object not found
```

Cause

The resource does not exist in the system. The URI in the request is incorrect or the resource has not been added to the configuration.

Action

Verify the URI of the resource you are attempting to change or delete and retry the request.

HTTP 404 error "Page not found" when using a write method

Symptom

Using the GET method is successful, but attempting a POST, PUT, or DELETE method results in an HTTP 404 "Page not found" error.

Cause

The REST API access mode is set to `read-only`.

Action

Set the REST API access mode to `read-write`.

```
switch(config) # https-server rest access-mode read-write
```

Enabling the read-write mode on the REST API allows POST, PUT, and DELETE operations to be called on all configurable elements in the switch database.

Logout Fails

Symptom

An attempt to log out of the REST API from a script or curl command fails.

Cause

The session cookie was not supplied or does not contain the correct session token.

Action

1. Repeat the command and send the correct session cookie or modify the script to send the correct session cookie.
2. If the session cookie has been lost and it is not possible to log out of the session, wait for the session idle time limit to expire.
When the session idle timeout expires, the session is terminated automatically.

Support and Other Resources

Subtopics

[Accessing HPE Aruba Networking Support](#)

[Accessing Updates](#)

[Warranty Information](#)

[Regulatory Information](#)

Accessing HPE Aruba Networking Support

HPE Aruba Networking Support Services	https://www.hpe.com/us/en/networking/hpe - aruba - networking - support - services.html
AOS - CX Switch Software Documentation Portal	https://arubanetworking.hpe.com/techdocs/ArubaDocPortal/content/new-portal/aoscx.html
HPE Aruba Networking Support Portal	https://networkingsupport.hpe.com/home
North America telephone	1 - 800 - 943 - 4526 (US & Canada Toll - Free Number) +1 - 650 - 750 - 0350 (Backup—Toll Number)
International telephone	https://www.hpe.com/psnow/doc/a50011948enw

Be sure to collect the following information before contacting Support:

- Technical support registration number (if applicable)
- Product name, model or version, and serial number
- Operating system name and version
- Firmware version
- Error messages
- Product-specific reports and logs
- Add-on products or components
- Third-party products or components

Other useful sites

Other websites that can be used to find information:

HPE Aruba Networking Developer Hub	https://developer.arubanetworks.com/hpe - aruba - networking - aoscx/docs/about
Airheads social forums and Knowledge Base	https://community.arubanetworks.com/
AOS - CX Software Technical Update channel on YouTube.	Videos on new features introduced in this release : https://www.youtube.com/playlist?list=PLsYGHuNuBZcbWPEjjHuVMqP - Q_UL3CskS
HPE Aruba Networking Hardware Documentation and Translations Portal	
HPE Aruba Networking software	https://networkingsupport.hpe.com/downloads https://networkingsupport.hpe.com/downloads
Software licensing and Feature Packs	https://licensemanagement.hpe.com/
End - of - Life information	https://networkingsupport.hpe.com/end - of - life

Accessing Updates

You can access updates from the HPE Aruba Networking Support Portal at <https://networkingsupport.hpe.com>.

Some software products provide a mechanism for accessing software updates through the product interface. Review your product documentation to identify the recommended software update method.

To subscribe to eNewsletters and alerts:

<https://networkingsupport.hpe.com/notifications/subscriptions> (requires an active HPE Aruba Networking Support Portal account to manage subscriptions). Security notices are viewable without an HPE Aruba Networking Support Portal account.

Warranty Information

To view warranty information for your product, go to <https://www.arubanetworks.com/support-services/product-warranties/>.

Regulatory Information

To view the regulatory information for your product, view the Safety and Compliance Information for Server, Storage, Power, Networking, and Rack Products, available at [**https://www.hpe.com/support/Safety-Compliance-EnterpriseProducts**](https://www.hpe.com/support/Safety-Compliance-EnterpriseProducts)

Additional regulatory information

HPE Aruba Networking is committed to providing our customers with information about the chemical substances in our products as needed to comply with legal requirements, environmental data (company programs, product recycling, energy efficiency), and safety information and compliance data, (RoHS and WEEE). For more information, see [**https://www.arubanetworks.com/company/about-us/environmental-citizenship/**](https://www.arubanetworks.com/company/about-us/environmental-citizenship/).

Documentation Feedback

HPE Aruba Networking is committed to providing documentation that meets your needs. To help us improve the documentation, send any errors, suggestions, or comments to Documentation Feedback ([**docsfeedback-switching@hpe.com**](mailto:docsfeedback-switching@hpe.com)). When submitting your feedback, include the document title, part number, edition, and publication date located on the front cover of the document. For online help content, include the product name, product version, help edition, and publication date located on the legal notices page.